

Appendix I: ResNet50V2 Tests 4

In [1]: `!nvidia-smi`

```
Fri Mar 14 23:08:01 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan   Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                    |                      |
MIG M. |
|=====+=====+=====+=====+
=====|
|    0   NVIDIA A100-SXM4-40GB                Off | 00000000:00:04.0 Off |
0|
| N/A    32C    P0              48W / 400W |      0MiB / 40960MiB |      0%
Default |
|              |                    |                      |
Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   Us
age      |
|=====+=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!/pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`
`from tensorflow.keras.applications import ResNet50 # Access to pre-trained model`
`from tensorflow.keras import layers, models, optimizers # Access to building blo`

```

from tensorflow.keras.models import load_model # Ability to load best_saved mode
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis of model
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Function definitions

```

In [4]: def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224)) # Resize images
    x = tf.image.random_flip_left_right(x) # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2) # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2) # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32)) # One-hot encoding
    return x, y

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

```

```

# Plot Training and Validation Accuracy
plt.subplot(1, 2, 2)
plt.plot(epochs, train_acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()

```

Dropout(0.7), weight_decay(3e-1), l2(3e-1)

```

In [5]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")

        # Split temp data into equal validation (4000) and testing (4000) data
        X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
        print(f"X_temp_val.shape: {X_temp_val.shape}")
        print(f"y_temp_val.shape: {y_temp_val.shape}")
        print(f"X_temp_test.shape: {X_temp_test.shape}")
        print(f"y_temp_test.shape: {y_temp_test.shape}\n")

        # Split test data into validation (5000) and testing (5000)
        X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

        # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
        X_val = np.concatenate((X_val, X_temp_val), axis=0)
        y_val = np.concatenate((y_val, y_temp_val), axis=0)
        X_test = np.concatenate((X_test, X_temp_test), axis=0)
        y_test = np.concatenate((y_test, y_temp_test), axis=0)

        print(f"X_train.shape: {X_train.shape}")
        print(f"y_train.shape: {y_train.shape}")
        print(f"X_val.shape: {X_val.shape}")
        print(f"y_val.shape: {y_val.shape}")
        print(f"X_test.shape: {X_test.shape}")
        print(f"y_test.shape: {y_test.shape}\n")

        display_imgs(X_train, y_train)

        # Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
        X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
        display_imgs(X_train, y_train)

        labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                        'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                        'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                        'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                        'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo

```

```

        'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea',
        'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
        'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
        'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
        'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
        'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                    .map(augment_dataset, num_parallel_calls=tf.data.experimen
                    .batch(batch_size)
                    .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
              .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                               tf.squeeze(tf.one_hot(y, depth=100, dtype=
              .batch(batch_size)
              .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

```

```

#### <<<<<<<<<Pre-trained model>>>>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample)) # Should be <class 'tuple'>
    print(len(sample)) # Should be 2
    print(type(sample[0]), type(sample[1])) # Both should be <class 'tensorflow.Tensor'>
    print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'>
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir='./Logs')
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-
    loss='categorical_crossentropy',
    metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
    patience=5, # Num. epochs with no improvement
    restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#    factor=0.1, # Reduce Lr by a factor
#    patience=3, # Num epochs w/ no improvement
#    min_lr=1e-6, # Min Lr
#    verbose=1)
#tensorboard = TensorBoard(log_dir='./Logs', # Logs directory
#    histogram_freq=1, # Logs histograms for weights/acc

```

```

# write_graph=True, # Logs graph of model
# write_images=True) # Log images like weight histog
checkpoint = ModelCheckpoint('best_model_v2.keras', # Path where the best mo
    monitor='val_loss', # What to monitor (validati
    save_best_only=True, # Only save model when val
    mode='min', # Looking to minimise the validation
    verbose=1) # Print out message when saving mode
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v2.keras', # Pa
    monitor='val_loss', # What to monitor (validati
    save_best_only=True, # Only save model when val
    mode='min', # Looking to minimise the validation
    verbose=1) # Print out message when saving mode
#cv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
    batch_size=batch_size, callbacks=[early_stopping, checkp

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v2.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')

```

```

plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                          .batch(batch_size)
                          .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v2.keras')
results = model_fine_tune.evaluate(test_dataset)

```

```

test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437 ————— 13s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

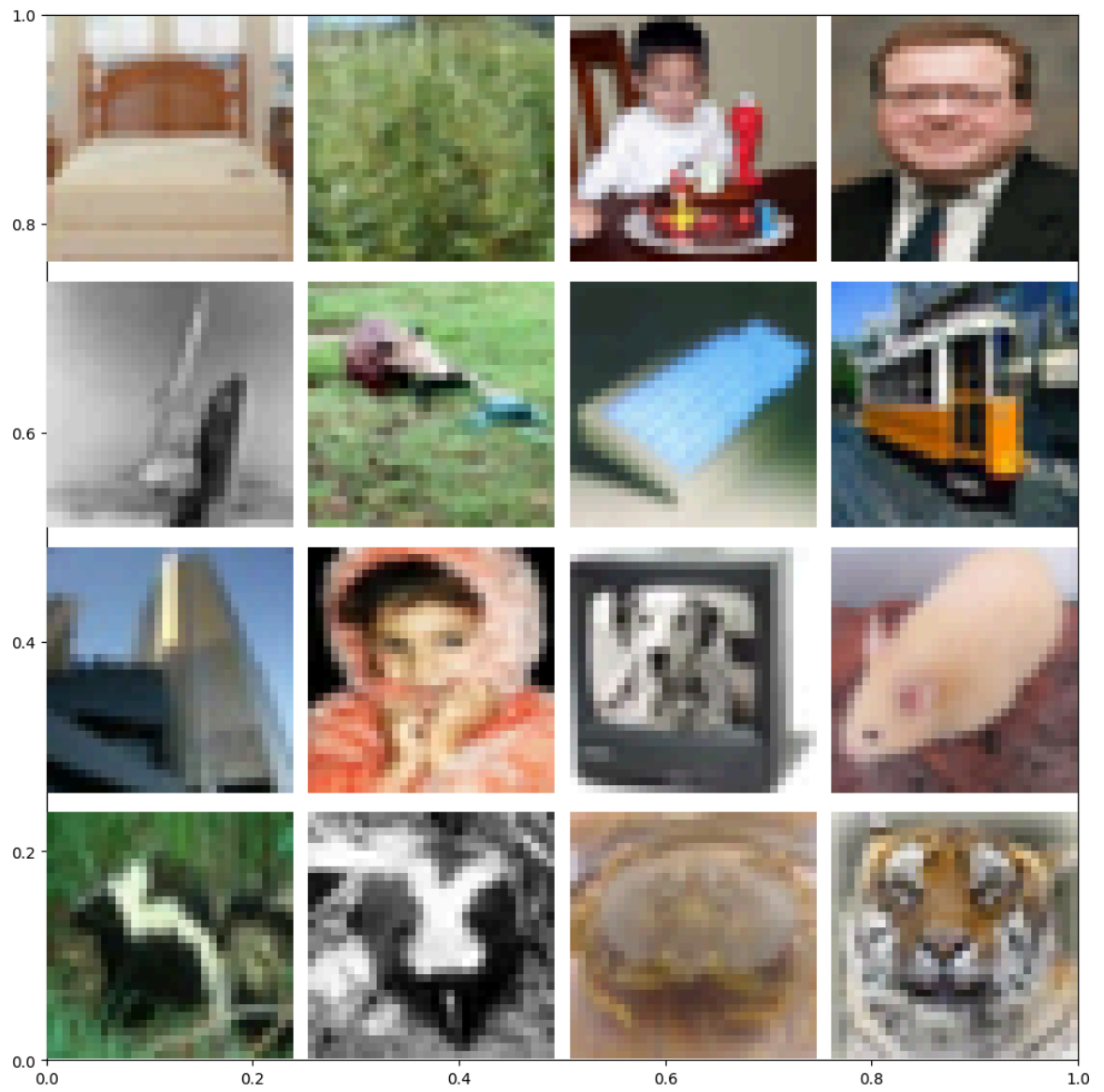
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

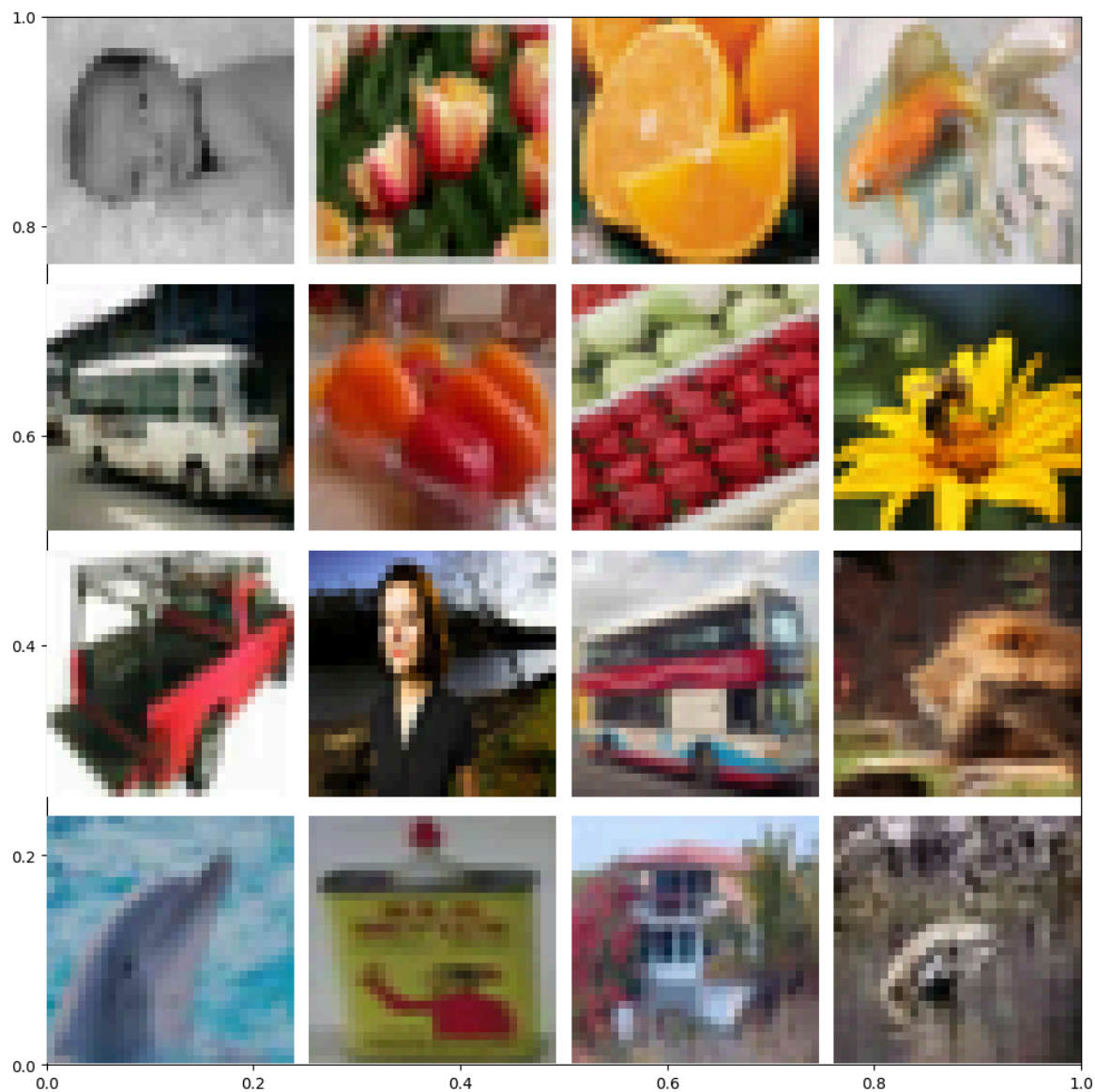
y_test.shape: (9000, 1)

labels[1070].shape: (1,)

imgs[1070].shape: (32, 32, 3)

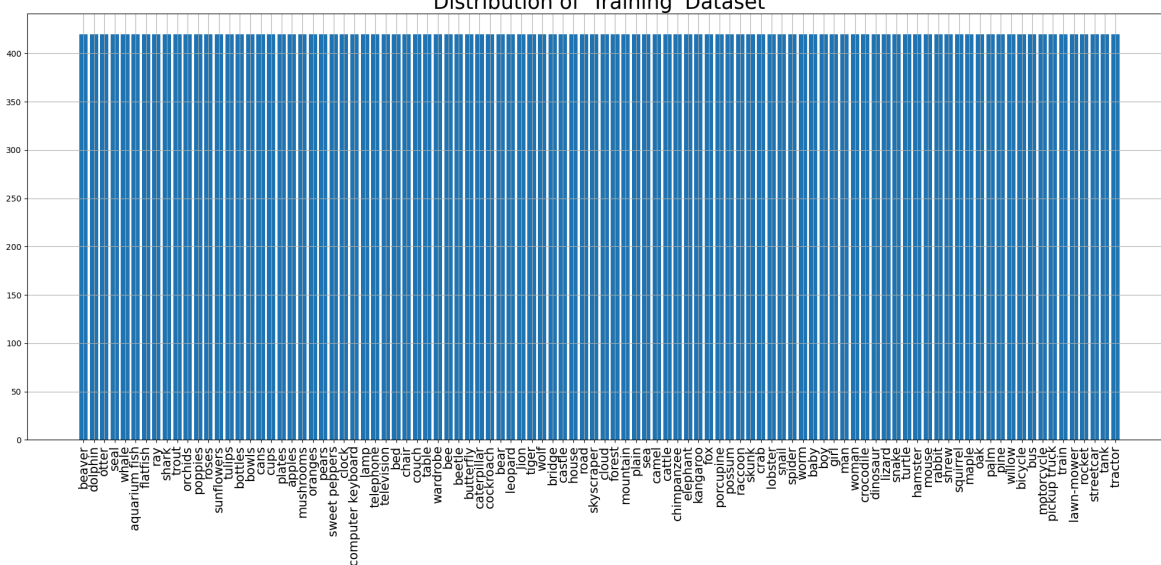


labels[36059].shape: (1,)
imgs[36059].shape: (32, 32, 3)

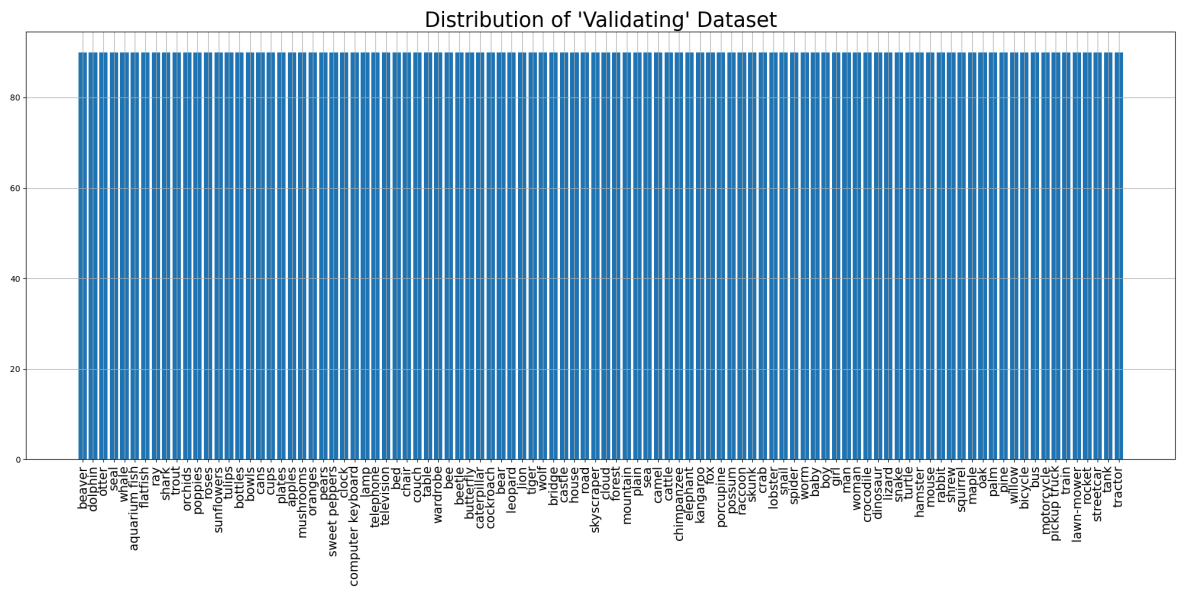


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

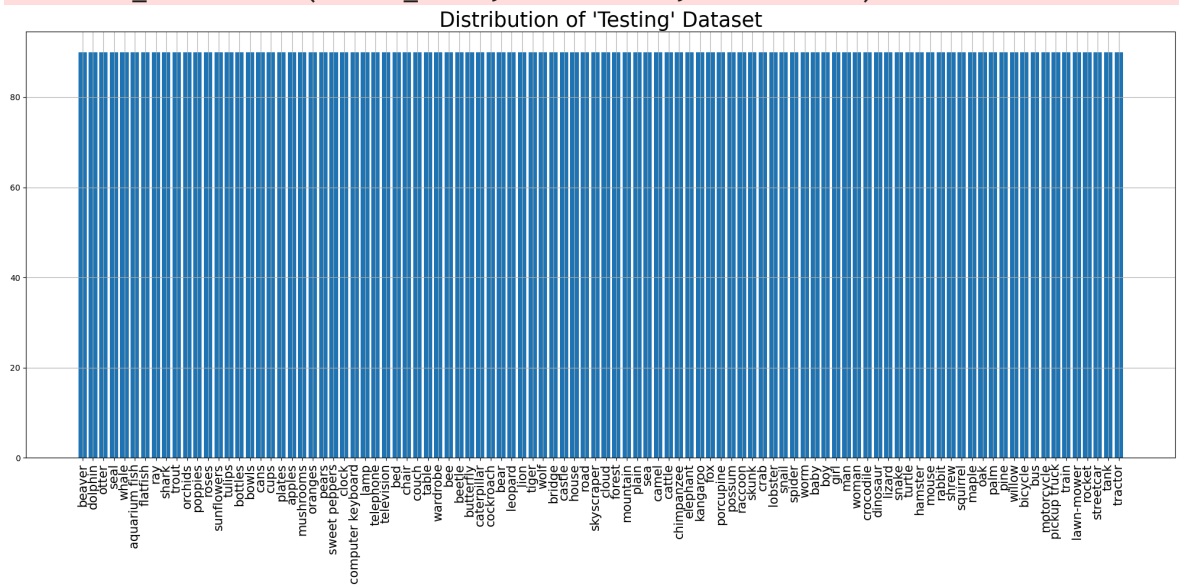
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5

94668760/94668760  **5s** 0us/step

<class 'tuple'>

2

<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

<class 'numpy.ndarray'> <class 'numpy.ndarray'>

(64, 224, 224, 3) (64, 100)

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 34ms/step - accuracy: 0.2654 - f1_score: 0.2594 - loss: 3.1989 - precision: 0.6251
Epoch 1: val_loss improved from inf to 1.58182, saving model to best_model_v2.keras as
657/657 ————— 56s 58ms/step - accuracy: 0.2656 - f1_score: 0.2595 - loss: 3.1979 - precision: 0.6252 - val_accuracy: 0.5690 - val_f1_score: 0.5634 - val_loss: 1.5818 - val_precision: 0.8273
Epoch 2/25
655/657 ————— 0s 23ms/step - accuracy: 0.4726 - f1_score: 0.4674 - loss: 1.9409 - precision: 0.7259
Epoch 2: val_loss improved from 1.58182 to 1.47065, saving model to best_model_v2.keras
657/657 ————— 19s 30ms/step - accuracy: 0.4727 - f1_score: 0.4674 - loss: 1.9406 - precision: 0.7259 - val_accuracy: 0.5936 - val_f1_score: 0.5904 - val_loss: 1.4706 - val_precision: 0.8419
Epoch 3/25
657/657 ————— 0s 23ms/step - accuracy: 0.5125 - f1_score: 0.5078 - loss: 1.7824 - precision: 0.7479
Epoch 3: val_loss improved from 1.47065 to 1.41093, saving model to best_model_v2.keras
657/657 ————— 19s 30ms/step - accuracy: 0.5125 - f1_score: 0.5078 - loss: 1.7823 - precision: 0.7479 - val_accuracy: 0.6036 - val_f1_score: 0.5990 - val_loss: 1.4109 - val_precision: 0.8222
Epoch 4/25
657/657 ————— 0s 23ms/step - accuracy: 0.5262 - f1_score: 0.5220 - loss: 1.7018 - precision: 0.7545
Epoch 4: val_loss improved from 1.41093 to 1.39254, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5262 - f1_score: 0.5221 - loss: 1.7018 - precision: 0.7546 - val_accuracy: 0.6086 - val_f1_score: 0.6053 - val_loss: 1.3925 - val_precision: 0.8307
Epoch 5/25
656/657 ————— 0s 23ms/step - accuracy: 0.5393 - f1_score: 0.5355 - loss: 1.6441 - precision: 0.7630
Epoch 5: val_loss improved from 1.39254 to 1.37253, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5393 - f1_score: 0.5355 - loss: 1.6440 - precision: 0.7630 - val_accuracy: 0.6184 - val_f1_score: 0.6138 - val_loss: 1.3725 - val_precision: 0.8384
Epoch 6/25
655/657 ————— 0s 23ms/step - accuracy: 0.5465 - f1_score: 0.5433 - loss: 1.6078 - precision: 0.7685
Epoch 6: val_loss improved from 1.37253 to 1.35086, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5465 - f1_score: 0.5433 - loss: 1.6078 - precision: 0.7685 - val_accuracy: 0.6206 - val_f1_score: 0.6174 - val_loss: 1.3509 - val_precision: 0.8281
Epoch 7/25
656/657 ————— 0s 23ms/step - accuracy: 0.5566 - f1_score: 0.5528 - loss: 1.5783 - precision: 0.7691
Epoch 7: val_loss improved from 1.35086 to 1.34855, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5567 - f1_score: 0.5528 - loss: 1.5783 - precision: 0.7691 - val_accuracy: 0.6171 - val_f1_score: 0.6146 - val_loss: 1.3486 - val_precision: 0.8219
Epoch 8/25
655/657 ————— 0s 23ms/step - accuracy: 0.5549 - f1_score: 0.5509 - loss: 1.5664 - precision: 0.7697
Epoch 8: val_loss did not improve from 1.34855

657/657 ————— **18s** 28ms/step - accuracy: 0.5549 - f1_score: 0.5509
- loss: 1.5663 - precision: 0.7698 - val_accuracy: 0.6206 - val_f1_score: 0.6179
- val_loss: 1.3531 - val_precision: 0.8195
Epoch 9/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5587 - f1_score: 0.5544 -
loss: 1.5487 - precision: 0.7722
Epoch 9: val_loss improved from 1.34855 to 1.33750, saving model to best_model_v
2.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5588 - f1_score: 0.5545
- loss: 1.5486 - precision: 0.7722 - val_accuracy: 0.6174 - val_f1_score: 0.6144
- val_loss: 1.3375 - val_precision: 0.8223
Epoch 10/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5682 - f1_score: 0.5647 -
loss: 1.5118 - precision: 0.7774
Epoch 10: val_loss improved from 1.33750 to 1.33651, saving model to best_model_v
2.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5682 - f1_score: 0.5647
- loss: 1.5118 - precision: 0.7774 - val_accuracy: 0.6213 - val_f1_score: 0.6197
- val_loss: 1.3365 - val_precision: 0.8249
Epoch 11/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5674 - f1_score: 0.5637 -
loss: 1.5117 - precision: 0.7742
Epoch 11: val_loss improved from 1.33651 to 1.32277, saving model to best_model_v
2.keras

657/657 ————— **20s** 30ms/step - accuracy: 0.5675 - f1_score: 0.5637
- loss: 1.5116 - precision: 0.7742 - val_accuracy: 0.6252 - val_f1_score: 0.6230
- val_loss: 1.3228 - val_precision: 0.8222
Epoch 12/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5730 - f1_score: 0.5692 -
loss: 1.4956 - precision: 0.7798
Epoch 12: val_loss did not improve from 1.32277

657/657 ————— **18s** 28ms/step - accuracy: 0.5730 - f1_score: 0.5693
- loss: 1.4956 - precision: 0.7798 - val_accuracy: 0.6253 - val_f1_score: 0.6230
- val_loss: 1.3373 - val_precision: 0.8231
Epoch 13/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5697 - f1_score: 0.5660 -
loss: 1.4995 - precision: 0.7710
Epoch 13: val_loss improved from 1.32277 to 1.32116, saving model to best_model_v
2.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5698 - f1_score: 0.5661
- loss: 1.4995 - precision: 0.7710 - val_accuracy: 0.6298 - val_f1_score: 0.6277
- val_loss: 1.3212 - val_precision: 0.8261
Epoch 14/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5736 - f1_score: 0.5697 -
loss: 1.4767 - precision: 0.7774
Epoch 14: val_loss did not improve from 1.32116

657/657 ————— **18s** 28ms/step - accuracy: 0.5736 - f1_score: 0.5697
- loss: 1.4767 - precision: 0.7774 - val_accuracy: 0.6259 - val_f1_score: 0.6228
- val_loss: 1.3278 - val_precision: 0.8274
Epoch 15/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5748 - f1_score: 0.5710 -
loss: 1.4778 - precision: 0.7789
Epoch 15: val_loss improved from 1.32116 to 1.31937, saving model to best_model_v
2.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5748 - f1_score: 0.5710
- loss: 1.4778 - precision: 0.7789 - val_accuracy: 0.6244 - val_f1_score: 0.6206
- val_loss: 1.3194 - val_precision: 0.8243
Epoch 16/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5773 - f1_score: 0.5739 -
loss: 1.4729 - precision: 0.7746

Epoch 16: val_loss improved from 1.31937 to 1.30530, saving model to best_model_v2.keras

657/657  **19s** 29ms/step - accuracy: 0.5773 - f1_score: 0.5739
- loss: 1.4729 - precision: 0.7746 - val_accuracy: 0.6294 - val_f1_score: 0.6256
- val_loss: 1.3053 - val_precision: 0.8241

Epoch 17/25

657/657  **0s** 23ms/step - accuracy: 0.5755 - f1_score: 0.5725 -
loss: 1.4723 - precision: 0.7714

Epoch 17: val_loss did not improve from 1.30530

657/657  **18s** 28ms/step - accuracy: 0.5755 - f1_score: 0.5725
- loss: 1.4722 - precision: 0.7714 - val_accuracy: 0.6300 - val_f1_score: 0.6258
- val_loss: 1.3154 - val_precision: 0.8279

Epoch 18/25

657/657  **0s** 23ms/step - accuracy: 0.5830 - f1_score: 0.5795 -
loss: 1.4561 - precision: 0.7747

Epoch 18: val_loss did not improve from 1.30530

657/657  **18s** 28ms/step - accuracy: 0.5830 - f1_score: 0.5795
- loss: 1.4561 - precision: 0.7747 - val_accuracy: 0.6251 - val_f1_score: 0.6219
- val_loss: 1.3105 - val_precision: 0.8171

Epoch 19/25

655/657  **0s** 23ms/step - accuracy: 0.5844 - f1_score: 0.5816 -
loss: 1.4569 - precision: 0.7811

Epoch 19: val_loss improved from 1.30530 to 1.30179, saving model to best_model_v2.keras

657/657  **19s** 29ms/step - accuracy: 0.5845 - f1_score: 0.5816
- loss: 1.4568 - precision: 0.7811 - val_accuracy: 0.6284 - val_f1_score: 0.6258
- val_loss: 1.3018 - val_precision: 0.8192

Epoch 20/25

656/657  **0s** 23ms/step - accuracy: 0.5784 - f1_score: 0.5750 -
loss: 1.4554 - precision: 0.7749

Epoch 20: val_loss did not improve from 1.30179

657/657  **18s** 28ms/step - accuracy: 0.5784 - f1_score: 0.5750
- loss: 1.4554 - precision: 0.7749 - val_accuracy: 0.6268 - val_f1_score: 0.6248
- val_loss: 1.3157 - val_precision: 0.8243

Epoch 21/25

655/657  **0s** 23ms/step - accuracy: 0.5876 - f1_score: 0.5838 -
loss: 1.4432 - precision: 0.7829

Epoch 21: val_loss did not improve from 1.30179

657/657  **18s** 28ms/step - accuracy: 0.5876 - f1_score: 0.5839
- loss: 1.4431 - precision: 0.7829 - val_accuracy: 0.6300 - val_f1_score: 0.6275
- val_loss: 1.3107 - val_precision: 0.8242

Epoch 22/25

655/657  **0s** 23ms/step - accuracy: 0.5822 - f1_score: 0.5788 -
loss: 1.4529 - precision: 0.7776

Epoch 22: val_loss improved from 1.30179 to 1.30078, saving model to best_model_v2.keras

657/657  **19s** 29ms/step - accuracy: 0.5822 - f1_score: 0.5789
- loss: 1.4528 - precision: 0.7776 - val_accuracy: 0.6293 - val_f1_score: 0.6270
- val_loss: 1.3008 - val_precision: 0.8173

Epoch 23/25

657/657  **0s** 23ms/step - accuracy: 0.5853 - f1_score: 0.5819 -
loss: 1.4405 - precision: 0.7819

Epoch 23: val_loss did not improve from 1.30078

657/657  **18s** 28ms/step - accuracy: 0.5853 - f1_score: 0.5819
- loss: 1.4405 - precision: 0.7819 - val_accuracy: 0.6256 - val_f1_score: 0.6223
- val_loss: 1.3122 - val_precision: 0.8252

Epoch 24/25

656/657  **0s** 23ms/step - accuracy: 0.5856 - f1_score: 0.5821 -
loss: 1.4409 - precision: 0.7810

Epoch 24: val_loss did not improve from 1.30078

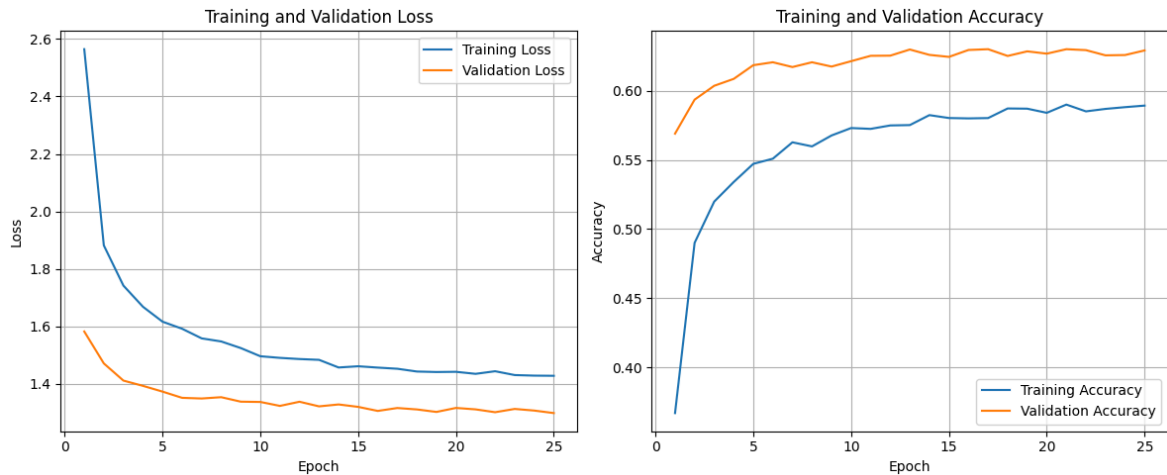
657/657 ————— **18s** 28ms/step - accuracy: 0.5856 - f1_score: 0.5821
 - loss: 1.4409 - precision: 0.7810 - val_accuracy: 0.6258 - val_f1_score: 0.6232
 - val_loss: 1.3065 - val_precision: 0.8170

Epoch 25/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5835 - f1_score: 0.5802 -
 loss: 1.4384 - precision: 0.7761

Epoch 25: val_loss improved from 1.30078 to 1.29781, saving model to best_model_v2.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5835 - f1_score: 0.5802
 - loss: 1.4384 - precision: 0.7761 - val_accuracy: 0.6291 - val_f1_score: 0.6268
 - val_loss: 1.2978 - val_precision: 0.8229



141/141 ————— **12s** 40ms/step - accuracy: 0.6236 - f1_score: 0.6156
 - loss: 1.3137 - precision: 0.8200

Test Accuracy: 63.22%

Test Loss: 1.2902672290802002

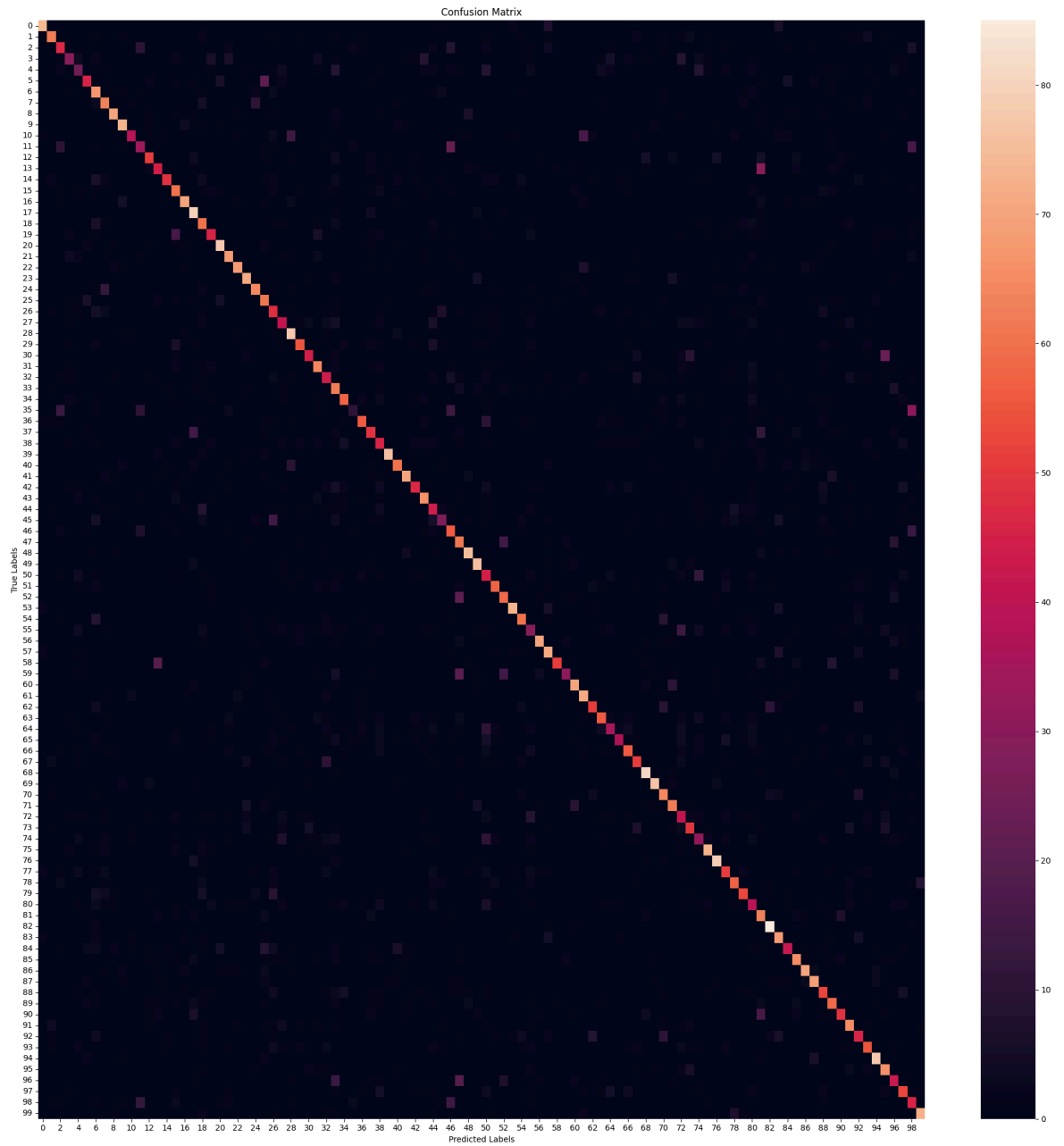
Test Precision: 82.56%

Test F1 Scores (Per Class): [84.57142 72.94117 52.808983 41.134747 31.578945 5
 8.227837 56.302513

64.2857	80.459755	80.21977	56.521732	40.71856	63.75	53.89221
66.666664	58.252422	80.459755	71.42857	55.81395	57.142853	82.53968
73.11827	80.473366	75.93583	73.56321	57.819897	44.85981	47.12643
75.72816	67.48465	59.602642	68.0851	48.61878	48.818893	67.83625
18.348621	69.135796	57.309933	47.66839	89.41175	70.17543	81.87134
61.437904	71.35134	46.31578	39.13043	50.22421	50.833332	81.52173
79.99999	39.999996	69.047615	62.433857	85.88235	67.796616	33.720924
80.92485	71.71716	70.83333	45.80152	77.34807	73.19587	62.962955
65.88235	45.45454	54.411762	65.11627	57.80347	84.37499	86.516846
63.681583	68.13186	40.59405	58.139526	35.63218	80.66298	84.491974
58.620686	68.23529	64.63414	41.052628	50.806446	85.858574	66.99506
54.54545	71.03825	79.54544	77.52808	58.24175	63.783775	61.63522
68.783066	53.48836	61.452507	85.71427	63.15789	45.4054	59.886997
42.592587	80.89887					

Average Test F1 Scores:63.10

282/282 ————— **17s** 37ms/step



	precision	recall	f1-score	support
beaver	0.87	0.82	0.85	90
dolphin	0.78	0.69	0.73	90
otter	0.53	0.52	0.53	90
seal	0.57	0.32	0.41	90
whale	0.39	0.27	0.32	90
aquarium fish	0.68	0.51	0.58	90
flatfish	0.45	0.74	0.56	90
ray	0.59	0.70	0.64	90
shark	0.83	0.78	0.80	90
trout	0.79	0.81	0.80	90
orchids	0.81	0.43	0.57	90
poppies	0.44	0.38	0.41	90
roses	0.73	0.57	0.64	90
sunflowers	0.58	0.50	0.54	90
tulips	0.86	0.54	0.67	90
bottles	0.52	0.67	0.58	90
bowls	0.83	0.78	0.80	90
cans	0.60	0.89	0.71	90
cups	0.48	0.67	0.56	90
plates	0.65	0.51	0.57	90
apples	0.79	0.87	0.83	90
mushrooms	0.71	0.76	0.73	90
oranges	0.86	0.76	0.80	90
pears	0.73	0.79	0.76	90
sweet peppers	0.76	0.71	0.74	90
clock	0.50	0.68	0.58	90
computer keyboard	0.39	0.53	0.45	90
lamp	0.49	0.46	0.47	90
telephone	0.67	0.87	0.76	90
television	0.75	0.61	0.67	90
bed	0.74	0.50	0.60	90
chair	0.65	0.71	0.68	90
couch	0.48	0.49	0.49	90
table	0.38	0.69	0.49	90
wardrobe	0.72	0.64	0.68	90
bee	0.53	0.11	0.18	90
beetle	0.78	0.62	0.69	90
butterfly	0.60	0.54	0.57	90
caterpillar	0.45	0.51	0.48	90
cockroach	0.95	0.84	0.89	90
bear	0.74	0.67	0.70	90
leopard	0.86	0.78	0.82	90
lion	0.75	0.52	0.61	90
tiger	0.69	0.73	0.71	90
wolf	0.44	0.49	0.46	90
bridge	0.56	0.30	0.39	90
castle	0.42	0.62	0.50	90
house	0.41	0.68	0.51	90
road	0.80	0.83	0.82	90
skyscraper	0.76	0.84	0.80	90
cloud	0.33	0.50	0.40	90
forest	0.74	0.64	0.69	90
mountain	0.60	0.66	0.62	90
plain	0.91	0.81	0.86	90
sea	0.69	0.67	0.68	90
camel	0.35	0.32	0.34	90
cattle	0.84	0.78	0.81	90
chimpanzee	0.66	0.79	0.72	90

elephant	0.94	0.57	0.71	90
kangaroo	0.73	0.33	0.46	90
fox	0.77	0.78	0.77	90
porcupine	0.68	0.79	0.73	90
possum	0.71	0.57	0.63	90
raccoon	0.70	0.62	0.66	90
skunk	0.55	0.39	0.45	90
crab	0.80	0.41	0.54	90
lobster	0.68	0.62	0.65	90
snail	0.61	0.57	0.59	90
spider	0.79	0.90	0.84	90
worm	0.88	0.86	0.87	90
baby	0.58	0.71	0.64	90
boy	0.67	0.69	0.68	90
girl	0.37	0.46	0.41	90
man	0.62	0.56	0.58	90
woman	0.37	0.34	0.36	90
crocodile	0.80	0.81	0.81	90
dinosaur	0.81	0.88	0.84	90
lizard	0.61	0.57	0.59	90
snake	0.72	0.64	0.68	90
turtle	0.72	0.59	0.65	90
hamster	0.39	0.43	0.41	90
mouse	0.40	0.70	0.51	90
rabbit	0.79	0.94	0.86	90
shrew	0.60	0.76	0.67	90
squirrel	0.66	0.47	0.55	90
maple	0.70	0.72	0.71	90
oak	0.81	0.78	0.80	90
palm	0.78	0.77	0.78	90
pine	0.58	0.59	0.58	90
willow	0.62	0.66	0.64	90
bicycle	0.71	0.54	0.62	90
bus	0.66	0.72	0.69	90
motorcycle	0.56	0.51	0.53	90
pickup truck	0.62	0.61	0.61	90
train	0.85	0.87	0.86	90
lawn-mower	0.55	0.73	0.63	90
rocket	0.44	0.47	0.45	90
streetcar	0.61	0.59	0.60	90
tank	0.37	0.51	0.43	90
tractor	0.82	0.80	0.81	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)



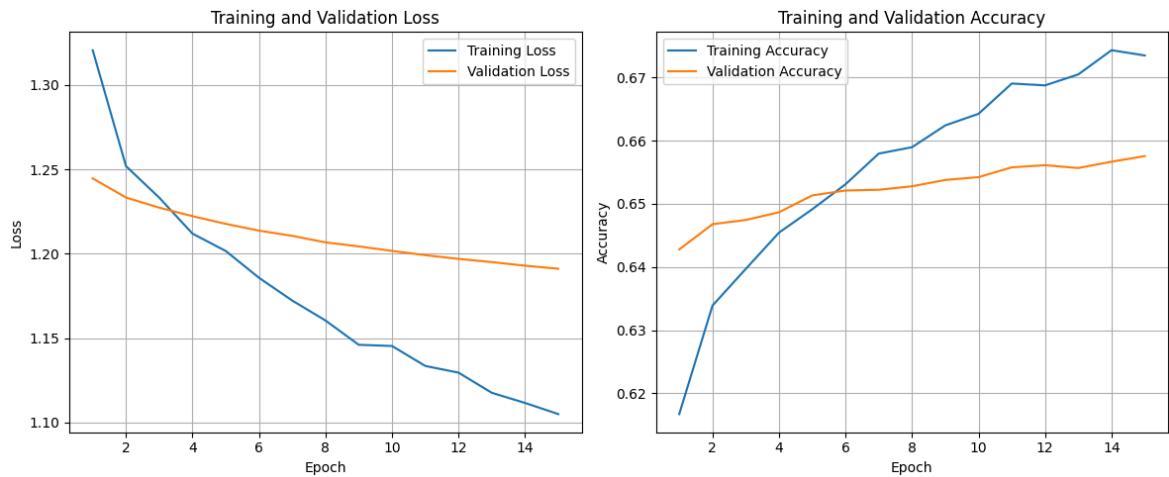
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 26ms/step - accuracy: 0.6086 - f1_score: 0.6059
- loss: 1.3521 - precision: 0.7966
Epoch 1: val_loss improved from inf to 1.24463, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6087 - f1_score: 0.6059 - loss: 1.3520 - precision: 0.7966 - val_accuracy: 0.6428 - val_f1_score: 0.6406 - val_loss: 1.2446 - val_precision: 0.8284
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6319 - f1_score: 0.6289 - loss: 1.2525 - precision: 0.8103
Epoch 2: val_loss improved from 1.24463 to 1.23330, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6319 - f1_score: 0.6289 - loss: 1.2525 - precision: 0.8103 - val_accuracy: 0.6468 - val_f1_score: 0.6446 - val_loss: 1.2333 - val_precision: 0.8268
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6386 - f1_score: 0.6354 - loss: 1.2321 - precision: 0.8107
Epoch 3: val_loss improved from 1.23330 to 1.22729, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6386 - f1_score: 0.6355 - loss: 1.2321 - precision: 0.8107 - val_accuracy: 0.6474 - val_f1_score: 0.6453 - val_loss: 1.2273 - val_precision: 0.8242
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6471 - f1_score: 0.6442 - loss: 1.2060 - precision: 0.8169
Epoch 4: val_loss improved from 1.22729 to 1.22223, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6471 - f1_score: 0.6442 - loss: 1.2060 - precision: 0.8169 - val_accuracy: 0.6487 - val_f1_score: 0.6466 - val_loss: 1.2222 - val_precision: 0.8250
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6480 - f1_score: 0.6451 - loss: 1.1996 - precision: 0.8191
Epoch 5: val_loss improved from 1.22223 to 1.21769, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6480 - f1_score: 0.6451 - loss: 1.1996 - precision: 0.8191 - val_accuracy: 0.6513 - val_f1_score: 0.6493 - val_loss: 1.2177 - val_precision: 0.8244
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6532 - f1_score: 0.6503 - loss: 1.1796 - precision: 0.8250
Epoch 6: val_loss improved from 1.21769 to 1.21368, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6532 - f1_score: 0.6503 - loss: 1.1796 - precision: 0.8250 - val_accuracy: 0.6521 - val_f1_score: 0.6503 - val_loss: 1.2137 - val_precision: 0.8243
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6600 - f1_score: 0.6571 - loss: 1.1629 - precision: 0.8263
Epoch 7: val_loss improved from 1.21368 to 1.21056, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6600 - f1_score: 0.6571 - loss: 1.1629 - precision: 0.8263 - val_accuracy: 0.6522 - val_f1_score: 0.6505 - val_loss: 1.2106 - val_precision: 0.8241
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6594 - f1_score: 0.6566 - loss: 1.1549 - precision: 0.8294
Epoch 8: val_loss improved from 1.21056 to 1.20673, saving model to best_model_fine_tune_v2.keras

```
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6594 - f1_score: 0.656
6 - loss: 1.1549 - precision: 0.8294 - val_accuracy: 0.6528 - val_f1_score: 0.651
1 - val_loss: 1.2067 - val_precision: 0.8237
Epoch 9/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6638 - f1_score: 0.6612
- loss: 1.1400 - precision: 0.8327
Epoch 9: val_loss improved from 1.20673 to 1.20432, saving model to best_model_fi
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6638 - f1_score: 0.661
2 - loss: 1.1400 - precision: 0.8327 - val_accuracy: 0.6538 - val_f1_score: 0.652
3 - val_loss: 1.2043 - val_precision: 0.8232
Epoch 10/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6658 - f1_score: 0.6632
- loss: 1.1360 - precision: 0.8323
Epoch 10: val_loss improved from 1.20432 to 1.20166, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6658 - f1_score: 0.663
2 - loss: 1.1361 - precision: 0.8323 - val_accuracy: 0.6542 - val_f1_score: 0.652
5 - val_loss: 1.2017 - val_precision: 0.8245
Epoch 11/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6714 - f1_score: 0.6687
- loss: 1.1212 - precision: 0.8366
Epoch 11: val_loss improved from 1.20166 to 1.19912, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6714 - f1_score: 0.668
7 - loss: 1.1212 - precision: 0.8366 - val_accuracy: 0.6558 - val_f1_score: 0.654
1 - val_loss: 1.1991 - val_precision: 0.8249
Epoch 12/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6708 - f1_score: 0.6681
- loss: 1.1200 - precision: 0.8348
Epoch 12: val_loss improved from 1.19912 to 1.19692, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6708 - f1_score: 0.668
1 - loss: 1.1200 - precision: 0.8348 - val_accuracy: 0.6561 - val_f1_score: 0.654
3 - val_loss: 1.1969 - val_precision: 0.8262
Epoch 13/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6739 - f1_score: 0.6712
- loss: 1.1029 - precision: 0.8366
Epoch 13: val_loss improved from 1.19692 to 1.19502, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6739 - f1_score: 0.671
2 - loss: 1.1029 - precision: 0.8366 - val_accuracy: 0.6557 - val_f1_score: 0.654
2 - val_loss: 1.1950 - val_precision: 0.8259
Epoch 14/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.6765 - f1_score: 0.6740
- loss: 1.0987 - precision: 0.8414
Epoch 14: val_loss improved from 1.19502 to 1.19290, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6765 - f1_score: 0.674
0 - loss: 1.0988 - precision: 0.8414 - val_accuracy: 0.6567 - val_f1_score: 0.655
0 - val_loss: 1.1929 - val_precision: 0.8259
Epoch 15/15
1313/1314 ————— 0s 23ms/step - accuracy: 0.6765 - f1_score: 0.6743
- loss: 1.0925 - precision: 0.8406
Epoch 15: val_loss improved from 1.19290 to 1.19108, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6765 - f1_score: 0.674
2 - loss: 1.0925 - precision: 0.8406 - val_accuracy: 0.6576 - val_f1_score: 0.655
9 - val_loss: 1.1911 - val_precision: 0.8257
```



141/141 ————— **11s** 41ms/step - accuracy: 0.6515 - f1_score: 0.6438

- loss: 1.2108 - precision: 0.8186

Test Accuracy: 65.88%

Test Loss: 1.1845217943191528

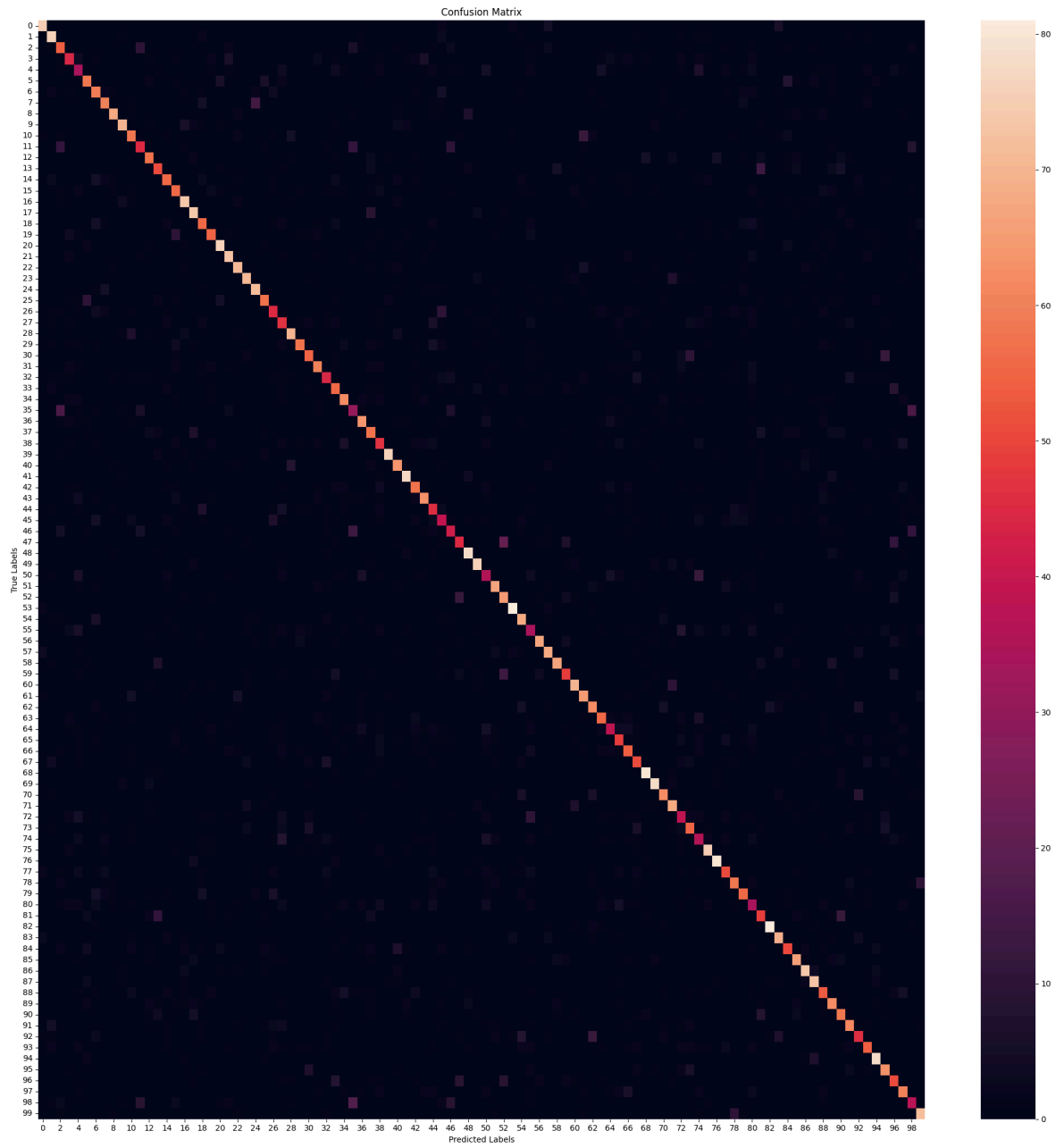
Test Precision: 82.74%

Test F1 Scores (Per Class): [83.798874 77.15736 54.82233 52.694607 37.36263 64.80447 59.595955

69.36415	79.99999	80.92485	63.73626	53.89221	64.77273	58.959538
65.88235	61.202175	77.08333	75.89743	58.947365	64.70587	82.16216
77.24867	78.453026	79.775276	78.68852	68.67469	51.42857	45.999996
78.65168	71.25	63.5838	69.76743	52.694607	56.852787	68.888885
35.63218	70.329666	60.63829	51.366108	87.86127	67.36841	85.08286
64.04494	71.99999	49.46236	43.575413	48.554905	54.878044	81.675385
82.608696	40.935665	74.86033	64.676605	89.502754	69.38775	36.464085
79.761894	74.725266	78.82352	59.036137	77.77777	71.8232	67.02702
65.11627	47.13375	56.97674	59.66851	57.303364	86.02149	83.87096
70.85714	72.34043	43.428566	61.111103	39.344257	80.64516	88.888885
58.959538	66.666664	65.49707	36.55913	53.551907	91.01122	73.01586
59.171593	71.35134	81.31868	77.24867	59.21787	65.60846	61.052628
68.15642	55.421684	61.714287	84.32432	67.37968	53.124992	65.57377
42.774563	77.83783					

Average Test F1 Scores:65.79

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.84	0.83	0.84	90
dolphin	0.71	0.84	0.77	90
otter	0.50	0.60	0.55	90
seal	0.57	0.49	0.53	90
whale	0.37	0.38	0.37	90
aquarium fish	0.65	0.64	0.65	90
flatfish	0.55	0.66	0.60	90
ray	0.72	0.67	0.69	90
shark	0.85	0.76	0.80	90
trout	0.84	0.78	0.81	90
orchids	0.63	0.64	0.64	90
poppies	0.58	0.50	0.54	90
roses	0.66	0.63	0.65	90
sunflowers	0.61	0.57	0.59	90
tulips	0.70	0.62	0.66	90
bottles	0.60	0.62	0.61	90
bowls	0.73	0.82	0.77	90
cans	0.70	0.82	0.76	90
cups	0.56	0.62	0.59	90
plates	0.69	0.61	0.65	90
apples	0.80	0.84	0.82	90
mushrooms	0.74	0.81	0.77	90
oranges	0.78	0.79	0.78	90
pears	0.81	0.79	0.80	90
sweet peppers	0.77	0.80	0.79	90
clock	0.75	0.63	0.69	90
computer keyboard	0.53	0.50	0.51	90
lamp	0.42	0.52	0.47	90
telephone	0.80	0.78	0.79	90
television	0.81	0.63	0.71	90
bed	0.67	0.61	0.64	90
chair	0.73	0.67	0.70	90
couch	0.57	0.49	0.53	90
table	0.52	0.62	0.57	90
wardrobe	0.69	0.69	0.69	90
bee	0.37	0.34	0.36	90
beetle	0.70	0.71	0.71	90
butterfly	0.58	0.63	0.61	90
caterpillar	0.51	0.52	0.51	90
cockroach	0.92	0.84	0.88	90
bear	0.64	0.71	0.67	90
leopard	0.85	0.86	0.85	90
lion	0.65	0.63	0.64	90
tiger	0.74	0.70	0.72	90
wolf	0.48	0.51	0.49	90
bridge	0.44	0.43	0.44	90
castle	0.51	0.47	0.49	90
house	0.61	0.50	0.55	90
road	0.77	0.87	0.82	90
skyscraper	0.81	0.84	0.83	90
cloud	0.43	0.39	0.41	90
forest	0.75	0.74	0.75	90
mountain	0.59	0.72	0.65	90
plain	0.89	0.90	0.90	90
sea	0.64	0.76	0.69	90
camel	0.37	0.38	0.37	90
cattle	0.86	0.74	0.80	90
chimpanzee	0.74	0.76	0.75	90

elephant	0.84	0.74	0.79	90
kangaroo	0.64	0.54	0.59	90
fox	0.78	0.78	0.78	90
porcupine	0.71	0.72	0.72	90
possum	0.65	0.69	0.67	90
raccoon	0.68	0.62	0.65	90
skunk	0.56	0.42	0.48	90
crab	0.60	0.54	0.57	90
lobster	0.59	0.60	0.60	90
snail	0.58	0.57	0.57	90
spider	0.83	0.89	0.86	90
worm	0.81	0.87	0.84	90
baby	0.73	0.69	0.71	90
boy	0.69	0.76	0.72	90
girl	0.45	0.42	0.44	90
man	0.61	0.61	0.61	90
woman	0.39	0.40	0.39	90
crocodile	0.78	0.83	0.81	90
dinosaur	0.89	0.89	0.89	90
lizard	0.61	0.57	0.59	90
snake	0.69	0.64	0.67	90
turtle	0.69	0.62	0.65	90
hamster	0.35	0.38	0.37	90
mouse	0.53	0.54	0.54	90
rabbit	0.92	0.90	0.91	90
shrew	0.70	0.77	0.73	90
squirrel	0.63	0.56	0.59	90
maple	0.69	0.73	0.71	90
oak	0.80	0.82	0.81	90
palm	0.74	0.81	0.77	90
pine	0.60	0.59	0.59	90
willow	0.63	0.69	0.66	90
bicycle	0.58	0.64	0.61	90
bus	0.69	0.68	0.68	90
motorcycle	0.61	0.51	0.55	90
pickup truck	0.64	0.60	0.62	90
train	0.82	0.87	0.84	90
lawn-mower	0.65	0.70	0.67	90
rocket	0.50	0.57	0.53	90
streetcar	0.65	0.67	0.66	90
tank	0.45	0.41	0.43	90
tractor	0.76	0.80	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

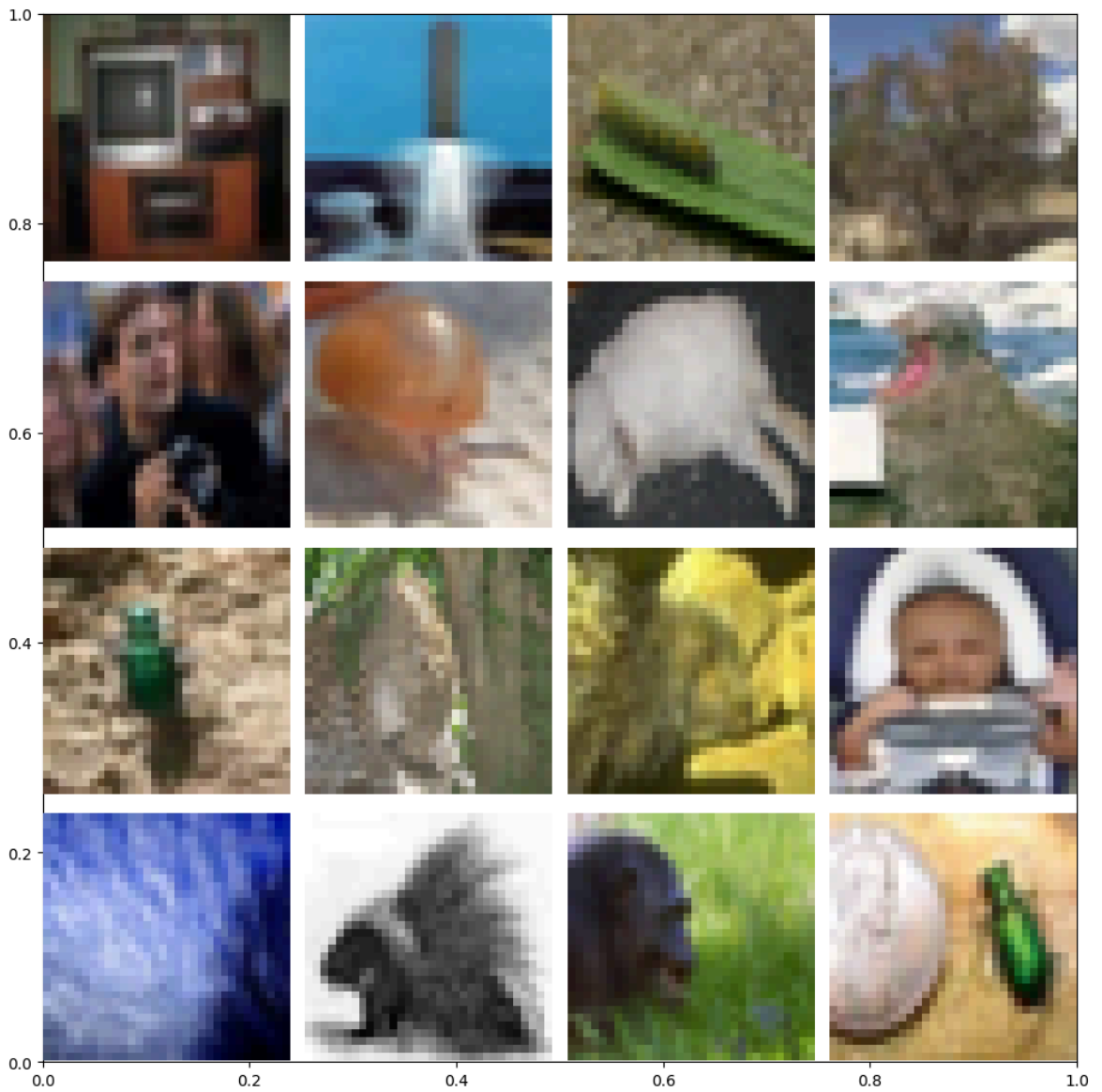
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

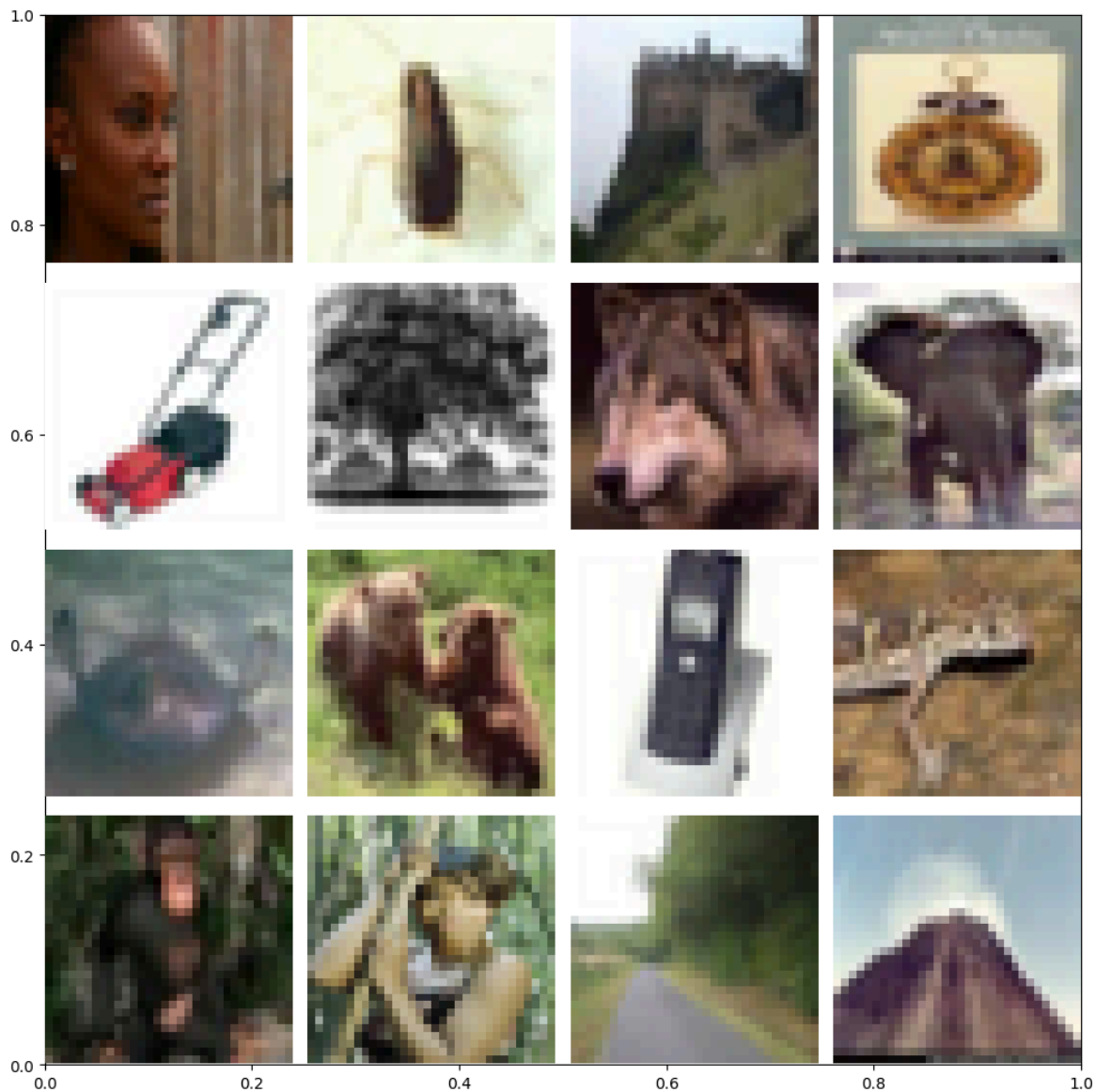
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[15334].shape: (1,)  
imgs[15334].shape: (32, 32, 3)
```

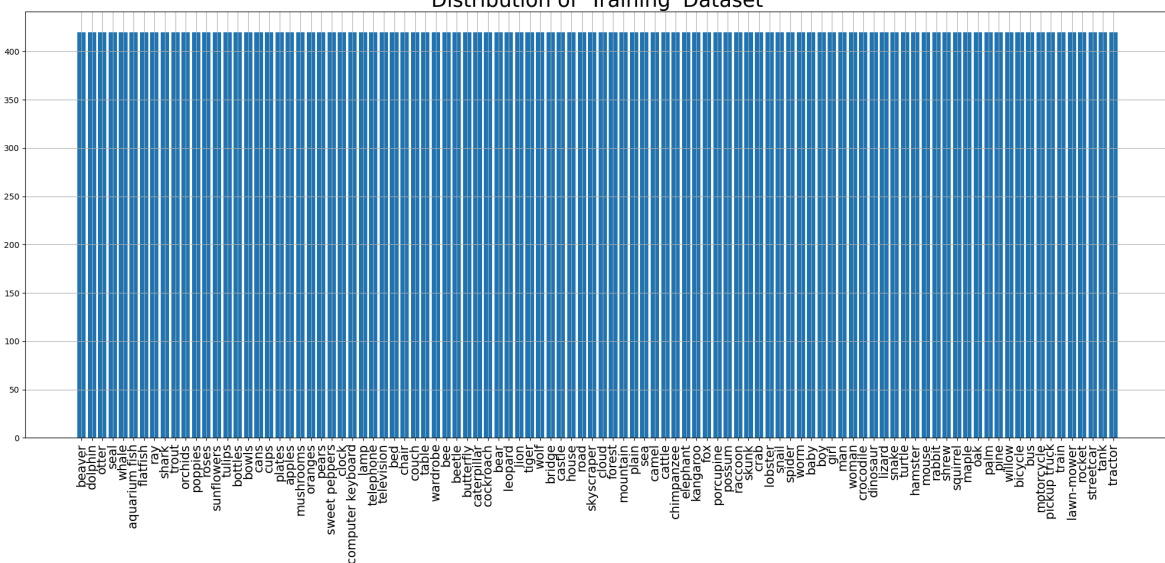


```
labels[29693].shape: (1,)  
imgs[29693].shape: (32, 32, 3)
```

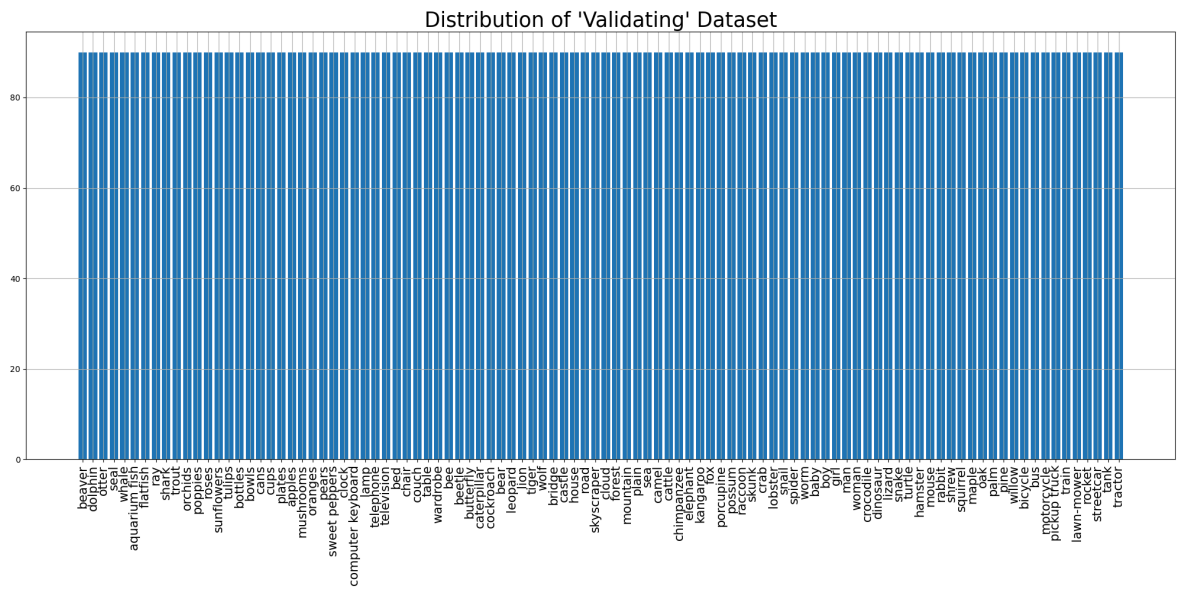


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

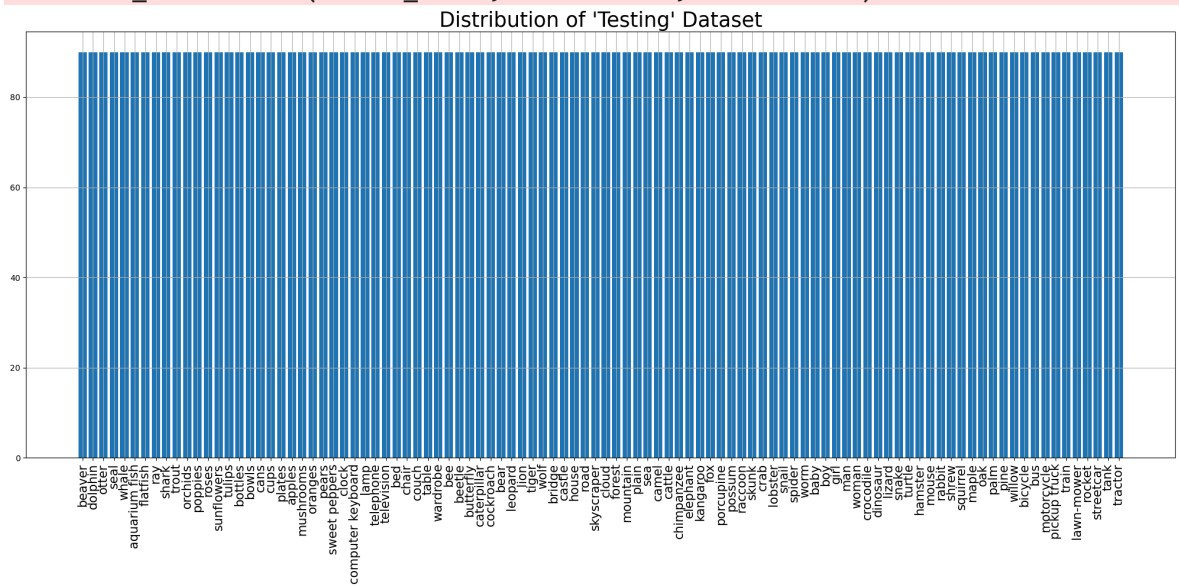
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— 0s 28ms/step - accuracy: 0.2630 - f1_score: 0.2559 - loss: 3.1942 - precision: 0.6289
Epoch 1: val_loss improved from inf to 1.58368, saving model to best_model_v2.keras as
657/657 ————— 40s 45ms/step - accuracy: 0.2631 - f1_score: 0.2561 - loss: 3.1932 - precision: 0.6290 - val_accuracy: 0.5776 - val_f1_score: 0.5698 - val_loss: 1.5837 - val_precision: 0.8350
Epoch 2/25
657/657 ————— 0s 23ms/step - accuracy: 0.4760 - f1_score: 0.4716 - loss: 1.9423 - precision: 0.7275
Epoch 2: val_loss improved from 1.58368 to 1.47329, saving model to best_model_v2.keras
657/657 ————— 20s 30ms/step - accuracy: 0.4760 - f1_score: 0.4716 - loss: 1.9422 - precision: 0.7275 - val_accuracy: 0.5937 - val_f1_score: 0.5899 - val_loss: 1.4733 - val_precision: 0.8270
Epoch 3/25
655/657 ————— 0s 24ms/step - accuracy: 0.5129 - f1_score: 0.5084 - loss: 1.7747 - precision: 0.7564
Epoch 3: val_loss improved from 1.47329 to 1.42262, saving model to best_model_v2.keras
657/657 ————— 19s 30ms/step - accuracy: 0.5129 - f1_score: 0.5084 - loss: 1.7746 - precision: 0.7564 - val_accuracy: 0.6013 - val_f1_score: 0.5974 - val_loss: 1.4226 - val_precision: 0.8383
Epoch 4/25
656/657 ————— 0s 23ms/step - accuracy: 0.5262 - f1_score: 0.5221 - loss: 1.7072 - precision: 0.7574
Epoch 4: val_loss improved from 1.42262 to 1.38554, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5262 - f1_score: 0.5221 - loss: 1.7071 - precision: 0.7574 - val_accuracy: 0.6131 - val_f1_score: 0.6103 - val_loss: 1.3855 - val_precision: 0.8317
Epoch 5/25
657/657 ————— 0s 23ms/step - accuracy: 0.5330 - f1_score: 0.5283 - loss: 1.6602 - precision: 0.7584
Epoch 5: val_loss improved from 1.38554 to 1.37380, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5330 - f1_score: 0.5283 - loss: 1.6602 - precision: 0.7584 - val_accuracy: 0.6169 - val_f1_score: 0.6125 - val_loss: 1.3738 - val_precision: 0.8343
Epoch 6/25
655/657 ————— 0s 23ms/step - accuracy: 0.5439 - f1_score: 0.5393 - loss: 1.6203 - precision: 0.7656
Epoch 6: val_loss improved from 1.37380 to 1.36147, saving model to best_model_v2.keras
657/657 ————— 19s 29ms/step - accuracy: 0.5439 - f1_score: 0.5393 - loss: 1.6202 - precision: 0.7656 - val_accuracy: 0.6158 - val_f1_score: 0.6144 - val_loss: 1.3615 - val_precision: 0.8238
Epoch 7/25
655/657 ————— 0s 23ms/step - accuracy: 0.5504 - f1_score: 0.5463 - loss: 1.5886 - precision: 0.7699
Epoch 7: val_loss did not improve from 1.36147
657/657 ————— 18s 28ms/step - accuracy: 0.5505 - f1_score: 0.5464 - loss: 1.5885 - precision: 0.7699 - val_accuracy: 0.6137 - val_f1_score: 0.6094 - val_loss: 1.3616 - val_precision: 0.8292
Epoch 8/25
656/657 ————— 0s 23ms/step - accuracy: 0.5573 - f1_score: 0.5530 - loss: 1.5660 - precision: 0.7679
Epoch 8: val_loss improved from 1.36147 to 1.34036, saving model to best_model_v2.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.5573 - f1_score: 0.5530
- loss: 1.5659 - precision: 0.7679 - val_accuracy: 0.6210 - val_f1_score: 0.6178
- val_loss: 1.3404 - val_precision: 0.8299
Epoch 9/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5631 - f1_score: 0.5585 -
loss: 1.5410 - precision: 0.7698
Epoch 9: val_loss improved from 1.34036 to 1.33717, saving model to best_model_v
2.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.5631 - f1_score: 0.5586
- loss: 1.5409 - precision: 0.7698 - val_accuracy: 0.6243 - val_f1_score: 0.6206
- val_loss: 1.3372 - val_precision: 0.8297
Epoch 10/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5684 - f1_score: 0.5640 -
loss: 1.5282 - precision: 0.7756
Epoch 10: val_loss did not improve from 1.33717

657/657 ————— **18s** 28ms/step - accuracy: 0.5685 - f1_score: 0.5640
- loss: 1.5282 - precision: 0.7756 - val_accuracy: 0.6194 - val_f1_score: 0.6153
- val_loss: 1.3407 - val_precision: 0.8216
Epoch 11/25

656/657 ————— **0s** 23ms/step - accuracy: 0.5694 - f1_score: 0.5659 -
loss: 1.5149 - precision: 0.7761
Epoch 11: val_loss improved from 1.33717 to 1.31994, saving model to best_model_v
2.keras

657/657 ————— **19s** 29ms/step - accuracy: 0.5694 - f1_score: 0.5659
- loss: 1.5148 - precision: 0.7761 - val_accuracy: 0.6299 - val_f1_score: 0.6257
- val_loss: 1.3199 - val_precision: 0.8249
Epoch 12/25

656/657 ————— **0s** 23ms/step - accuracy: 0.5683 - f1_score: 0.5641 -
loss: 1.5192 - precision: 0.7731
Epoch 12: val_loss did not improve from 1.31994

657/657 ————— **18s** 28ms/step - accuracy: 0.5683 - f1_score: 0.5641
- loss: 1.5191 - precision: 0.7731 - val_accuracy: 0.6243 - val_f1_score: 0.6206
- val_loss: 1.3240 - val_precision: 0.8253
Epoch 13/25

657/657 ————— **0s** 23ms/step - accuracy: 0.5755 - f1_score: 0.5714 -
loss: 1.4920 - precision: 0.7762
Epoch 13: val_loss did not improve from 1.31994

657/657 ————— **18s** 28ms/step - accuracy: 0.5755 - f1_score: 0.5714
- loss: 1.4920 - precision: 0.7762 - val_accuracy: 0.6247 - val_f1_score: 0.6208
- val_loss: 1.3292 - val_precision: 0.8219
Epoch 14/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5743 - f1_score: 0.5703 -
loss: 1.4920 - precision: 0.7750
Epoch 14: val_loss improved from 1.31994 to 1.31455, saving model to best_model_v
2.keras

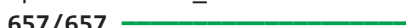
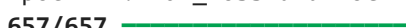
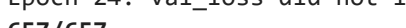
657/657 ————— **19s** 29ms/step - accuracy: 0.5743 - f1_score: 0.5704
- loss: 1.4919 - precision: 0.7750 - val_accuracy: 0.6266 - val_f1_score: 0.6226
- val_loss: 1.3145 - val_precision: 0.8246
Epoch 15/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5747 - f1_score: 0.5708 -
loss: 1.4903 - precision: 0.7729
Epoch 15: val_loss did not improve from 1.31455

657/657 ————— **18s** 28ms/step - accuracy: 0.5747 - f1_score: 0.5708
- loss: 1.4902 - precision: 0.7729 - val_accuracy: 0.6209 - val_f1_score: 0.6174
- val_loss: 1.3241 - val_precision: 0.8198
Epoch 16/25

655/657 ————— **0s** 23ms/step - accuracy: 0.5790 - f1_score: 0.5756 -
loss: 1.4805 - precision: 0.7757
Epoch 16: val_loss did not improve from 1.31455

657/657 ————— **18s** 28ms/step - accuracy: 0.5790 - f1_score: 0.5756

- loss: 1.4805 - precision: 0.7757 - val_accuracy: 0.6293 - val_f1_score: 0.6265
- val_loss: 1.3152 - val_precision: 0.8214
Epoch 17/25
657/657  **0s** 23ms/step - accuracy: 0.5764 - f1_score: 0.5725 -
loss: 1.4797 - precision: 0.7715
Epoch 17: val_loss did not improve from 1.31455
657/657  **18s** 28ms/step - accuracy: 0.5764 - f1_score: 0.5725
- loss: 1.4797 - precision: 0.7715 - val_accuracy: 0.6249 - val_f1_score: 0.6228
- val_loss: 1.3183 - val_precision: 0.8191
Epoch 18/25
657/657  **0s** 23ms/step - accuracy: 0.5758 - f1_score: 0.5724 -
loss: 1.4737 - precision: 0.7734
Epoch 18: val_loss did not improve from 1.31455
657/657  **18s** 28ms/step - accuracy: 0.5758 - f1_score: 0.5724
- loss: 1.4736 - precision: 0.7735 - val_accuracy: 0.6249 - val_f1_score: 0.6209
- val_loss: 1.3196 - val_precision: 0.8156
Epoch 19/25
656/657  **0s** 23ms/step - accuracy: 0.5810 - f1_score: 0.5775 -
loss: 1.4655 - precision: 0.7777
Epoch 19: val_loss improved from 1.31455 to 1.30428, saving model to best_model_v
2.keras
657/657  **19s** 29ms/step - accuracy: 0.5810 - f1_score: 0.5775
- loss: 1.4654 - precision: 0.7777 - val_accuracy: 0.6313 - val_f1_score: 0.6288
- val_loss: 1.3043 - val_precision: 0.8180
Epoch 20/25
656/657  **0s** 23ms/step - accuracy: 0.5784 - f1_score: 0.5745 -
loss: 1.4618 - precision: 0.7793
Epoch 20: val_loss improved from 1.30428 to 1.30310, saving model to best_model_v
2.keras
657/657  **19s** 29ms/step - accuracy: 0.5784 - f1_score: 0.5745
- loss: 1.4617 - precision: 0.7793 - val_accuracy: 0.6318 - val_f1_score: 0.6300
- val_loss: 1.3031 - val_precision: 0.8274
Epoch 21/25
656/657  **0s** 23ms/step - accuracy: 0.5796 - f1_score: 0.5763 -
loss: 1.4536 - precision: 0.7793
Epoch 21: val_loss did not improve from 1.30310
657/657  **18s** 28ms/step - accuracy: 0.5796 - f1_score: 0.5763
- loss: 1.4535 - precision: 0.7793 - val_accuracy: 0.6340 - val_f1_score: 0.6298
- val_loss: 1.3087 - val_precision: 0.8317
Epoch 22/25
656/657  **0s** 23ms/step - accuracy: 0.5832 - f1_score: 0.5797 -
loss: 1.4479 - precision: 0.7767
Epoch 22: val_loss did not improve from 1.30310
657/657  **18s** 28ms/step - accuracy: 0.5832 - f1_score: 0.5797
- loss: 1.4478 - precision: 0.7767 - val_accuracy: 0.6341 - val_f1_score: 0.6320
- val_loss: 1.3124 - val_precision: 0.8191
Epoch 23/25
656/657  **0s** 23ms/step - accuracy: 0.5829 - f1_score: 0.5797 -
loss: 1.4429 - precision: 0.7744
Epoch 23: val_loss improved from 1.30310 to 1.29402, saving model to best_model_v
2.keras
657/657  **19s** 30ms/step - accuracy: 0.5829 - f1_score: 0.5797
- loss: 1.4428 - precision: 0.7744 - val_accuracy: 0.6321 - val_f1_score: 0.6297
- val_loss: 1.2940 - val_precision: 0.8241
Epoch 24/25
656/657  **0s** 23ms/step - accuracy: 0.5872 - f1_score: 0.5832 -
loss: 1.4395 - precision: 0.7789
Epoch 24: val_loss did not improve from 1.29402
657/657  **18s** 28ms/step - accuracy: 0.5872 - f1_score: 0.5832
- loss: 1.4395 - precision: 0.7789 - val_accuracy: 0.6281 - val_f1_score: 0.6238

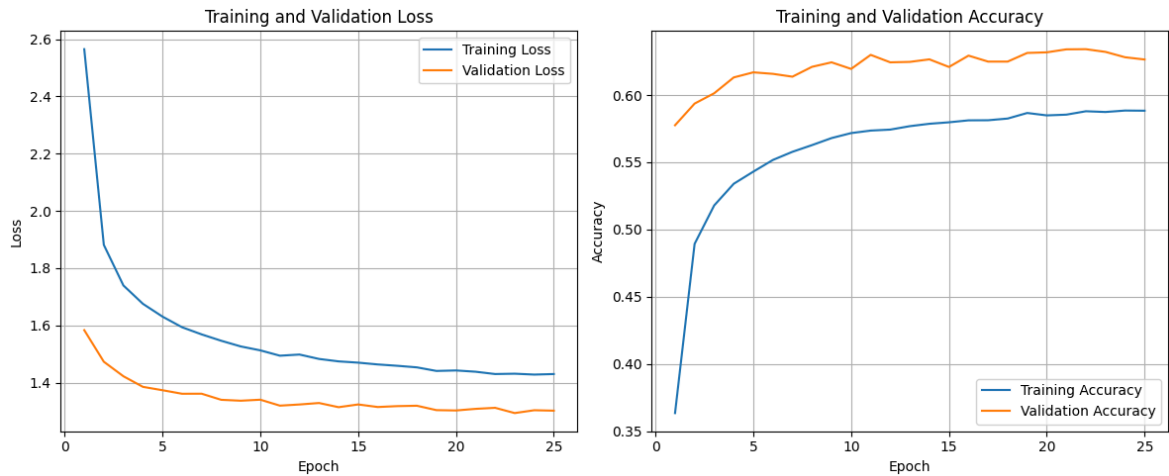
- val_loss: 1.3039 - val_precision: 0.8244

Epoch 25/25

655/657 ————— 0s 23ms/step - accuracy: 0.5853 - f1_score: 0.5817 - loss: 1.4457 - precision: 0.7786

Epoch 25: val_loss did not improve from 1.29402

657/657 ————— 18s 28ms/step - accuracy: 0.5853 - f1_score: 0.5817 - loss: 1.4456 - precision: 0.7786 - val_accuracy: 0.6264 - val_f1_score: 0.6231 - val_loss: 1.3024 - val_precision: 0.8272



141/141 ————— 11s 38ms/step - accuracy: 0.6235 - f1_score: 0.6165 - loss: 1.3138 - precision: 0.8101

Test Accuracy: 63.33%

Test Loss: 1.285614013671875

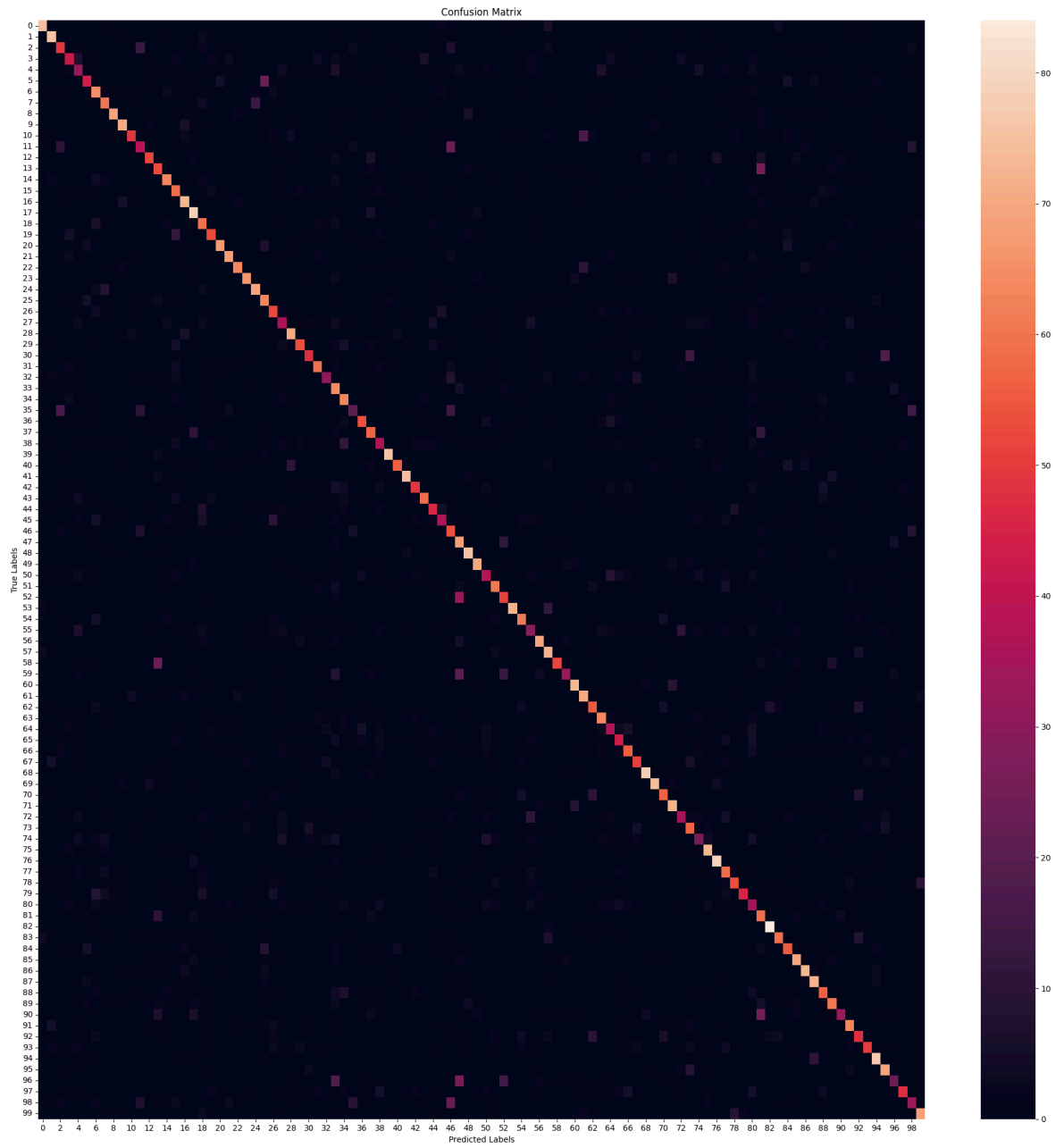
Test Precision: 81.69%

Test F1 Scores (Per Class): [84.74575 77.948715 52.68817 50.602406 35.955055 5 5.345905 58.447487

66.304344	80.701744	80.	62.820507	45.61403	65.4088	50.7177
69.23077	56.459324	74.111664	74.881516	54.298634	62.72189	81.21211
75.97765	77.300606	77.19298	77.09496	59.999996	53.88601	41.142853
77.34807	66.25767	59.493668	68.57143	43.055553	47.05882	61.46341
31.343279	66.666664	60.31745	43.786976	88.372086	68.71165	84.74575
62.025307	69.82249	50.82873	48.05194	44.999996	52.107273	78.75647
80.92485	45.783127	69.31818	55.13513	86.74699	68.888885	35.36585
75.82417	67.60563	71.23287	49.999992	79.78141	70.35175	61.797745
67.02127	44.31137	49.999992	61.87845	59.999992	84.7826	85.057465
67.85713	75.39267	42.94478	59.685856	34.666664	76.43979	82.7225
60.20408	64.28571	60.40268	33.98058	46.456684	87.49999	71.08433
59.7826	74.19354	79.78141	75.39267	58.947365	64.55026	49.999992
67.74193	54.02298	60.975605	81.91489	69.69697	33.57664	62.66666
40.49079	76.83616					

Average Test F1 Scores:63.23

282/282 ————— 11s 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.83	0.85	90
dolphin	0.72	0.84	0.78	90
otter	0.51	0.54	0.53	90
seal	0.55	0.47	0.51	90
whale	0.36	0.36	0.36	90
aquarium fish	0.64	0.49	0.55	90
flatfish	0.50	0.71	0.58	90
ray	0.65	0.68	0.66	90
shark	0.85	0.77	0.81	90
trout	0.82	0.78	0.80	90
orchids	0.74	0.54	0.63	90
poppies	0.48	0.43	0.46	90
roses	0.75	0.58	0.65	90
sunflowers	0.45	0.59	0.51	90
tulips	0.68	0.70	0.69	90
bottles	0.50	0.66	0.56	90
bowls	0.68	0.81	0.74	90
cans	0.65	0.88	0.75	90
cups	0.46	0.67	0.54	90
plates	0.67	0.59	0.63	90
apples	0.89	0.74	0.81	90
mushrooms	0.76	0.76	0.76	90
oranges	0.86	0.70	0.77	90
pears	0.81	0.73	0.77	90
sweet peppers	0.78	0.77	0.77	90
clock	0.53	0.70	0.60	90
computer keyboard	0.50	0.58	0.54	90
lamp	0.42	0.40	0.41	90
telephone	0.77	0.78	0.77	90
television	0.73	0.60	0.66	90
bed	0.69	0.52	0.59	90
chair	0.71	0.67	0.69	90
couch	0.57	0.34	0.43	90
table	0.35	0.71	0.47	90
wardrobe	0.55	0.70	0.61	90
bee	0.48	0.23	0.31	90
beetle	0.75	0.60	0.67	90
butterfly	0.58	0.63	0.60	90
caterpillar	0.47	0.41	0.44	90
cockroach	0.93	0.84	0.88	90
bear	0.77	0.62	0.69	90
leopard	0.86	0.83	0.85	90
lion	0.72	0.54	0.62	90
tiger	0.75	0.66	0.70	90
wolf	0.51	0.51	0.51	90
bridge	0.58	0.41	0.48	90
castle	0.36	0.60	0.45	90
house	0.40	0.76	0.52	90
road	0.74	0.84	0.79	90
skyscraper	0.84	0.78	0.81	90
cloud	0.50	0.42	0.46	90
forest	0.71	0.68	0.69	90
mountain	0.54	0.57	0.55	90
plain	0.95	0.80	0.87	90
sea	0.69	0.69	0.69	90
camel	0.40	0.32	0.36	90
cattle	0.75	0.77	0.76	90
chimpanzee	0.59	0.80	0.68	90

elephant	0.93	0.58	0.71	90
kangaroo	0.73	0.37	0.49	90
fox	0.78	0.81	0.80	90
porcupine	0.64	0.78	0.70	90
possum	0.62	0.61	0.62	90
raccoon	0.64	0.70	0.67	90
skunk	0.48	0.41	0.44	90
crab	0.52	0.48	0.50	90
lobster	0.62	0.62	0.62	90
snail	0.64	0.57	0.60	90
spider	0.83	0.87	0.85	90
worm	0.88	0.82	0.85	90
baby	0.73	0.63	0.68	90
boy	0.71	0.80	0.75	90
girl	0.48	0.39	0.43	90
man	0.56	0.63	0.60	90
woman	0.43	0.29	0.35	90
crocodile	0.72	0.81	0.76	90
dinosaur	0.78	0.88	0.83	90
lizard	0.56	0.66	0.60	90
snake	0.69	0.60	0.64	90
turtle	0.76	0.50	0.60	90
hamster	0.30	0.39	0.34	90
mouse	0.36	0.66	0.46	90
rabbit	0.82	0.93	0.88	90
shrew	0.78	0.66	0.71	90
squirrel	0.59	0.61	0.60	90
maple	0.72	0.77	0.74	90
oak	0.78	0.81	0.80	90
palm	0.71	0.80	0.75	90
pine	0.56	0.62	0.59	90
willow	0.62	0.68	0.65	90
bicycle	0.79	0.37	0.50	90
bus	0.66	0.70	0.68	90
motorcycle	0.56	0.52	0.54	90
pickup truck	0.68	0.56	0.61	90
train	0.79	0.86	0.82	90
lawn-mower	0.64	0.77	0.70	90
rocket	0.49	0.26	0.34	90
streetcar	0.78	0.52	0.63	90
tank	0.45	0.37	0.40	90
tractor	0.78	0.76	0.77	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)



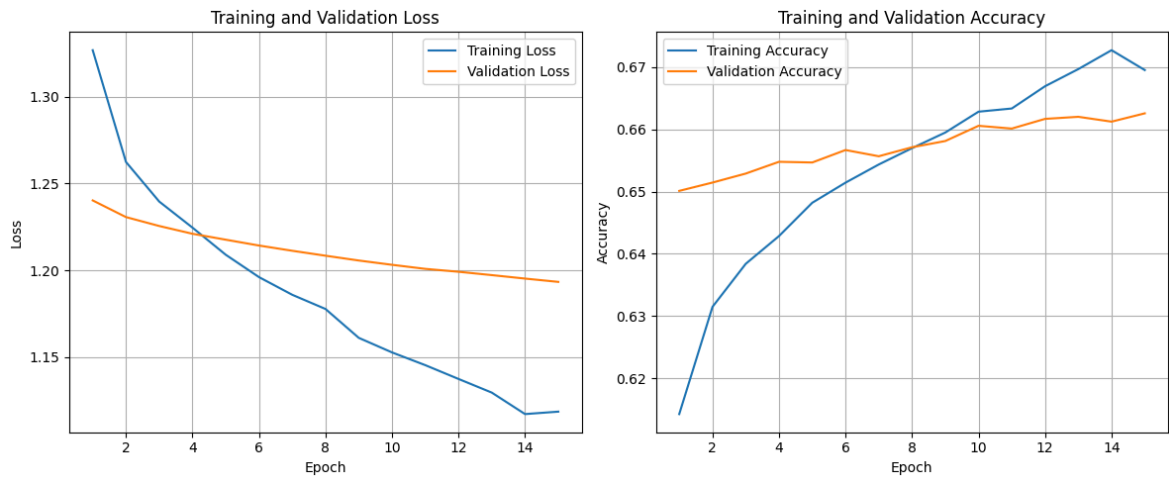
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 26ms/step - accuracy: 0.6028 - f1_score: 0.5997
- loss: 1.3624 - precision: 0.7873
Epoch 1: val_loss improved from inf to 1.24024, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.6028 - f1_score: 0.5997
- loss: 1.3623 - precision: 0.7873 - val_accuracy: 0.6501 - val_f1_score: 0.6480
- val_loss: 1.2402 - val_precision: 0.8298
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6289 - f1_score: 0.6259
- loss: 1.2675 - precision: 0.8066
Epoch 2: val_loss improved from 1.24024 to 1.23065, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6289 - f1_score: 0.6259
- loss: 1.2675 - precision: 0.8066 - val_accuracy: 0.6514 - val_f1_score: 0.6494
- val_loss: 1.2307 - val_precision: 0.8283
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6362 - f1_score: 0.6333
- loss: 1.2384 - precision: 0.8109
Epoch 3: val_loss improved from 1.23065 to 1.22552, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6362 - f1_score: 0.6333
- loss: 1.2384 - precision: 0.8109 - val_accuracy: 0.6529 - val_f1_score: 0.6510
- val_loss: 1.2255 - val_precision: 0.8298
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6429 - f1_score: 0.6398
- loss: 1.2233 - precision: 0.8160
Epoch 4: val_loss improved from 1.22552 to 1.22105, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6429 - f1_score: 0.6398
- loss: 1.2233 - precision: 0.8160 - val_accuracy: 0.6548 - val_f1_score: 0.6529
- val_loss: 1.2210 - val_precision: 0.8301
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6512 - f1_score: 0.6482
- loss: 1.1959 - precision: 0.8225
Epoch 5: val_loss improved from 1.22105 to 1.21763, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6512 - f1_score: 0.6481
- loss: 1.1959 - precision: 0.8225 - val_accuracy: 0.6547 - val_f1_score: 0.6529
- val_loss: 1.2176 - val_precision: 0.8293
Epoch 6/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6519 - f1_score: 0.6489
- loss: 1.1879 - precision: 0.8235
Epoch 6: val_loss improved from 1.21763 to 1.21427, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6519 - f1_score: 0.6489
- loss: 1.1879 - precision: 0.8235 - val_accuracy: 0.6567 - val_f1_score: 0.6549
- val_loss: 1.2143 - val_precision: 0.8313
Epoch 7/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6567 - f1_score: 0.6543
- loss: 1.1783 - precision: 0.8272
Epoch 7: val_loss improved from 1.21427 to 1.21125, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6567 - f1_score: 0.6543
- loss: 1.1783 - precision: 0.8272 - val_accuracy: 0.6557 - val_f1_score: 0.6539
- val_loss: 1.2113 - val_precision: 0.8301
Epoch 8/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6571 - f1_score: 0.6546
- loss: 1.1673 - precision: 0.8252
Epoch 8: val_loss improved from 1.21125 to 1.20841, saving model to best_model_fine_tune_v2.keras

```
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6571 - f1_score: 0.654
6 - loss: 1.1673 - precision: 0.8252 - val_accuracy: 0.6571 - val_f1_score: 0.655
5 - val_loss: 1.2084 - val_precision: 0.8303
Epoch 9/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6627 - f1_score: 0.6599
- loss: 1.1472 - precision: 0.8314
Epoch 9: val_loss improved from 1.20841 to 1.20564, saving model to best_model_fi
ne_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6627 - f1_score: 0.659
9 - loss: 1.1472 - precision: 0.8314 - val_accuracy: 0.6581 - val_f1_score: 0.656
4 - val_loss: 1.2056 - val_precision: 0.8318
Epoch 10/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6648 - f1_score: 0.6624
- loss: 1.1403 - precision: 0.8338
Epoch 10: val_loss improved from 1.20564 to 1.20317, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6648 - f1_score: 0.662
4 - loss: 1.1403 - precision: 0.8338 - val_accuracy: 0.6606 - val_f1_score: 0.659
0 - val_loss: 1.2032 - val_precision: 0.8317
Epoch 11/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6681 - f1_score: 0.6657
- loss: 1.1298 - precision: 0.8365
Epoch 11: val_loss improved from 1.20317 to 1.20084, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6681 - f1_score: 0.665
7 - loss: 1.1298 - precision: 0.8365 - val_accuracy: 0.6601 - val_f1_score: 0.658
6 - val_loss: 1.2008 - val_precision: 0.8315
Epoch 12/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6709 - f1_score: 0.6684
- loss: 1.1275 - precision: 0.8395
Epoch 12: val_loss improved from 1.20084 to 1.19916, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6709 - f1_score: 0.668
4 - loss: 1.1275 - precision: 0.8395 - val_accuracy: 0.6617 - val_f1_score: 0.660
2 - val_loss: 1.1992 - val_precision: 0.8322
Epoch 13/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6729 - f1_score: 0.6702
- loss: 1.1136 - precision: 0.8400
Epoch 13: val_loss improved from 1.19916 to 1.19724, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6729 - f1_score: 0.670
2 - loss: 1.1136 - precision: 0.8400 - val_accuracy: 0.6620 - val_f1_score: 0.660
3 - val_loss: 1.1972 - val_precision: 0.8310
Epoch 14/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.6761 - f1_score: 0.6735
- loss: 1.1024 - precision: 0.8390
Epoch 14: val_loss improved from 1.19724 to 1.19524, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6761 - f1_score: 0.673
5 - loss: 1.1024 - precision: 0.8390 - val_accuracy: 0.6612 - val_f1_score: 0.659
8 - val_loss: 1.1952 - val_precision: 0.8314
Epoch 15/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.6744 - f1_score: 0.6718
- loss: 1.1003 - precision: 0.8428
Epoch 15: val_loss improved from 1.19524 to 1.19334, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.6744 - f1_score: 0.671
8 - loss: 1.1004 - precision: 0.8428 - val_accuracy: 0.6626 - val_f1_score: 0.661
2 - val_loss: 1.1933 - val_precision: 0.8328
```

141/141 ————— **12s** 38ms/step - accuracy: 0.6536 - f1_score: 0.6457

- loss: 1.2064 - precision: 0.8177

Test Accuracy: 66.27%

Test Loss: 1.181408166885376

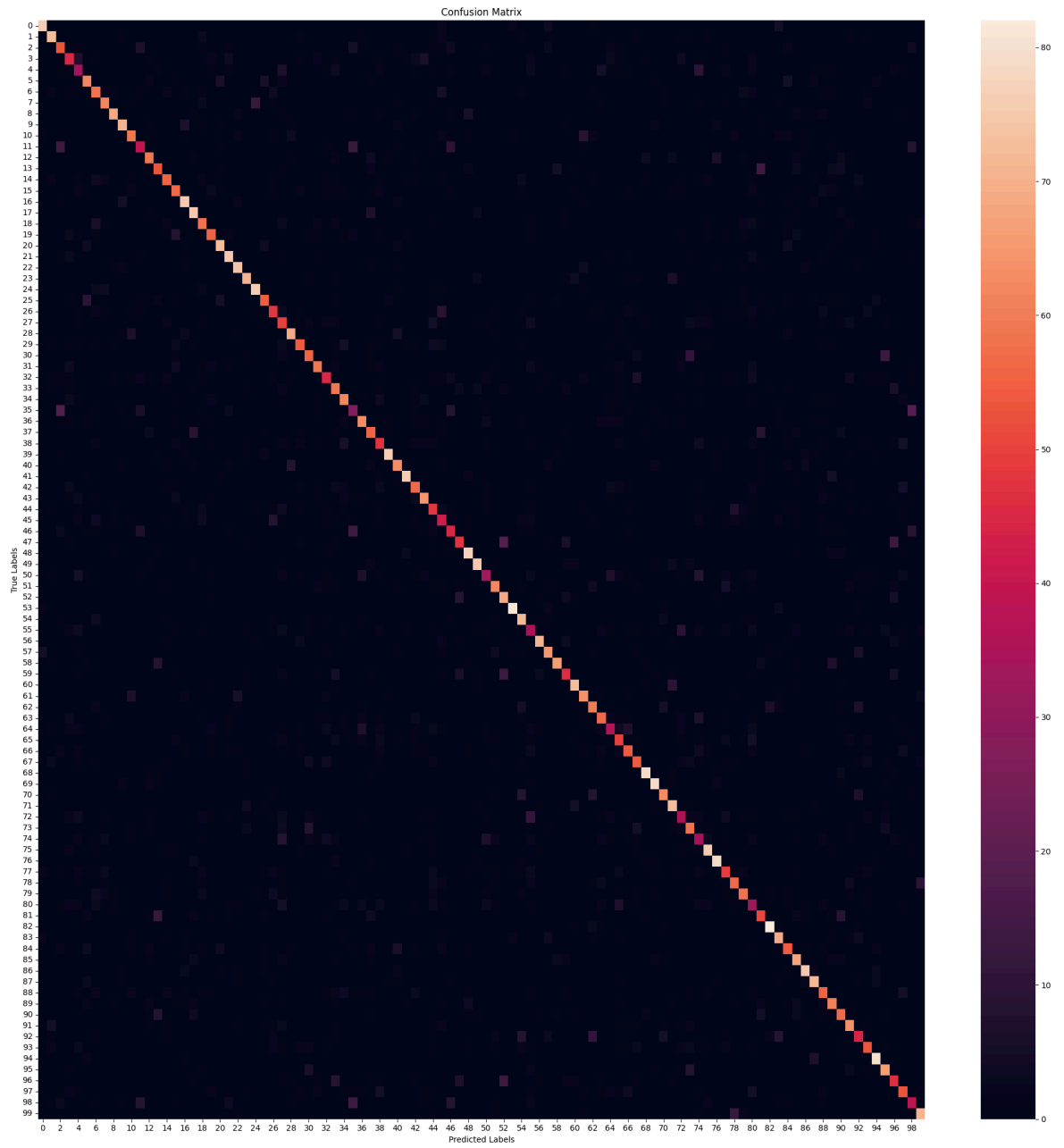
Test Precision: 82.71%

Test F1 Scores (Per Class): [85.39326 78.07486 54.639168 51.136356 38.20224 67.02702 58.883244

71.67629	82.352936	80.22598	67.415726	48.48484	67.816086	59.550552
65.49707	62.569828	77.7202	76.923065	62.365585	65.11627	79.347824
79.365074	80.43478	78.88888	83.060104	66.26506	55.49132	46.29629
77.9661	65.060234	63.636356	69.82249	53.89221	57.843132	67.3913
30.58823	68.852455	61.538452	51.933693	87.86127	69.23077	85.8757
64.04494	73.863625	51.06383	46.408836	51.42857	57.66871	80.628265
82.872925	43.137253	71.67629	66.99028	91.01122	70.243904	38.461533
82.558136	72.625694	79.04191	55.421684	80.21977	72.72727	64.51612
65.14285	46.249996	54.945045	61.363636	61.714287	86.33879	84.946236
72.41379	75.64766	42.682922	62.032085	36.956516	81.28342	85.405396
56.97674	64.04494	67.83625	34.782604	55.737698	89.13043	72.63157
61.363636	71.20418	81.96721	78.68852	62.92134	65.59139	61.290314
68.449196	52.071007	62.352936	84.656075	71.35134	52.51396	60.227264
44.06779	78.02197					

Average Test F1 Scores:66.13

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.86	0.84	0.85	90
dolphin	0.75	0.81	0.78	90
otter	0.51	0.59	0.55	90
seal	0.52	0.50	0.51	90
whale	0.39	0.38	0.38	90
aquarium fish	0.65	0.69	0.67	90
flatfish	0.54	0.64	0.59	90
ray	0.75	0.69	0.72	90
shark	0.87	0.77	0.82	90
trout	0.82	0.79	0.80	90
orchids	0.68	0.66	0.67	90
poppies	0.53	0.44	0.48	90
roses	0.70	0.66	0.68	90
sunflowers	0.60	0.59	0.60	90
tulips	0.69	0.62	0.65	90
bottles	0.63	0.63	0.63	90
bowls	0.73	0.83	0.78	90
cans	0.71	0.83	0.77	90
cups	0.60	0.64	0.62	90
plates	0.69	0.62	0.65	90
apples	0.78	0.81	0.79	90
mushrooms	0.76	0.83	0.79	90
oranges	0.79	0.82	0.80	90
pears	0.79	0.79	0.79	90
sweet peppers	0.82	0.84	0.83	90
clock	0.72	0.61	0.66	90
computer keyboard	0.58	0.53	0.55	90
lamp	0.40	0.56	0.46	90
telephone	0.79	0.77	0.78	90
television	0.71	0.60	0.65	90
bed	0.65	0.62	0.64	90
chair	0.75	0.66	0.70	90
couch	0.58	0.50	0.54	90
table	0.52	0.66	0.58	90
wardrobe	0.66	0.69	0.67	90
bee	0.33	0.29	0.31	90
beetle	0.68	0.70	0.69	90
butterfly	0.61	0.62	0.62	90
caterpillar	0.52	0.52	0.52	90
cockroach	0.92	0.84	0.88	90
bear	0.68	0.70	0.69	90
leopard	0.87	0.84	0.86	90
lion	0.65	0.63	0.64	90
tiger	0.76	0.72	0.74	90
wolf	0.49	0.53	0.51	90
bridge	0.46	0.47	0.46	90
castle	0.53	0.50	0.51	90
house	0.64	0.52	0.58	90
road	0.76	0.86	0.81	90
skyscraper	0.82	0.83	0.83	90
cloud	0.52	0.37	0.43	90
forest	0.75	0.69	0.72	90
mountain	0.59	0.77	0.67	90
plain	0.92	0.90	0.91	90
sea	0.63	0.80	0.70	90
camel	0.38	0.39	0.38	90
cattle	0.87	0.79	0.83	90
chimpanzee	0.73	0.72	0.73	90

elephant	0.86	0.73	0.79	90
kangaroo	0.61	0.51	0.55	90
fox	0.79	0.81	0.80	90
porcupine	0.74	0.71	0.72	90
possum	0.62	0.67	0.65	90
raccoon	0.67	0.63	0.65	90
skunk	0.53	0.41	0.46	90
crab	0.54	0.56	0.55	90
lobster	0.63	0.60	0.61	90
snail	0.64	0.60	0.62	90
spider	0.85	0.88	0.86	90
worm	0.82	0.88	0.85	90
baby	0.75	0.70	0.72	90
boy	0.71	0.81	0.76	90
girl	0.47	0.39	0.43	90
man	0.60	0.64	0.62	90
woman	0.36	0.38	0.37	90
crocodile	0.78	0.84	0.81	90
dinosaur	0.83	0.88	0.85	90
lizard	0.60	0.54	0.57	90
snake	0.65	0.63	0.64	90
turtle	0.72	0.64	0.68	90
hamster	0.34	0.36	0.35	90
mouse	0.55	0.57	0.56	90
rabbit	0.87	0.91	0.89	90
shrew	0.69	0.77	0.73	90
squirrel	0.63	0.60	0.61	90
maple	0.67	0.76	0.71	90
oak	0.81	0.83	0.82	90
palm	0.77	0.80	0.79	90
pine	0.64	0.62	0.63	90
willow	0.64	0.68	0.66	90
bicycle	0.59	0.63	0.61	90
bus	0.66	0.71	0.68	90
motorcycle	0.56	0.49	0.52	90
pickup truck	0.66	0.59	0.62	90
train	0.81	0.89	0.85	90
lawn-mower	0.69	0.73	0.71	90
rocket	0.52	0.51	0.52	90
streetcar	0.61	0.59	0.60	90
tank	0.45	0.43	0.44	90
tractor	0.77	0.79	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

Dropout(0.7), L2Reg(3e-1), weight_decay(3e-2), train_augmentation

```
In [6]: for repeat_2_times in range(2):
        ##### <<<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

        .map(augment_dataset, num_parallel_calls=tf.data.experimental
        .batch(batch_size)
        .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-1) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.7), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                              patience=5, # Num. epochs with no improvement
                              restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                              factor=0.1, # Reduce lr by a factor
#                              patience=3, # Num epochs w/ no improvement
#                              min_lr=1e-6, # Min lr
#                              verbose=1)

#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/acc
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v3.keras', # Path where the best model is saved
                            monitor='val_loss', # What to monitor (validation loss)
                            save_best_only=True, # Only save model when val loss is best
                            mode='min', # Looking to minimize the validation loss
                            verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v3.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model

#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v3.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n")

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```



```

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v3.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```

```
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

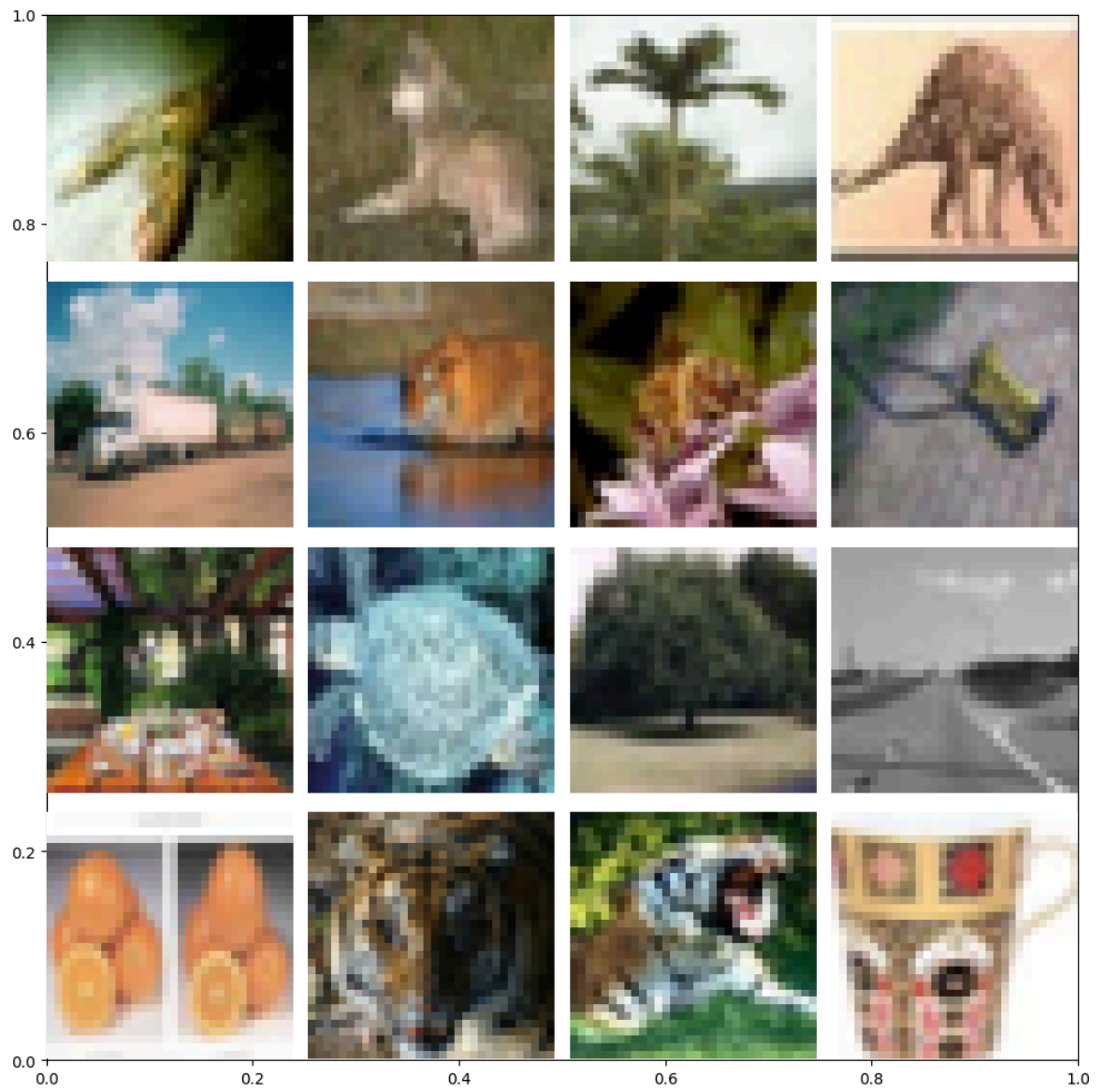
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

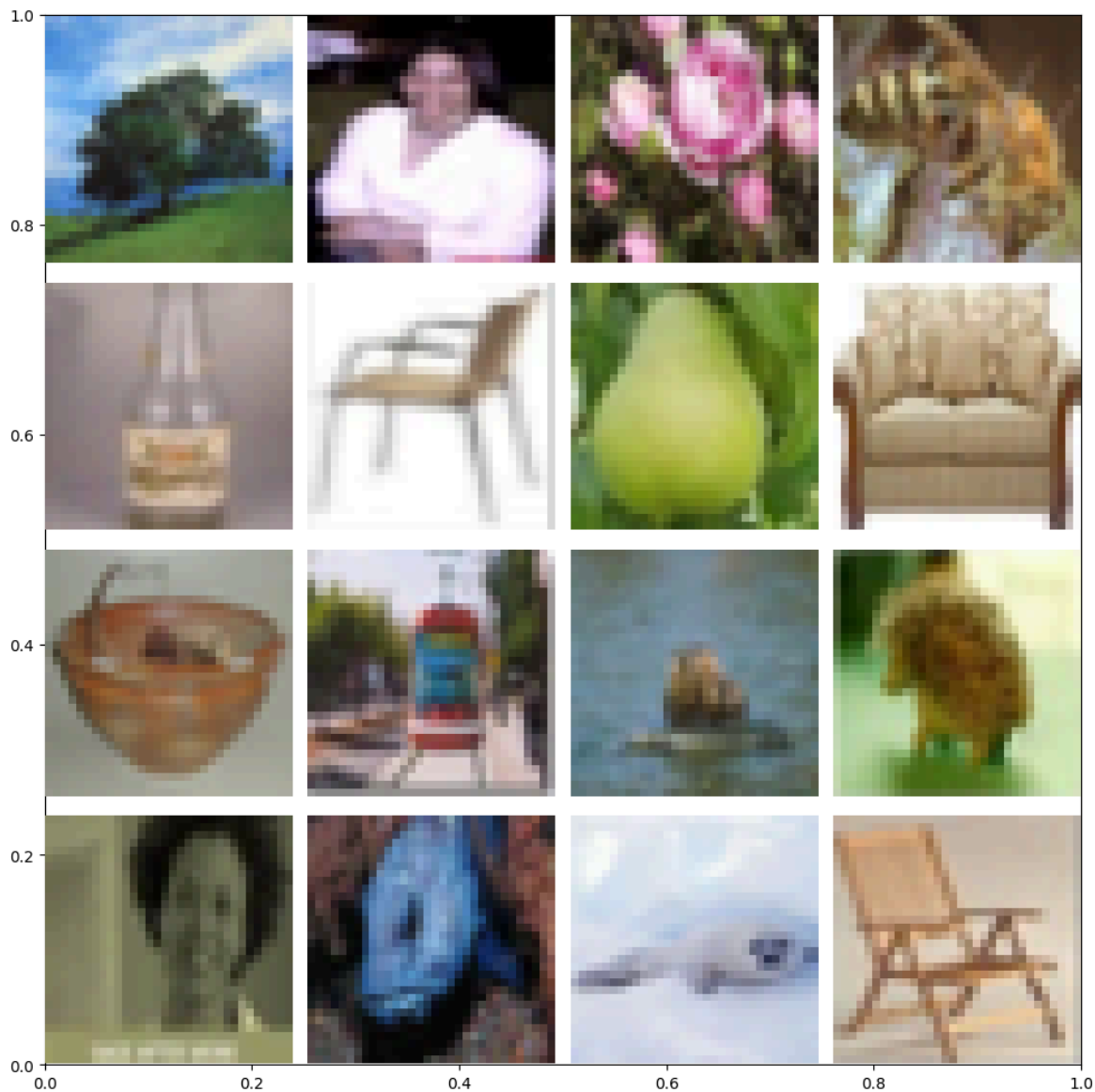
labels[28620].shape: (1,)

imgs[28620].shape: (32, 32, 3)



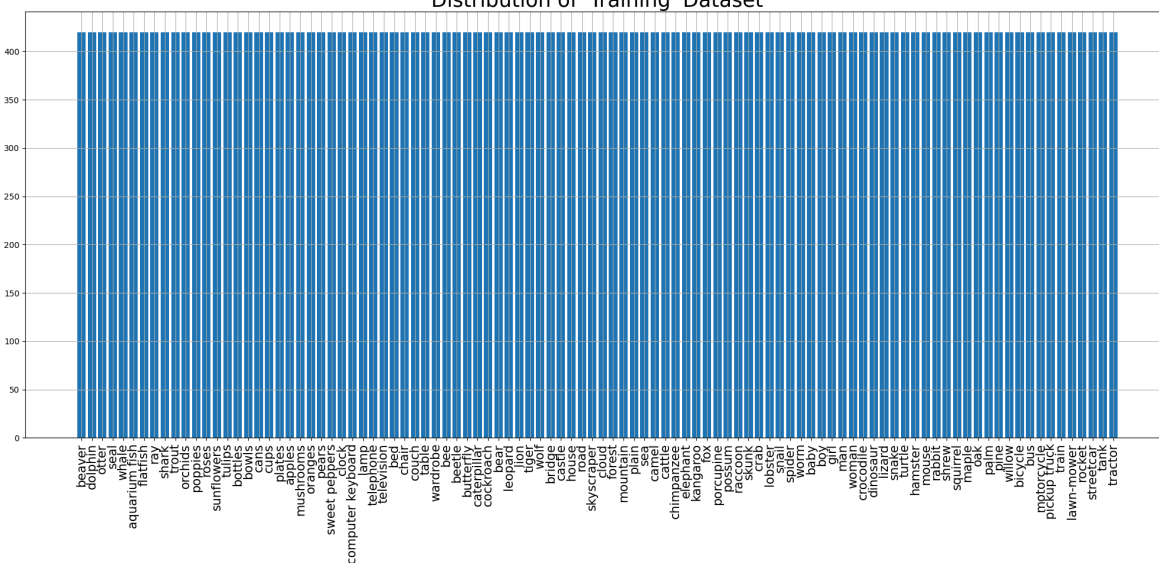
labels[28896].shape: (1,)

imgs[28896].shape: (32, 32, 3)

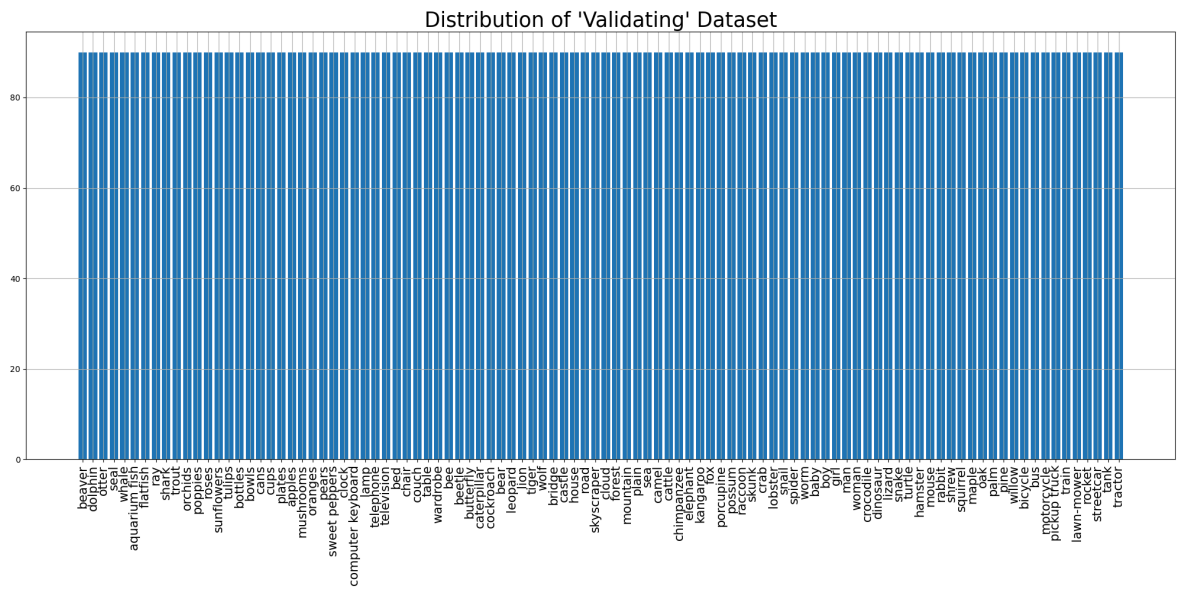


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

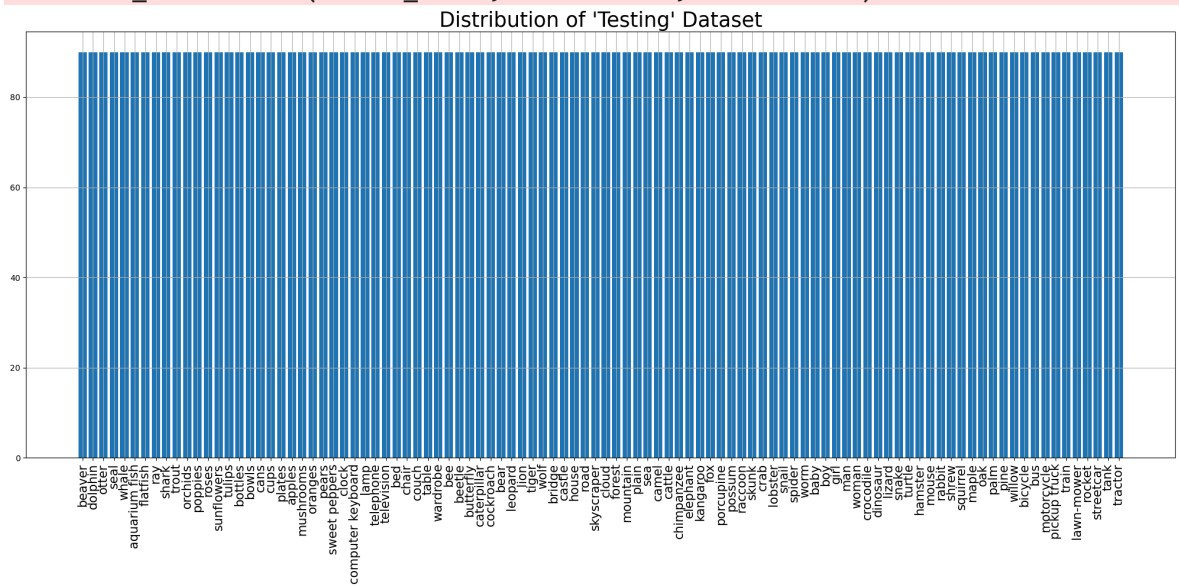
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)

◀  ▶

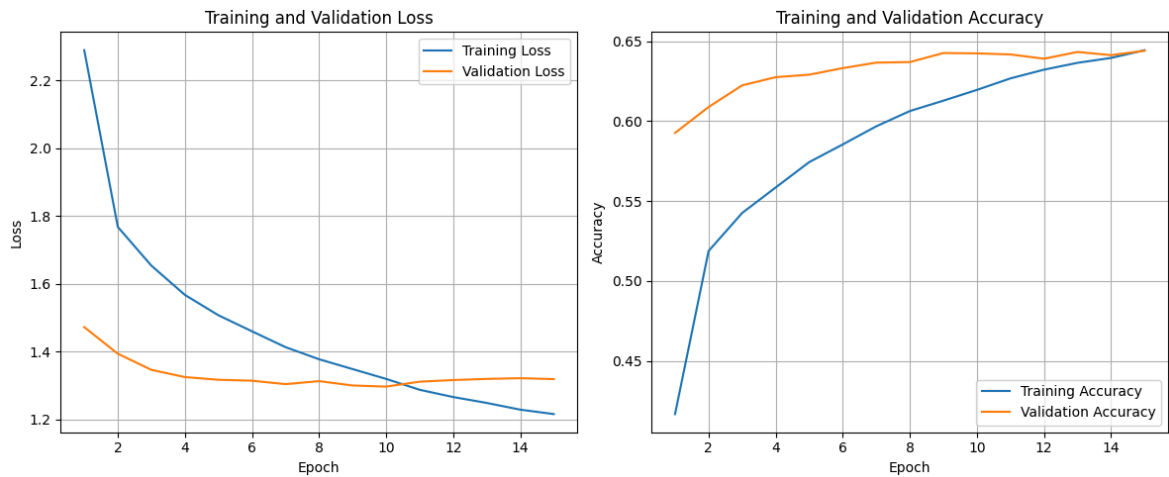
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1314/1314 ————— 0s 26ms/step - accuracy: 0.3248 - f1_score: 0.3191
- loss: 2.8235 - precision: 0.6642
Epoch 1: val_loss improved from inf to 1.47206, saving model to best_model_v3.keras
1314/1314 ————— 56s 34ms/step - accuracy: 0.3248 - f1_score: 0.319
2 - loss: 2.8231 - precision: 0.6642 - val_accuracy: 0.5927 - val_f1_score: 0.590
6 - val_loss: 1.4721 - val_precision: 0.8248
Epoch 2/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.5126 - f1_score: 0.5090
- loss: 1.7989 - precision: 0.7392
Epoch 2: val_loss improved from 1.47206 to 1.39350, saving model to best_model_v
3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5126 - f1_score: 0.509
0 - loss: 1.7988 - precision: 0.7393 - val_accuracy: 0.6089 - val_f1_score: 0.608
2 - val_loss: 1.3935 - val_precision: 0.8197
Epoch 3/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.5408 - f1_score: 0.5377
- loss: 1.6627 - precision: 0.7511
Epoch 3: val_loss improved from 1.39350 to 1.34591, saving model to best_model_v
3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5408 - f1_score: 0.537
7 - loss: 1.6627 - precision: 0.7511 - val_accuracy: 0.6224 - val_f1_score: 0.621
9 - val_loss: 1.3459 - val_precision: 0.8169
Epoch 4/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.5566 - f1_score: 0.5535
- loss: 1.5649 - precision: 0.7595
Epoch 4: val_loss improved from 1.34591 to 1.32453, saving model to best_model_v
3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5566 - f1_score: 0.553
5 - loss: 1.5650 - precision: 0.7595 - val_accuracy: 0.6276 - val_f1_score: 0.625
4 - val_loss: 1.3245 - val_precision: 0.8108
Epoch 5/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.5737 - f1_score: 0.5708
- loss: 1.5001 - precision: 0.7622
Epoch 5: val_loss improved from 1.32453 to 1.31659, saving model to best_model_v
3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5737 - f1_score: 0.570
8 - loss: 1.5001 - precision: 0.7622 - val_accuracy: 0.6291 - val_f1_score: 0.628
9 - val_loss: 1.3166 - val_precision: 0.8080
Epoch 6/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.5872 - f1_score: 0.5848
- loss: 1.4475 - precision: 0.7696
Epoch 6: val_loss improved from 1.31659 to 1.31383, saving model to best_model_v
3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5872 - f1_score: 0.584
8 - loss: 1.4475 - precision: 0.7696 - val_accuracy: 0.6332 - val_f1_score: 0.631
0 - val_loss: 1.3138 - val_precision: 0.8068
Epoch 7/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.5999 - f1_score: 0.5973
- loss: 1.3985 - precision: 0.7785
Epoch 7: val_loss improved from 1.31383 to 1.30369, saving model to best_model_v
3.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.5999 - f1_score: 0.597
3 - loss: 1.3985 - precision: 0.7785 - val_accuracy: 0.6367 - val_f1_score: 0.635
1 - val_loss: 1.3037 - val_precision: 0.8083
Epoch 8/25
1314/1314 ————— 0s 23ms/step - accuracy: 0.6114 - f1_score: 0.6087
- loss: 1.3520 - precision: 0.7798
Epoch 8: val_loss did not improve from 1.30369

1314/1314 ————— **34s** 26ms/step - accuracy: 0.6114 - f1_score: 0.6087 - loss: 1.3520 - precision: 0.7798 - val_accuracy: 0.6370 - val_f1_score: 0.6352 - val_loss: 1.3125 - val_precision: 0.8023
Epoch 9/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6174 - f1_score: 0.6150 - loss: 1.3218 - precision: 0.7822
Epoch 9: val_loss improved from 1.30369 to 1.29969, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.6174 - f1_score: 0.6150 - loss: 1.3219 - precision: 0.7822 - val_accuracy: 0.6427 - val_f1_score: 0.6409 - val_loss: 1.2997 - val_precision: 0.8048
Epoch 10/25
1314/1314 ————— **0s** 23ms/step - accuracy: 0.6279 - f1_score: 0.6254 - loss: 1.2904 - precision: 0.7880
Epoch 10: val_loss improved from 1.29969 to 1.29646, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.6279 - f1_score: 0.6254 - loss: 1.2904 - precision: 0.7879 - val_accuracy: 0.6424 - val_f1_score: 0.6385 - val_loss: 1.2965 - val_precision: 0.8031
Epoch 11/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6333 - f1_score: 0.6311 - loss: 1.2531 - precision: 0.7871
Epoch 11: val_loss did not improve from 1.29646
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6333 - f1_score: 0.6311 - loss: 1.2531 - precision: 0.7870 - val_accuracy: 0.6418 - val_f1_score: 0.6398 - val_loss: 1.3109 - val_precision: 0.7965
Epoch 12/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6407 - f1_score: 0.6385 - loss: 1.2340 - precision: 0.7950
Epoch 12: val_loss did not improve from 1.29646
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6406 - f1_score: 0.6385 - loss: 1.2340 - precision: 0.7950 - val_accuracy: 0.6391 - val_f1_score: 0.6375 - val_loss: 1.3158 - val_precision: 0.7946
Epoch 13/25
1313/1314 ————— **0s** 23ms/step - accuracy: 0.6434 - f1_score: 0.6413 - loss: 1.2112 - precision: 0.7918
Epoch 13: val_loss did not improve from 1.29646
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6434 - f1_score: 0.6413 - loss: 1.2112 - precision: 0.7918 - val_accuracy: 0.6433 - val_f1_score: 0.6422 - val_loss: 1.3192 - val_precision: 0.7910
Epoch 14/25
1312/1314 ————— **0s** 23ms/step - accuracy: 0.6476 - f1_score: 0.6456 - loss: 1.1883 - precision: 0.7958
Epoch 14: val_loss did not improve from 1.29646
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6476 - f1_score: 0.6456 - loss: 1.1884 - precision: 0.7958 - val_accuracy: 0.6413 - val_f1_score: 0.6396 - val_loss: 1.3212 - val_precision: 0.7959
Epoch 15/25
1314/1314 ————— **0s** 23ms/step - accuracy: 0.6527 - f1_score: 0.6510 - loss: 1.1774 - precision: 0.7978
Epoch 15: val_loss did not improve from 1.29646
1314/1314 ————— **34s** 26ms/step - accuracy: 0.6527 - f1_score: 0.6510 - loss: 1.1774 - precision: 0.7978 - val_accuracy: 0.6441 - val_f1_score: 0.6434 - val_loss: 1.3186 - val_precision: 0.7912



141/141 ————— **11s 40ms/step** - accuracy: 0.6289 - f1_score: 0.6183

- loss: 1.3107 - precision: 0.7904

Test Accuracy: 64.31%

Test Loss: 1.2886775732040405

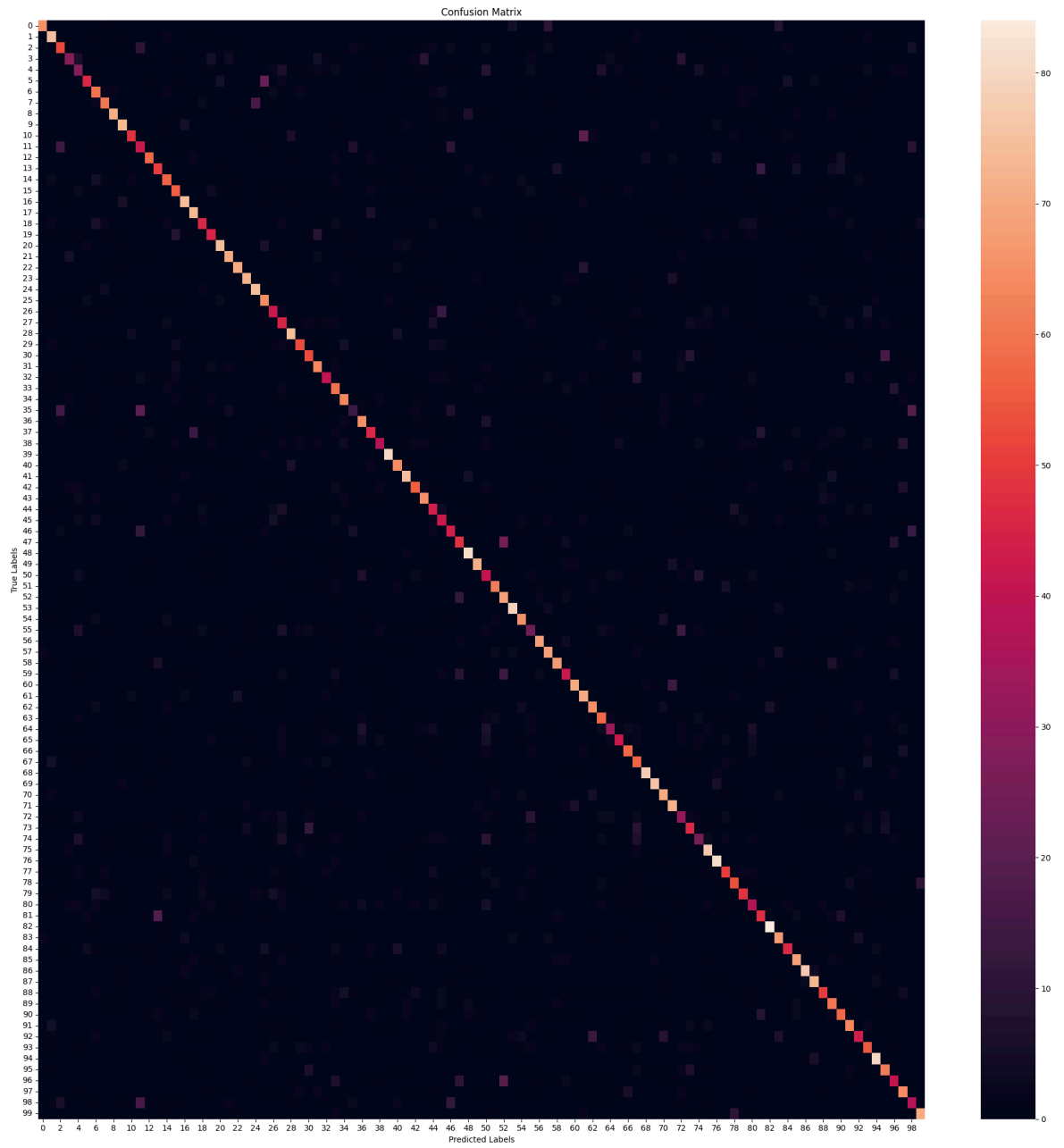
Test Precision: 80.32%

Test F1 Scores (Per Class): [80.74533 75.75757 54.166664 41.428566 32.558136 6
1.33333 59.999996

70.520226	81.60919	78.918915	61.53846	42.424236	71.16564	59.171593
59.999992	61.538452	77.08333	75.64766	54.43787	53.89221	82.22221
77.34807	79.54544	79.55801	78.30688	60.952377	48.837204	44.554447
81.31868	62.352936	58.88888	67.02127	52.564095	54.54545	69.61325
20.96774	69.89246	55.421684	52.41379	90.90909	67.72487	81.52173
64.	71.8232	46.073296	43.29896	50.574715	53.631275	82.653046
81.60919	39.234444	70.45454	62.100452	86.33879	71.03825	28.742514
78.16091	70.46631	76.13636	52.1739	80.	68.29268	71.03825
61.375652	43.83561	53.503185	65.53671	55.072456	86.666664	84.91619
71.428566	75.39267	34.636868	55.75757	32.467525	78.172585	83.50515
58.479523	63.529408	57.142853	41.988945	54.02298	92.30768	69.07216
57.861633	70.83333	86.03351	77.65957	61.349686	60.999996	63.73626
64.2857	55.84415	55.27637	86.48647	66.31015	47.953213	63.681583
41.758232	77.34807	]				

Average Test F1 Scores:63.94

282/282 ————— **11s 28ms/step**



	precision	recall	f1-score	support
beaver	0.92	0.72	0.81	90
dolphin	0.69	0.83	0.76	90
otter	0.51	0.58	0.54	90
seal	0.58	0.32	0.41	90
whale	0.34	0.31	0.33	90
aquarium fish	0.77	0.51	0.61	90
flatfish	0.55	0.67	0.60	90
ray	0.73	0.68	0.71	90
shark	0.85	0.79	0.82	90
trout	0.77	0.81	0.79	90
orchids	0.73	0.53	0.62	90
poppies	0.39	0.47	0.42	90
roses	0.79	0.64	0.71	90
sunflowers	0.63	0.56	0.59	90
tulips	0.57	0.63	0.60	90
bottles	0.61	0.62	0.62	90
bowls	0.73	0.82	0.77	90
cans	0.71	0.81	0.76	90
cups	0.58	0.51	0.54	90
plates	0.58	0.50	0.54	90
apples	0.82	0.82	0.82	90
mushrooms	0.77	0.78	0.77	90
oranges	0.81	0.78	0.80	90
pears	0.79	0.80	0.80	90
sweet peppers	0.75	0.82	0.78	90
clock	0.53	0.71	0.61	90
computer keyboard	0.51	0.47	0.49	90
lamp	0.40	0.50	0.45	90
telephone	0.80	0.82	0.81	90
television	0.66	0.59	0.62	90
bed	0.59	0.59	0.59	90
chair	0.64	0.70	0.67	90
couch	0.62	0.46	0.53	90
table	0.46	0.67	0.55	90
wardrobe	0.69	0.70	0.70	90
bee	0.38	0.14	0.21	90
beetle	0.68	0.72	0.70	90
butterfly	0.61	0.51	0.55	90
caterpillar	0.69	0.42	0.52	90
cockroach	0.93	0.89	0.91	90
bear	0.65	0.71	0.68	90
leopard	0.80	0.83	0.82	90
lion	0.66	0.62	0.64	90
tiger	0.71	0.72	0.72	90
wolf	0.44	0.49	0.46	90
bridge	0.41	0.47	0.44	90
castle	0.52	0.49	0.51	90
house	0.54	0.53	0.54	90
road	0.76	0.90	0.83	90
skyscraper	0.85	0.79	0.82	90
cloud	0.34	0.46	0.39	90
forest	0.72	0.69	0.70	90
mountain	0.53	0.76	0.62	90
plain	0.85	0.88	0.86	90
sea	0.70	0.72	0.71	90
camel	0.31	0.27	0.29	90
cattle	0.81	0.76	0.78	90
chimpanzee	0.66	0.76	0.70	90

elephant	0.78	0.74	0.76	90
kangaroo	0.59	0.47	0.52	90
fox	0.82	0.78	0.80	90
porcupine	0.61	0.78	0.68	90
possum	0.70	0.72	0.71	90
raccoon	0.59	0.64	0.61	90
skunk	0.57	0.36	0.44	90
crab	0.63	0.47	0.54	90
lobster	0.67	0.64	0.66	90
snail	0.49	0.63	0.55	90
spider	0.87	0.87	0.87	90
worm	0.86	0.84	0.85	90
baby	0.66	0.78	0.71	90
boy	0.71	0.80	0.75	90
girl	0.35	0.34	0.35	90
man	0.61	0.51	0.55	90
woman	0.39	0.28	0.32	90
crocodile	0.72	0.86	0.78	90
dinosaur	0.78	0.90	0.84	90
lizard	0.62	0.56	0.58	90
snake	0.68	0.60	0.64	90
turtle	0.62	0.53	0.57	90
hamster	0.42	0.42	0.42	90
mouse	0.56	0.52	0.54	90
rabbit	0.91	0.93	0.92	90
shrew	0.64	0.74	0.69	90
squirrel	0.67	0.51	0.58	90
maple	0.67	0.76	0.71	90
oak	0.87	0.86	0.86	90
palm	0.74	0.81	0.77	90
pine	0.68	0.56	0.61	90
willow	0.55	0.68	0.61	90
bicycle	0.63	0.64	0.64	90
bus	0.59	0.70	0.64	90
motorcycle	0.68	0.48	0.56	90
pickup truck	0.50	0.61	0.55	90
train	0.84	0.89	0.86	90
lawn-mower	0.64	0.69	0.66	90
rocket	0.51	0.46	0.48	90
streetcar	0.58	0.71	0.64	90
tank	0.41	0.42	0.42	90
tractor	0.77	0.78	0.77	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15

1970/1971 ————— **0s** 26ms/step - accuracy: 0.6458 - f1_score: 0.6423
- loss: 1.2123 - precision: 0.8088

Epoch 1: val_loss improved from inf to 1.26976, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **73s** 31ms/step - accuracy: 0.6458 - f1_score: 0.6423 - loss: 1.2123 - precision: 0.8088 - val_accuracy: 0.6533 - val_f1_score: 0.6513 - val_loss: 1.2698 - val_precision: 0.8061

Epoch 2/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.6654 - f1_score: 0.6628 - loss: 1.1321 - precision: 0.8168

Epoch 2: val_loss improved from 1.26976 to 1.26492, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.6654 - f1_score: 0.6628 - loss: 1.1321 - precision: 0.8168 - val_accuracy: 0.6550 - val_f1_score: 0.6533 - val_loss: 1.2649 - val_precision: 0.8024

Epoch 3/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.6717 - f1_score: 0.6698 - loss: 1.1001 - precision: 0.8190

Epoch 3: val_loss improved from 1.26492 to 1.26155, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.6717 - f1_score: 0.6698 - loss: 1.1001 - precision: 0.8190 - val_accuracy: 0.6551 - val_f1_score: 0.6538 - val_loss: 1.2615 - val_precision: 0.8021

Epoch 4/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.6778 - f1_score: 0.6759 - loss: 1.0822 - precision: 0.8216

Epoch 4: val_loss improved from 1.26155 to 1.25875, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.6778 - f1_score: 0.6758 - loss: 1.0823 - precision: 0.8216 - val_accuracy: 0.6570 - val_f1_score: 0.6557 - val_loss: 1.2588 - val_precision: 0.8013

Epoch 5/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.6826 - f1_score: 0.6807 - loss: 1.0694 - precision: 0.8271

Epoch 5: val_loss improved from 1.25875 to 1.25590, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.6826 - f1_score: 0.6807 - loss: 1.0694 - precision: 0.8271 - val_accuracy: 0.6583 - val_f1_score: 0.6571 - val_loss: 1.2559 - val_precision: 0.8016

Epoch 6/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.6869 - f1_score: 0.6852 - loss: 1.0429 - precision: 0.8329

Epoch 6: val_loss improved from 1.25590 to 1.25387, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.6869 - f1_score: 0.6852 - loss: 1.0429 - precision: 0.8329 - val_accuracy: 0.6589 - val_f1_score: 0.6579 - val_loss: 1.2539 - val_precision: 0.8016

Epoch 7/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.6914 - f1_score: 0.6894 - loss: 1.0357 - precision: 0.8330

Epoch 7: val_loss improved from 1.25387 to 1.25260, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **51s** 26ms/step - accuracy: 0.6914 - f1_score: 0.6894 - loss: 1.0357 - precision: 0.8330 - val_accuracy: 0.6591 - val_f1_score: 0.6580 - val_loss: 1.2526 - val_precision: 0.8018

Epoch 8/15

1971/1971 ————— **0s** 24ms/step - accuracy: 0.6899 - f1_score: 0.6880 - loss: 1.0292 - precision: 0.8341

Epoch 8: val_loss improved from 1.25260 to 1.25036, saving model to best_model_fi

ne_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6898 - f1_score: 0.688
0 - loss: 1.0292 - precision: 0.8341 - val_accuracy: 0.6594 - val_f1_score: 0.658
4 - val_loss: 1.2504 - val_precision: 0.8024

Epoch 9/15

1970/1971 ————— 0s 24ms/step - accuracy: 0.6939 - f1_score: 0.6920
- loss: 1.0201 - precision: 0.8349

Epoch 9: val_loss improved from 1.25036 to 1.24839, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6939 - f1_score: 0.692
0 - loss: 1.0201 - precision: 0.8349 - val_accuracy: 0.6604 - val_f1_score: 0.659
5 - val_loss: 1.2484 - val_precision: 0.8017

Epoch 10/15

1970/1971 ————— 0s 24ms/step - accuracy: 0.6948 - f1_score: 0.6931
- loss: 1.0173 - precision: 0.8336

Epoch 10: val_loss improved from 1.24839 to 1.24812, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6948 - f1_score: 0.693
0 - loss: 1.0173 - precision: 0.8336 - val_accuracy: 0.6602 - val_f1_score: 0.659
4 - val_loss: 1.2481 - val_precision: 0.8015

Epoch 11/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.6979 - f1_score: 0.6962
- loss: 1.0005 - precision: 0.8376

Epoch 11: val_loss improved from 1.24812 to 1.24752, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6979 - f1_score: 0.696
2 - loss: 1.0006 - precision: 0.8376 - val_accuracy: 0.6606 - val_f1_score: 0.659
7 - val_loss: 1.2475 - val_precision: 0.8016

Epoch 12/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.7013 - f1_score: 0.6996
- loss: 0.9960 - precision: 0.8420

Epoch 12: val_loss improved from 1.24752 to 1.24532, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.7013 - f1_score: 0.699
6 - loss: 0.9960 - precision: 0.8420 - val_accuracy: 0.6616 - val_f1_score: 0.660
6 - val_loss: 1.2453 - val_precision: 0.8037

Epoch 13/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.7018 - f1_score: 0.7002
- loss: 0.9882 - precision: 0.8437

Epoch 13: val_loss improved from 1.24532 to 1.24505, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.7018 - f1_score: 0.700
2 - loss: 0.9882 - precision: 0.8437 - val_accuracy: 0.6620 - val_f1_score: 0.661
0 - val_loss: 1.2451 - val_precision: 0.8007

Epoch 14/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.7047 - f1_score: 0.7031
- loss: 0.9778 - precision: 0.8442

Epoch 14: val_loss improved from 1.24505 to 1.24431, saving model to best_model_fine_tune_v3.keras

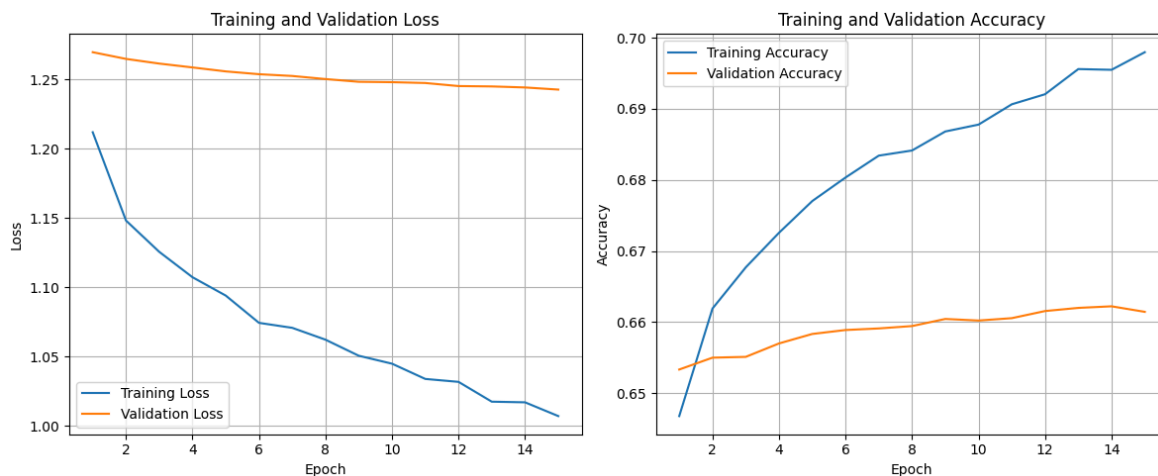
1971/1971 ————— 51s 26ms/step - accuracy: 0.7047 - f1_score: 0.703
0 - loss: 0.9778 - precision: 0.8442 - val_accuracy: 0.6622 - val_f1_score: 0.661
5 - val_loss: 1.2443 - val_precision: 0.8010

Epoch 15/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.7063 - f1_score: 0.7046
- loss: 0.9704 - precision: 0.8433

Epoch 15: val_loss improved from 1.24431 to 1.24277, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.7063 - f1_score: 0.704
6 - loss: 0.9705 - precision: 0.8433 - val_accuracy: 0.6614 - val_f1_score: 0.660
7 - val_loss: 1.2428 - val_precision: 0.8013



141/141 ————— **12s 41ms/step** - accuracy: 0.6501 - f1_score: 0.6414

- loss: 1.2595 - precision: 0.7903

Test Accuracy: 66.28%

Test Loss: 1.2408325672149658

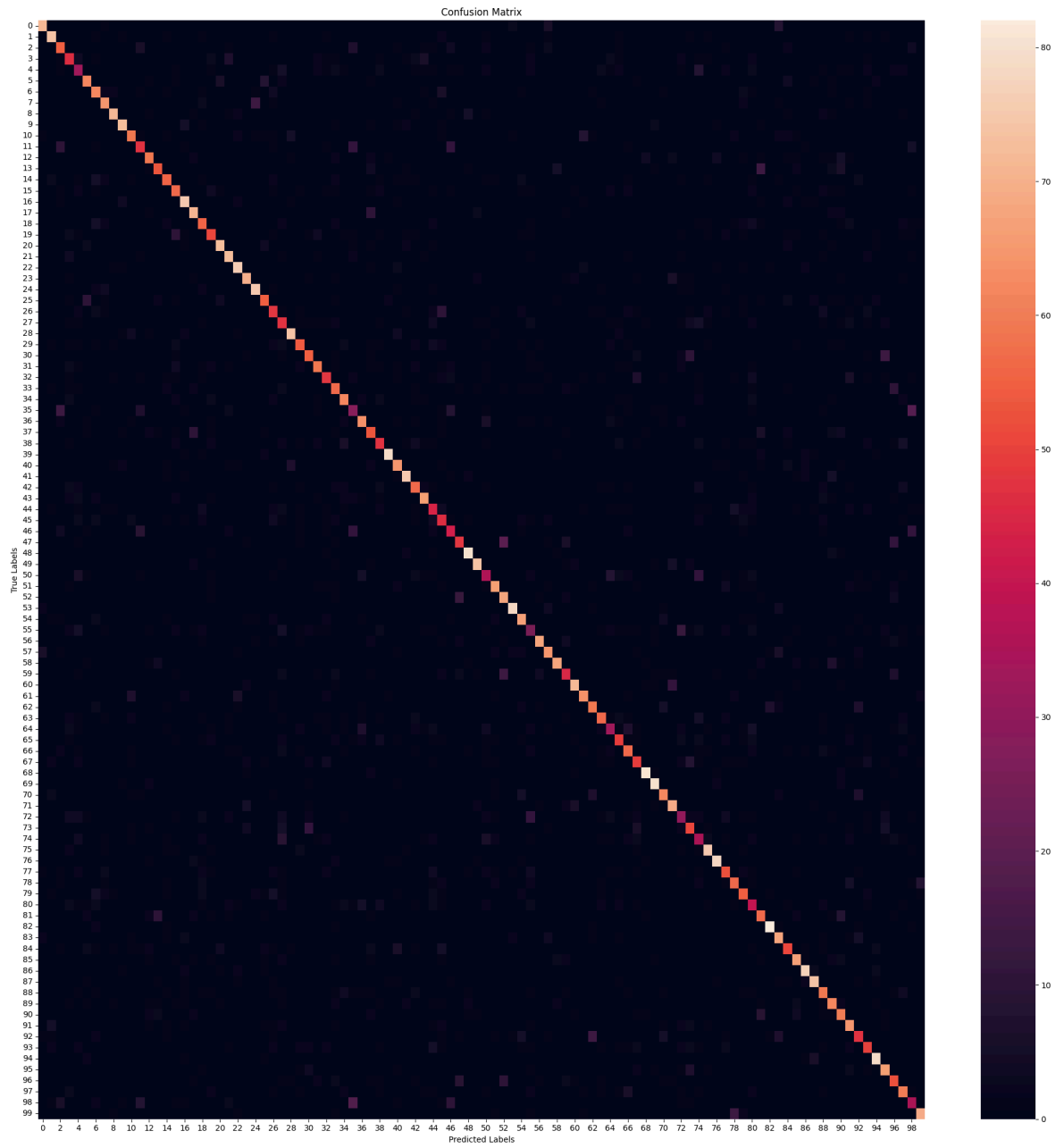
Test Precision: 79.90%

Test F1 Scores (Per Class): [82.285706 79.569885 57.291664 50.27322 37.078648 68.508286 61.306526

72.625694	83.236984	82.95453	67.816086	54.02298	69.005844	64.24241
66.27218	61.62162	78.12499	75.13227	59.893044	58.959538	81.11111
76.43979	80.85106	79.09604	81.52173	64.70587	51.891888	45.63106
82.22221	64.670654	63.5838	69.005844	57.83132	59.067356	67.75956
31.818176	71.50838	57.60869	55.172413	90.28571	69.148926	83.33333
62.983418	74.99999	46.31578	48.42105	48.3146	57.48503	86.02149
84.090904	42.857136	74.99999	66.99506	87.64044	70.526306	30.120478
80.473366	71.428566	79.53216	55.555553	80.89887	74.8538	67.04545
67.85713	42.03821	57.64705	61.290314	54.748596	89.01098	86.95651
70.85714	74.19354	36.363632	54.838707	37.634407	81.52173	86.666664
60.1156	65.517235	61.452507	42.93193	60.63829	89.61748	70.10309
59.999992	74.03315	84.153	81.767944	63.783775	64.58333	61.53846
66.32652	55.294113	58.139526	85.405396	69.47368	54.736835	64.86485
39.77272	79.31035					

Average Test F1 Scores:66.21

282/282 ————— **11s 28ms/step**



	precision	recall	f1-score	support
beaver	0.85	0.80	0.82	90
dolphin	0.77	0.82	0.80	90
otter	0.54	0.61	0.57	90
seal	0.49	0.51	0.50	90
whale	0.38	0.37	0.37	90
aquarium fish	0.68	0.69	0.69	90
flatfish	0.56	0.69	0.62	90
ray	0.73	0.72	0.73	90
shark	0.87	0.80	0.83	90
trout	0.85	0.81	0.83	90
orchids	0.70	0.66	0.68	90
poppies	0.56	0.52	0.54	90
roses	0.73	0.66	0.69	90
sunflowers	0.71	0.59	0.64	90
tulips	0.71	0.62	0.66	90
bottles	0.60	0.63	0.62	90
bowls	0.74	0.83	0.78	90
cans	0.72	0.79	0.75	90
cups	0.58	0.62	0.60	90
plates	0.61	0.57	0.59	90
apples	0.81	0.81	0.81	90
mushrooms	0.72	0.81	0.76	90
oranges	0.78	0.84	0.81	90
pears	0.80	0.78	0.79	90
sweet peppers	0.80	0.83	0.82	90
clock	0.69	0.61	0.65	90
computer keyboard	0.51	0.53	0.52	90
lamp	0.41	0.52	0.46	90
telephone	0.82	0.82	0.82	90
television	0.70	0.60	0.65	90
bed	0.66	0.61	0.64	90
chair	0.73	0.66	0.69	90
couch	0.63	0.53	0.58	90
table	0.55	0.63	0.59	90
wardrobe	0.67	0.69	0.68	90
bee	0.33	0.31	0.32	90
beetle	0.71	0.71	0.71	90
butterfly	0.56	0.59	0.58	90
caterpillar	0.57	0.52	0.54	90
cockroach	0.93	0.88	0.90	90
bear	0.66	0.72	0.69	90
leopard	0.83	0.83	0.83	90
lion	0.62	0.63	0.63	90
tiger	0.78	0.73	0.75	90
wolf	0.44	0.49	0.46	90
bridge	0.46	0.51	0.48	90
castle	0.49	0.48	0.48	90
house	0.62	0.53	0.57	90
road	0.83	0.89	0.86	90
skyscraper	0.86	0.82	0.84	90
cloud	0.46	0.40	0.43	90
forest	0.77	0.73	0.75	90
mountain	0.60	0.76	0.67	90
plain	0.89	0.87	0.88	90
sea	0.67	0.74	0.71	90
camel	0.33	0.28	0.30	90
cattle	0.86	0.76	0.80	90
chimpanzee	0.71	0.72	0.71	90

elephant	0.84	0.76	0.80	90
kangaroo	0.62	0.50	0.56	90
fox	0.82	0.80	0.81	90
porcupine	0.79	0.71	0.75	90
possum	0.69	0.66	0.67	90
raccoon	0.72	0.63	0.67	90
skunk	0.49	0.37	0.42	90
crab	0.61	0.54	0.58	90
lobster	0.59	0.63	0.61	90
snail	0.55	0.54	0.55	90
spider	0.88	0.90	0.89	90
worm	0.85	0.89	0.87	90
baby	0.73	0.69	0.71	90
boy	0.72	0.77	0.74	90
girl	0.40	0.33	0.36	90
man	0.53	0.57	0.55	90
woman	0.36	0.39	0.38	90
crocodile	0.80	0.83	0.82	90
dinosaur	0.87	0.87	0.87	90
lizard	0.63	0.58	0.60	90
snake	0.68	0.63	0.66	90
turtle	0.62	0.61	0.62	90
hamster	0.40	0.44	0.42	90
mouse	0.58	0.63	0.61	90
rabbit	0.88	0.91	0.90	90
shrew	0.65	0.76	0.70	90
squirrel	0.64	0.57	0.60	90
maple	0.74	0.74	0.74	90
oak	0.83	0.86	0.84	90
palm	0.81	0.82	0.82	90
pine	0.62	0.66	0.64	90
willow	0.61	0.69	0.65	90
bicycle	0.57	0.67	0.62	90
bus	0.61	0.72	0.66	90
motorcycle	0.59	0.52	0.55	90
pickup truck	0.61	0.56	0.58	90
train	0.83	0.88	0.85	90
lawn-mower	0.66	0.73	0.69	90
rocket	0.52	0.58	0.55	90
streetcar	0.63	0.67	0.65	90
tank	0.41	0.39	0.40	90
tractor	0.82	0.77	0.79	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

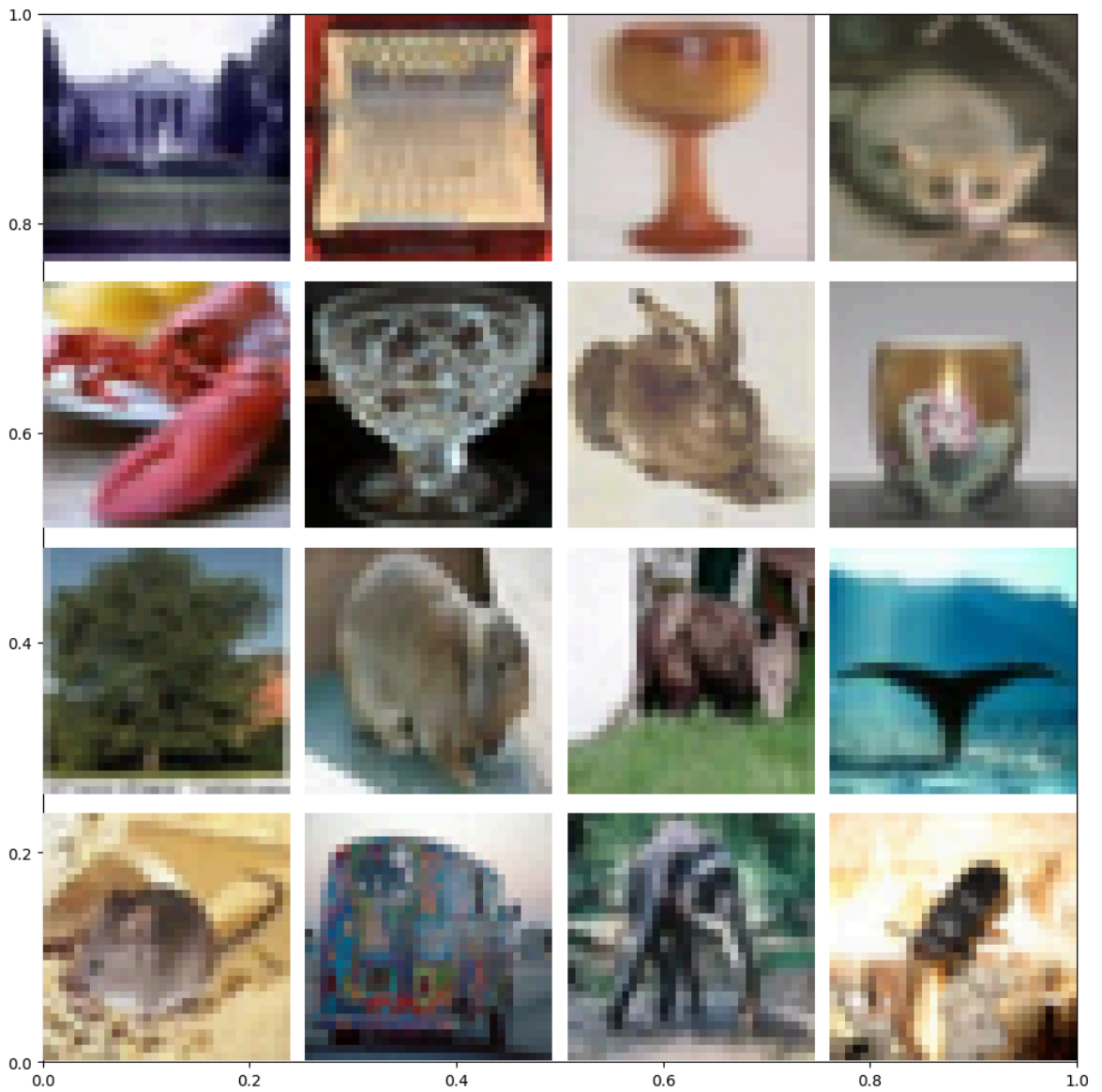
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

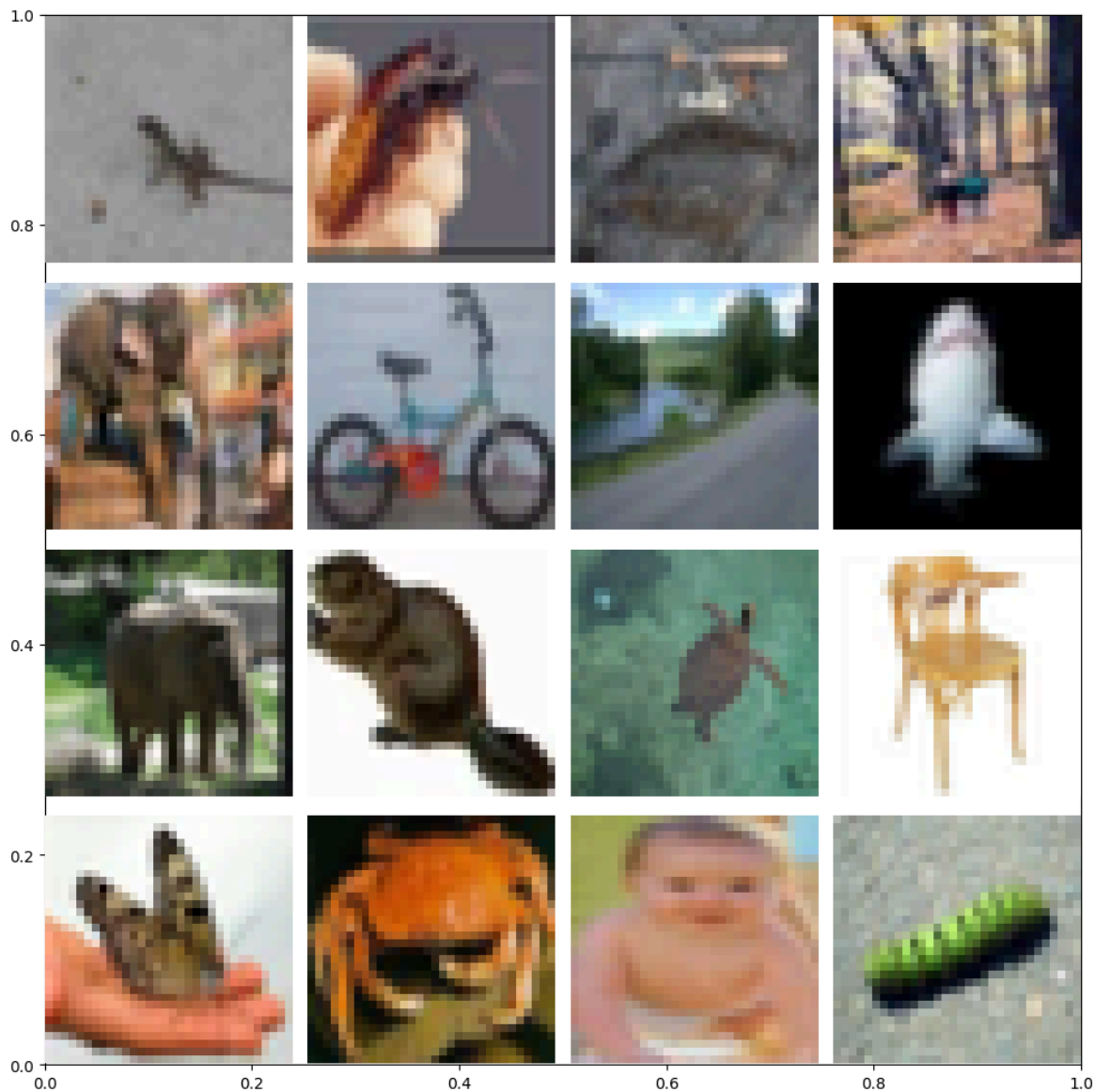
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[21278].shape: (1,)  
imgs[21278].shape: (32, 32, 3)
```

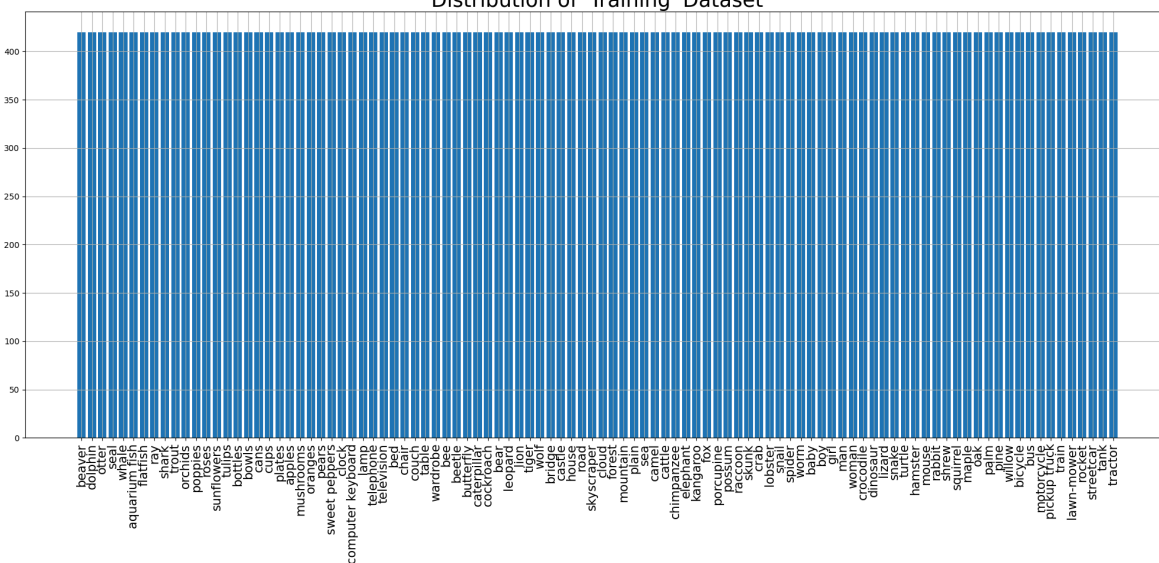


```
labels[10435].shape: (1,)  
imgs[10435].shape: (32, 32, 3)
```

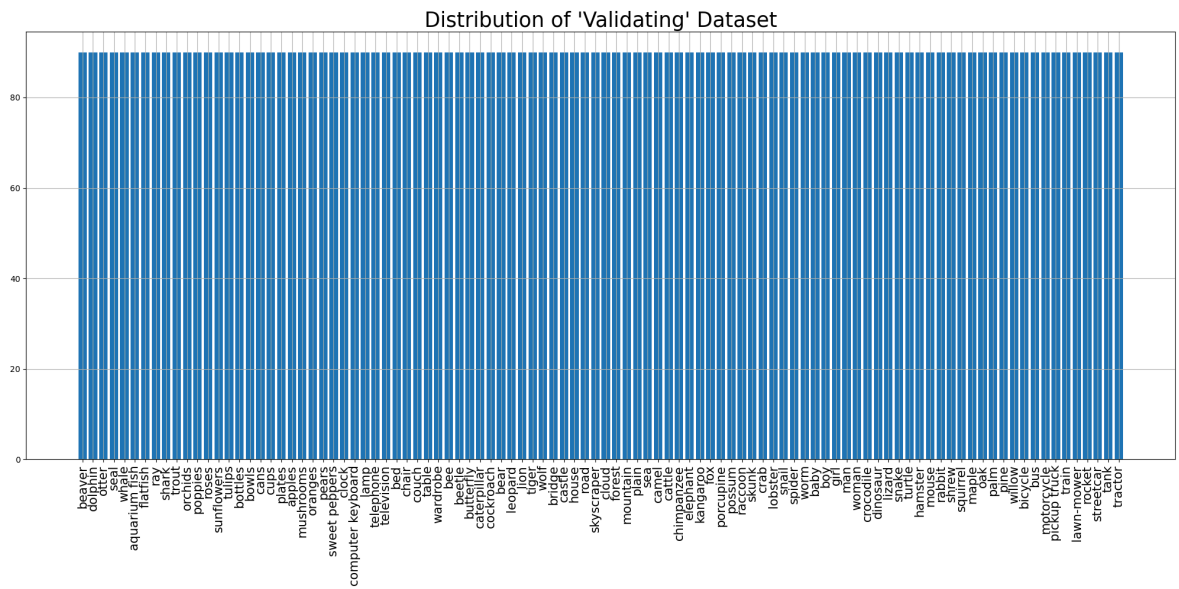


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

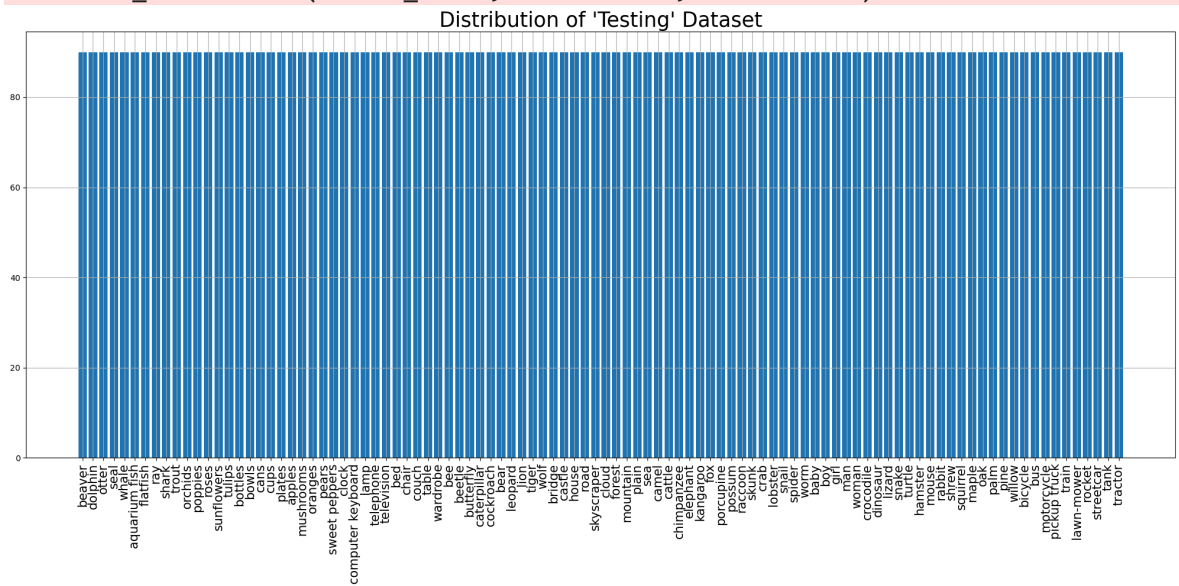
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)

◀  ▶

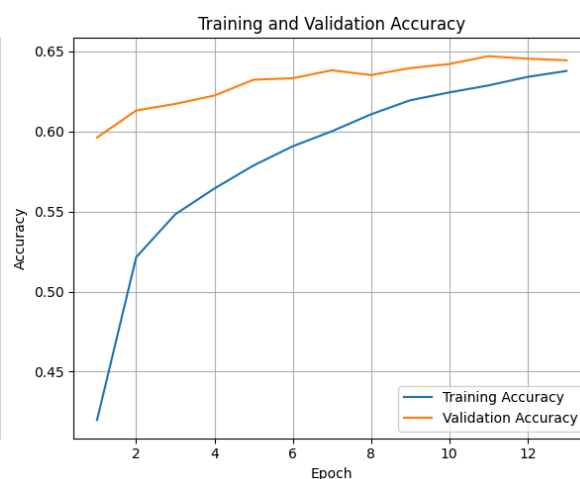
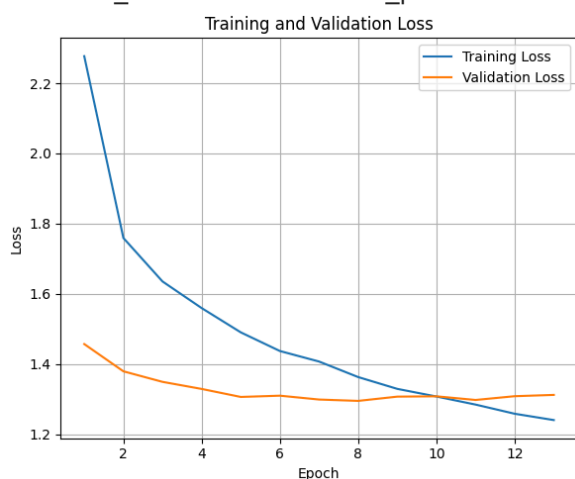
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

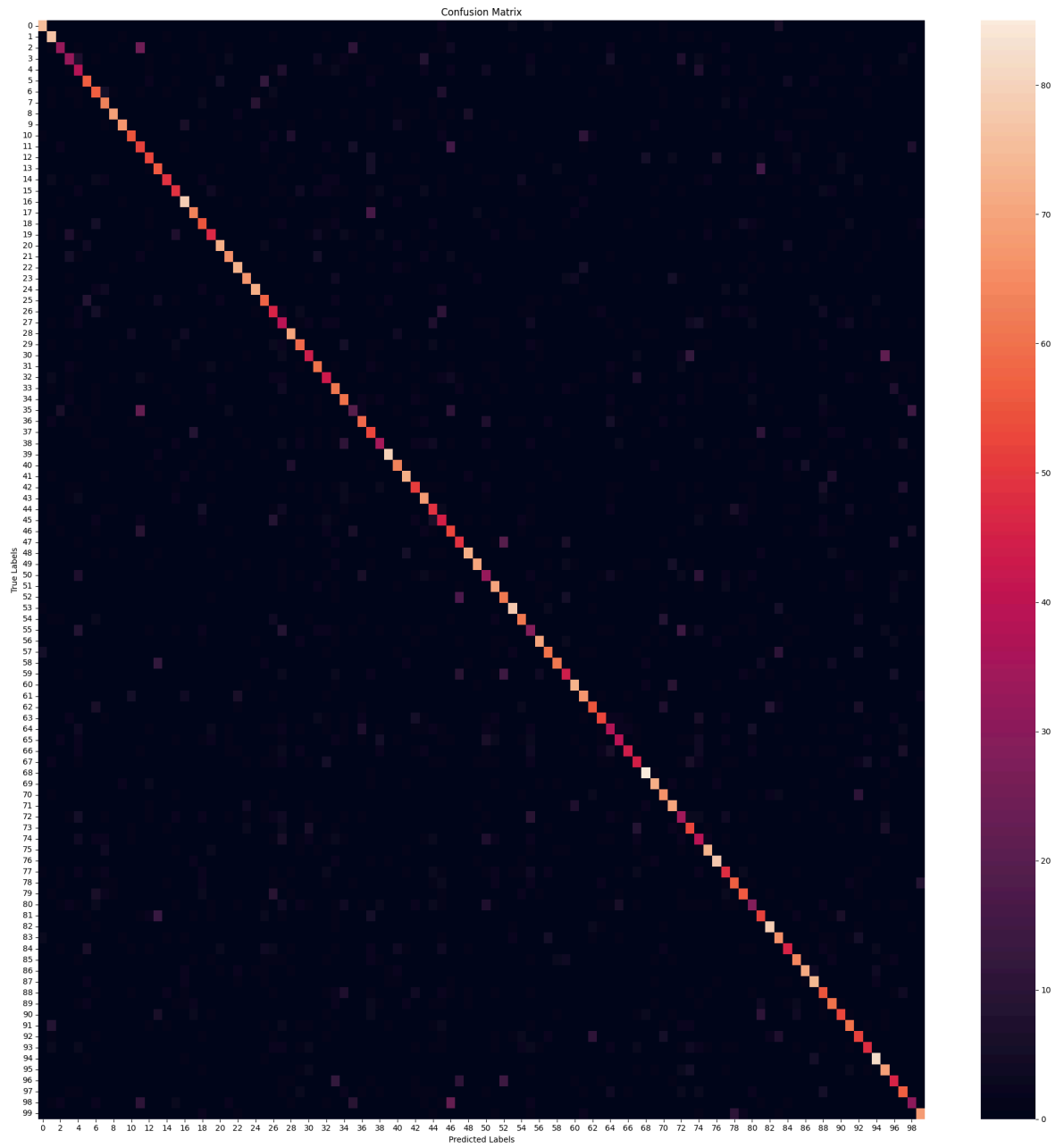
Epoch 1/25
1314/1314 ————— **0s** 27ms/step - accuracy: 0.3316 - f1_score: 0.3258
- loss: 2.7987 - precision: 0.6615
Epoch 1: val_loss improved from inf to 1.45662, saving model to best_model_v3.keras
1314/1314 ————— **56s** 35ms/step - accuracy: 0.3316 - f1_score: 0.3258
9 - loss: 2.7983 - precision: 0.6615 - val_accuracy: 0.5962 - val_f1_score: 0.5915
5 - val_loss: 1.4566 - val_precision: 0.8142
Epoch 2/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.5122 - f1_score: 0.5084
- loss: 1.7962 - precision: 0.7352
Epoch 2: val_loss improved from 1.45662 to 1.37898, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.5122 - f1_score: 0.5084
4 - loss: 1.7961 - precision: 0.7352 - val_accuracy: 0.6131 - val_f1_score: 0.6097
7 - val_loss: 1.3790 - val_precision: 0.8242
Epoch 3/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.5469 - f1_score: 0.5436
- loss: 1.6426 - precision: 0.7561
Epoch 3: val_loss improved from 1.37898 to 1.34892, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.5469 - f1_score: 0.5436
6 - loss: 1.6426 - precision: 0.7561 - val_accuracy: 0.6172 - val_f1_score: 0.6142
2 - val_loss: 1.3489 - val_precision: 0.8079
Epoch 4/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.5632 - f1_score: 0.5602
- loss: 1.5547 - precision: 0.7587
Epoch 4: val_loss improved from 1.34892 to 1.32874, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.5632 - f1_score: 0.5602
2 - loss: 1.5547 - precision: 0.7587 - val_accuracy: 0.6224 - val_f1_score: 0.6206
6 - val_loss: 1.3287 - val_precision: 0.8096
Epoch 5/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.5786 - f1_score: 0.5764
- loss: 1.4863 - precision: 0.7649
Epoch 5: val_loss improved from 1.32874 to 1.30592, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.5786 - f1_score: 0.5764
4 - loss: 1.4863 - precision: 0.7649 - val_accuracy: 0.6323 - val_f1_score: 0.6299
9 - val_loss: 1.3059 - val_precision: 0.8114
Epoch 6/25
1312/1314 ————— **0s** 24ms/step - accuracy: 0.5924 - f1_score: 0.5894
- loss: 1.4187 - precision: 0.7687
Epoch 6: val_loss did not improve from 1.30592
1314/1314 ————— **34s** 26ms/step - accuracy: 0.5924 - f1_score: 0.5894
4 - loss: 1.4187 - precision: 0.7687 - val_accuracy: 0.6332 - val_f1_score: 0.6317
7 - val_loss: 1.3095 - val_precision: 0.8070
Epoch 7/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.6051 - f1_score: 0.6023
- loss: 1.3823 - precision: 0.7792
Epoch 7: val_loss improved from 1.30592 to 1.29855, saving model to best_model_v3.keras
1314/1314 ————— **35s** 27ms/step - accuracy: 0.6051 - f1_score: 0.6023
3 - loss: 1.3824 - precision: 0.7792 - val_accuracy: 0.6382 - val_f1_score: 0.6361
1 - val_loss: 1.2986 - val_precision: 0.8071
Epoch 8/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.6154 - f1_score: 0.6128
- loss: 1.3417 - precision: 0.7847
Epoch 8: val_loss improved from 1.29855 to 1.29467, saving model to best_model_v3.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.6154 - f1_score: 0.6128 - loss: 1.3417 - precision: 0.7847 - val_accuracy: 0.6352 - val_f1_score: 0.6354 - val_loss: 1.2947 - val_precision: 0.8104
 Epoch 9/25
 1314/1314 ————— 0s 24ms/step - accuracy: 0.6246 - f1_score: 0.6224 - loss: 1.3021 - precision: 0.7885
 Epoch 9: val_loss did not improve from 1.29467
 1314/1314 ————— 34s 26ms/step - accuracy: 0.6246 - f1_score: 0.6224 - loss: 1.3021 - precision: 0.7885 - val_accuracy: 0.6396 - val_f1_score: 0.6385 - val_loss: 1.3068 - val_precision: 0.8042
 Epoch 10/25
 1313/1314 ————— 0s 24ms/step - accuracy: 0.6300 - f1_score: 0.6280 - loss: 1.2773 - precision: 0.7885
 Epoch 10: val_loss did not improve from 1.29467
 1314/1314 ————— 34s 26ms/step - accuracy: 0.6300 - f1_score: 0.6280 - loss: 1.2774 - precision: 0.7885 - val_accuracy: 0.6421 - val_f1_score: 0.6412 - val_loss: 1.3077 - val_precision: 0.7957
 Epoch 11/25
 1313/1314 ————— 0s 24ms/step - accuracy: 0.6349 - f1_score: 0.6329 - loss: 1.2478 - precision: 0.7898
 Epoch 11: val_loss did not improve from 1.29467
 1314/1314 ————— 34s 26ms/step - accuracy: 0.6349 - f1_score: 0.6329 - loss: 1.2479 - precision: 0.7898 - val_accuracy: 0.6470 - val_f1_score: 0.6452 - val_loss: 1.2973 - val_precision: 0.8072
 Epoch 12/25
 1312/1314 ————— 0s 23ms/step - accuracy: 0.6416 - f1_score: 0.6394 - loss: 1.2181 - precision: 0.7890
 Epoch 12: val_loss did not improve from 1.29467
 1314/1314 ————— 34s 26ms/step - accuracy: 0.6416 - f1_score: 0.6394 - loss: 1.2182 - precision: 0.7890 - val_accuracy: 0.6454 - val_f1_score: 0.6447 - val_loss: 1.3080 - val_precision: 0.8032
 Epoch 13/25
 1312/1314 ————— 0s 24ms/step - accuracy: 0.6460 - f1_score: 0.6444 - loss: 1.2000 - precision: 0.7904
 Epoch 13: val_loss did not improve from 1.29467
 1314/1314 ————— 34s 26ms/step - accuracy: 0.6459 - f1_score: 0.6444 - loss: 1.2001 - precision: 0.7904 - val_accuracy: 0.6444 - val_f1_score: 0.6435 - val_loss: 1.3117 - val_precision: 0.7927



141/141  **11s** 38ms/step - accuracy: 0.6260 - f1_score: 0.6189
- loss: 1.3173 - precision: 0.7930
Test Accuracy: 63.57%
Test Loss: 1.2967246770858765
Test Precision: 80.34%
Test F1 Scores (Per Class): [85.22727 75.49018 45.390064 41.290318 38.19095 6
5.89595 57.868015
69.23077 82.142845 82.71605 69.18238 47.27272 63.855415 60.21505
65.771805 57.64705 81.44328 73.68421 59.7826 56.6265 82.285706
76.300575 80.21977 78.82352 82.285706 61.375652 47.422672 36.448593
76.50272 65.19337 55.6962 64.86485 49.42528 56.744183 60.60605
26.027393 66.29213 50.7177 45.161285 90.39548 68.852455 79.12087
58.620686 70.83333 51.578945 46.15384 49.999992 54.143642 80.89887
81.39534 37.209297 72.16493 60.696507 86.18784 68.926544 31.111107
80. 69.36415 75.30864 53.012043 80.66298 72.43243 63.5838
66.25 42.04545 50.649338 62.41134 51.72413 87.17948 80.446915
68.04123 74.074066 37.569054 56.382973 37.254898 81.11111 84.32432
53.631275 66.27907 61.87845 34.355824 51.999992 85.56149 66.
56.097557 69.565216 80.681816 78.723404 56.701027 64.51612 61.9883
65.19337 54.736835 58.895695 86.31578 67.980286 51.685387 60.31745
38.216557 79.069756]
Average Test F1 Scores:63.54

282/282  **11s** 27ms/step



	precision	recall	f1-score	support
beaver	0.87	0.83	0.85	90
dolphin	0.68	0.86	0.75	90
otter	0.63	0.36	0.45	90
seal	0.49	0.36	0.41	90
whale	0.35	0.42	0.38	90
aquarium fish	0.69	0.63	0.66	90
flatfish	0.53	0.63	0.58	90
ray	0.68	0.70	0.69	90
shark	0.88	0.77	0.82	90
trout	0.93	0.74	0.83	90
orchids	0.80	0.61	0.69	90
poppies	0.40	0.58	0.47	90
roses	0.70	0.59	0.64	90
sunflowers	0.58	0.62	0.60	90
tulips	0.83	0.54	0.66	90
bottles	0.61	0.54	0.58	90
bowls	0.76	0.88	0.81	90
cans	0.78	0.70	0.74	90
cups	0.59	0.61	0.60	90
plates	0.62	0.52	0.57	90
apples	0.85	0.80	0.82	90
mushrooms	0.80	0.73	0.76	90
oranges	0.79	0.81	0.80	90
pears	0.84	0.74	0.79	90
sweet peppers	0.85	0.80	0.82	90
clock	0.59	0.64	0.61	90
computer keyboard	0.44	0.51	0.47	90
lamp	0.31	0.43	0.36	90
telephone	0.75	0.78	0.77	90
television	0.65	0.66	0.65	90
bed	0.65	0.49	0.56	90
chair	0.63	0.67	0.65	90
couch	0.51	0.48	0.49	90
table	0.48	0.68	0.56	90
wardrobe	0.56	0.67	0.61	90
bee	0.34	0.21	0.26	90
beetle	0.67	0.66	0.66	90
butterfly	0.45	0.59	0.51	90
caterpillar	0.54	0.39	0.45	90
cockroach	0.92	0.89	0.90	90
bear	0.68	0.70	0.69	90
leopard	0.78	0.80	0.79	90
lion	0.61	0.57	0.59	90
tiger	0.67	0.76	0.71	90
wolf	0.49	0.54	0.52	90
bridge	0.43	0.50	0.46	90
castle	0.43	0.59	0.50	90
house	0.54	0.54	0.54	90
road	0.82	0.80	0.81	90
skyscraper	0.85	0.78	0.81	90
cloud	0.39	0.36	0.37	90
forest	0.67	0.78	0.72	90
mountain	0.55	0.68	0.61	90
plain	0.86	0.87	0.86	90
sea	0.70	0.68	0.69	90
camel	0.31	0.31	0.31	90
cattle	0.82	0.78	0.80	90
chimpanzee	0.72	0.67	0.69	90

elephant	0.85	0.68	0.75	90
kangaroo	0.58	0.49	0.53	90
fox	0.80	0.81	0.81	90
porcupine	0.71	0.74	0.72	90
possum	0.66	0.61	0.64	90
raccoon	0.76	0.59	0.66	90
skunk	0.43	0.41	0.42	90
crab	0.61	0.43	0.51	90
lobster	0.86	0.49	0.62	90
snail	0.54	0.50	0.52	90
spider	0.81	0.94	0.87	90
worm	0.81	0.80	0.80	90
baby	0.63	0.73	0.68	90
boy	0.71	0.78	0.74	90
girl	0.37	0.38	0.38	90
man	0.55	0.59	0.57	90
woman	0.34	0.43	0.38	90
crocodile	0.81	0.81	0.81	90
dinosaur	0.82	0.87	0.84	90
lizard	0.54	0.53	0.54	90
snake	0.70	0.63	0.66	90
turtle	0.62	0.62	0.62	90
hamster	0.38	0.31	0.34	90
mouse	0.47	0.58	0.52	90
rabbit	0.82	0.88	0.85	90
shrew	0.60	0.73	0.66	90
squirrel	0.62	0.51	0.56	90
maple	0.68	0.71	0.70	90
oak	0.83	0.79	0.81	90
palm	0.76	0.82	0.79	90
pine	0.53	0.61	0.57	90
willow	0.62	0.67	0.65	90
bicycle	0.65	0.59	0.62	90
bus	0.65	0.67	0.66	90
motorcycle	0.52	0.58	0.55	90
pickup truck	0.66	0.53	0.59	90
train	0.82	0.91	0.86	90
lawn-mower	0.61	0.77	0.68	90
rocket	0.52	0.51	0.52	90
streetcar	0.58	0.63	0.60	90
tank	0.45	0.33	0.38	90
tractor	0.83	0.76	0.79	90
accuracy			0.64	9000
macro avg	0.64	0.64	0.64	9000
weighted avg	0.64	0.64	0.64	9000

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1969/1971 ————— **0s** 25ms/step - accuracy: 0.6336 - f1_score: 0.6320
- loss: 1.2577 - precision: 0.7988
Epoch 1: val_loss improved from inf to 1.26642, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **72s** 31ms/step - accuracy: 0.6336 - f1_score: 0.6320 - loss: 1.2577 - precision: 0.7988 - val_accuracy: 0.6477 - val_f1_score: 0.6470 - val_loss: 1.2664 - val_precision: 0.8087
Epoch 2/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.6532 - f1_score: 0.6513
- loss: 1.1760 - precision: 0.8138
Epoch 2: val_loss improved from 1.26642 to 1.26203, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.6532 - f1_score: 0.6513 - loss: 1.1761 - precision: 0.8138 - val_accuracy: 0.6511 - val_f1_score: 0.6505 - val_loss: 1.2620 - val_precision: 0.8076
Epoch 3/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.6665 - f1_score: 0.6645
- loss: 1.1375 - precision: 0.8177
Epoch 3: val_loss improved from 1.26203 to 1.25896, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.6665 - f1_score: 0.6645 - loss: 1.1376 - precision: 0.8177 - val_accuracy: 0.6527 - val_f1_score: 0.6518 - val_loss: 1.2590 - val_precision: 0.8073
Epoch 4/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.6677 - f1_score: 0.6657
- loss: 1.1192 - precision: 0.8155
Epoch 4: val_loss improved from 1.25896 to 1.25578, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.6677 - f1_score: 0.6657 - loss: 1.1193 - precision: 0.8155 - val_accuracy: 0.6551 - val_f1_score: 0.6542 - val_loss: 1.2558 - val_precision: 0.8072
Epoch 5/15
1971/1971 ————— **0s** 24ms/step - accuracy: 0.6727 - f1_score: 0.6709
- loss: 1.1056 - precision: 0.8200
Epoch 5: val_loss improved from 1.25578 to 1.25389, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.6727 - f1_score: 0.6709 - loss: 1.1056 - precision: 0.8200 - val_accuracy: 0.6561 - val_f1_score: 0.6551 - val_loss: 1.2539 - val_precision: 0.8075
Epoch 6/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.6742 - f1_score: 0.6724
- loss: 1.0933 - precision: 0.8240
Epoch 6: val_loss improved from 1.25389 to 1.25136, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.6742 - f1_score: 0.6724 - loss: 1.0934 - precision: 0.8240 - val_accuracy: 0.6574 - val_f1_score: 0.6563 - val_loss: 1.2514 - val_precision: 0.8076
Epoch 7/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.6815 - f1_score: 0.6795
- loss: 1.0781 - precision: 0.8282
Epoch 7: val_loss improved from 1.25136 to 1.24892, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **51s** 26ms/step - accuracy: 0.6815 - f1_score: 0.6795 - loss: 1.0781 - precision: 0.8281 - val_accuracy: 0.6592 - val_f1_score: 0.6581 - val_loss: 1.2489 - val_precision: 0.8073
Epoch 8/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.6798 - f1_score: 0.6777
- loss: 1.0681 - precision: 0.8297
Epoch 8: val_loss improved from 1.24892 to 1.24736, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6798 - f1_score: 0.677
7 - loss: 1.0681 - precision: 0.8297 - val_accuracy: 0.6597 - val_f1_score: 0.658
3 - val_loss: 1.2474 - val_precision: 0.8092

Epoch 9/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.6839 - f1_score: 0.6818
- loss: 1.0600 - precision: 0.8306

Epoch 9: val_loss improved from 1.24736 to 1.24584, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6839 - f1_score: 0.681
8 - loss: 1.0600 - precision: 0.8306 - val_accuracy: 0.6608 - val_f1_score: 0.659
7 - val_loss: 1.2458 - val_precision: 0.8104

Epoch 10/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.6866 - f1_score: 0.6849
- loss: 1.0540 - precision: 0.8344

Epoch 10: val_loss improved from 1.24584 to 1.24312, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6866 - f1_score: 0.684
9 - loss: 1.0540 - precision: 0.8344 - val_accuracy: 0.6609 - val_f1_score: 0.659
8 - val_loss: 1.2431 - val_precision: 0.8106

Epoch 11/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.6898 - f1_score: 0.6880
- loss: 1.0441 - precision: 0.8344

Epoch 11: val_loss improved from 1.24312 to 1.24255, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6898 - f1_score: 0.688
0 - loss: 1.0441 - precision: 0.8344 - val_accuracy: 0.6609 - val_f1_score: 0.659
7 - val_loss: 1.2426 - val_precision: 0.8083

Epoch 12/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.6910 - f1_score: 0.6892
- loss: 1.0348 - precision: 0.8337

Epoch 12: val_loss improved from 1.24255 to 1.24129, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6910 - f1_score: 0.689
2 - loss: 1.0348 - precision: 0.8337 - val_accuracy: 0.6609 - val_f1_score: 0.659
7 - val_loss: 1.2413 - val_precision: 0.8094

Epoch 13/15

1970/1971 ————— 0s 24ms/step - accuracy: 0.6930 - f1_score: 0.6911
- loss: 1.0289 - precision: 0.8390

Epoch 13: val_loss improved from 1.24129 to 1.23990, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6930 - f1_score: 0.691
1 - loss: 1.0289 - precision: 0.8390 - val_accuracy: 0.6612 - val_f1_score: 0.659
8 - val_loss: 1.2399 - val_precision: 0.8094

Epoch 14/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.6963 - f1_score: 0.6944
- loss: 1.0160 - precision: 0.8388

Epoch 14: val_loss improved from 1.23990 to 1.23864, saving model to best_model_fine_tune_v3.keras

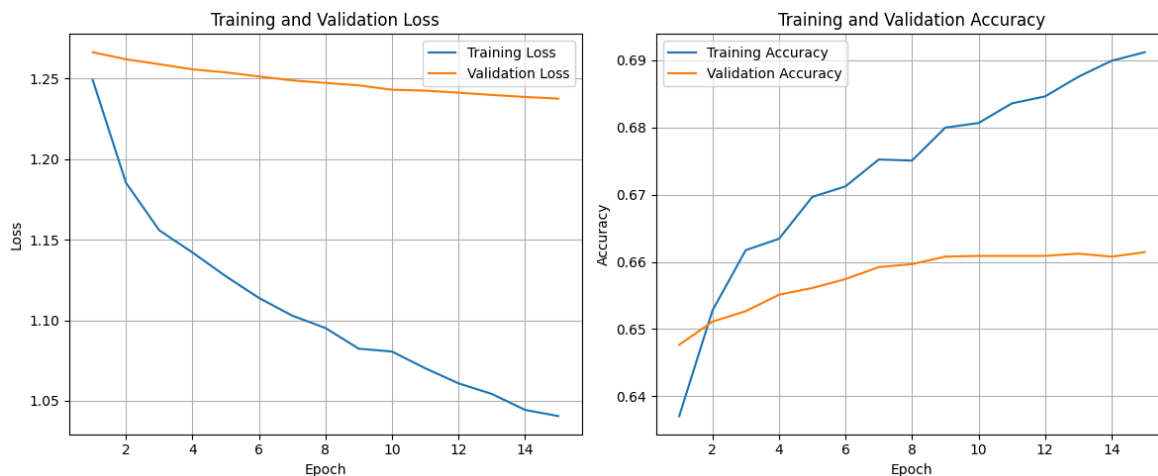
1971/1971 ————— 51s 26ms/step - accuracy: 0.6963 - f1_score: 0.694
4 - loss: 1.0160 - precision: 0.8388 - val_accuracy: 0.6608 - val_f1_score: 0.659
7 - val_loss: 1.2386 - val_precision: 0.8093

Epoch 15/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.6989 - f1_score: 0.6972
- loss: 1.0108 - precision: 0.8417

Epoch 15: val_loss improved from 1.23864 to 1.23760, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 51s 26ms/step - accuracy: 0.6988 - f1_score: 0.697
2 - loss: 1.0109 - precision: 0.8417 - val_accuracy: 0.6614 - val_f1_score: 0.660
1 - val_loss: 1.2376 - val_precision: 0.8105



141/141 ————— **11s** 38ms/step - accuracy: 0.6479 - f1_score: 0.6408

- loss: 1.2601 - precision: 0.7880

Test Accuracy: 65.73%

Test Loss: 1.2411272525787354

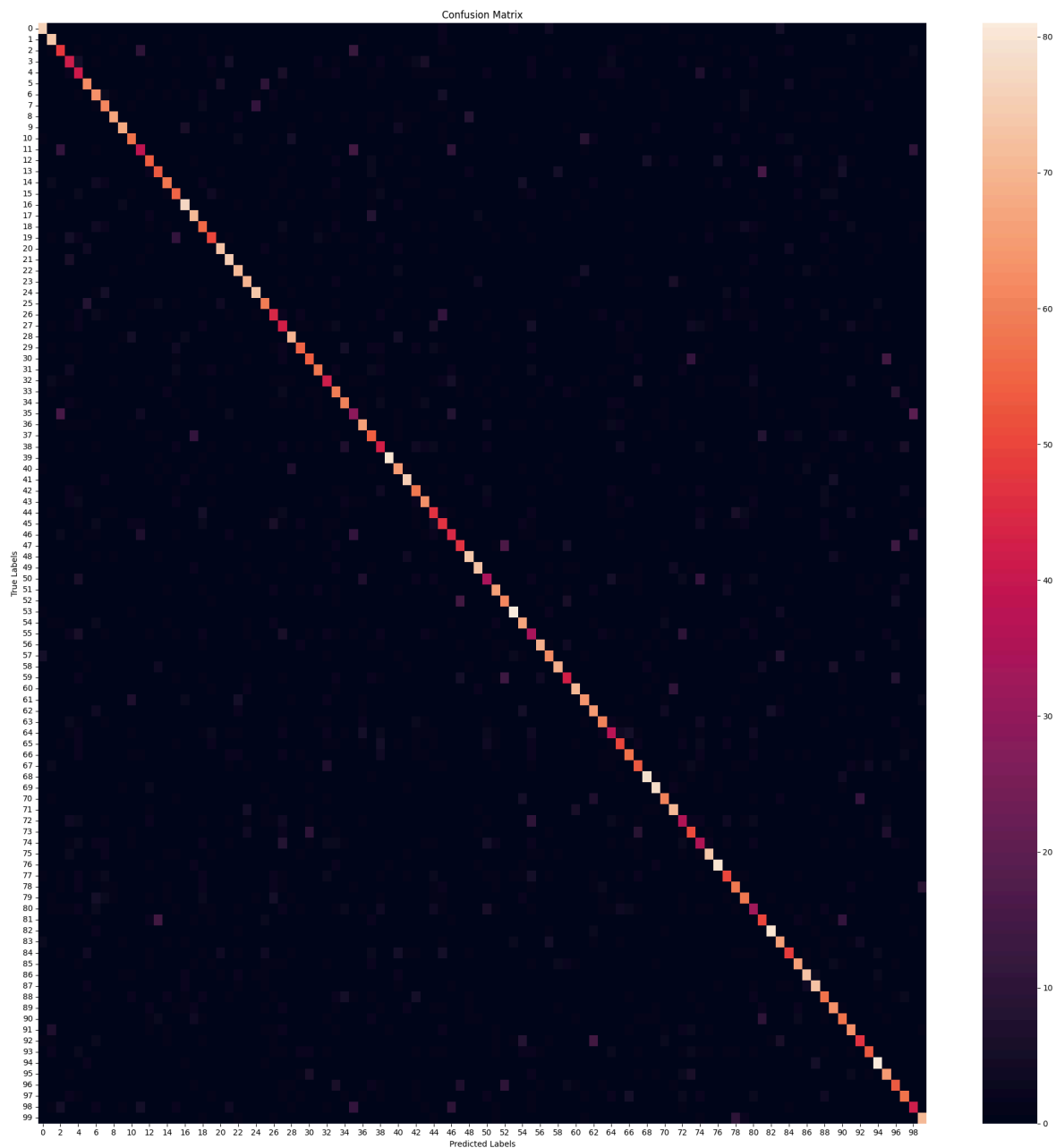
Test Precision: 80.14%

Test F1 Scores (Per Class): [84.91619 76.28865 52.747242 46.92737 41.62436 7 0.05649 62.376225

67.40331 80.722885 81.437126 65.89595 47.499996 62.427734 61.9883
69.87952 59.7826 80.628265 75.93583 58.333324 60.24096 80.66298
78.12499 76.756744 77.77777 81.767944 65.19337 51.764698 42.156857
76.92307 66.66666 63.157898 70.303024 49.999992 59.29648 67.0391
33.142853 71.73912 57.14285 46.99453 91.95402 68.42105 82.872925
62.365585 71.99999 51.933693 49.999992 49.438198 56.6265 80.43478
82.02247 40.96385 74.157295 63.541664 88.52458 68.71794 36.55913
81.6568 70.45454 80.23255 53.08641 80.681816 73.14285 68.06282
70.58823 44.57831 56.818176 64.04494 58.24175 87.29281 84.32432
69.36415 75.53191 41.176464 57.303364 38.251358 82.22221 86.95651
57.80347 65.53671 63.04347 36.871502 53.475933 89.385475 69.47368
58.33332 72.04301 80.21977 79.78141 61.375652 65.284966 61.290314
67.37968 56.969696 63.09523 86.63101 68.783066 54.92227 64.04494
46.408836 79.329605]

Average Test F1 Scores:65.70

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.85	0.84	0.85	90
dolphin	0.71	0.82	0.76	90
otter	0.52	0.53	0.53	90
seal	0.47	0.47	0.47	90
whale	0.38	0.46	0.42	90
aquarium fish	0.71	0.69	0.70	90
flatfish	0.56	0.70	0.62	90
ray	0.67	0.68	0.67	90
shark	0.88	0.74	0.81	90
trout	0.88	0.76	0.81	90
orchids	0.69	0.63	0.66	90
poppies	0.54	0.42	0.47	90
roses	0.65	0.60	0.62	90
sunflowers	0.65	0.59	0.62	90
tulips	0.76	0.64	0.70	90
bottles	0.59	0.61	0.60	90
bowls	0.76	0.86	0.81	90
cans	0.73	0.79	0.76	90
cups	0.55	0.62	0.58	90
plates	0.66	0.56	0.60	90
apples	0.80	0.81	0.81	90
mushrooms	0.74	0.83	0.78	90
oranges	0.75	0.79	0.77	90
pears	0.78	0.78	0.78	90
sweet peppers	0.81	0.82	0.82	90
clock	0.65	0.66	0.65	90
computer keyboard	0.54	0.49	0.51	90
lamp	0.38	0.48	0.42	90
telephone	0.76	0.78	0.77	90
television	0.73	0.61	0.67	90
bed	0.67	0.60	0.63	90
chair	0.77	0.64	0.70	90
couch	0.54	0.47	0.50	90
table	0.54	0.66	0.59	90
wardrobe	0.67	0.67	0.67	90
bee	0.34	0.32	0.33	90
beetle	0.70	0.73	0.72	90
butterfly	0.55	0.60	0.57	90
caterpillar	0.46	0.48	0.47	90
cockroach	0.95	0.89	0.92	90
bear	0.65	0.72	0.68	90
leopard	0.82	0.83	0.83	90
lion	0.60	0.64	0.62	90
tiger	0.74	0.70	0.72	90
wolf	0.52	0.52	0.52	90
bridge	0.48	0.52	0.50	90
castle	0.50	0.49	0.49	90
house	0.62	0.52	0.57	90
road	0.79	0.82	0.80	90
skyscraper	0.83	0.81	0.82	90
cloud	0.45	0.38	0.41	90
forest	0.75	0.73	0.74	90
mountain	0.60	0.68	0.64	90
plain	0.87	0.90	0.89	90
sea	0.64	0.74	0.69	90
camel	0.35	0.38	0.37	90
cattle	0.87	0.77	0.82	90
chimpanzee	0.72	0.69	0.70	90

elephant	0.84	0.77	0.80	90
kangaroo	0.60	0.48	0.53	90
fox	0.83	0.79	0.81	90
porcupine	0.75	0.71	0.73	90
possum	0.64	0.72	0.68	90
raccoon	0.75	0.67	0.71	90
skunk	0.49	0.41	0.45	90
crab	0.58	0.56	0.57	90
lobster	0.65	0.63	0.64	90
snail	0.58	0.59	0.58	90
spider	0.87	0.88	0.87	90
worm	0.82	0.87	0.84	90
baby	0.72	0.67	0.69	90
boy	0.72	0.79	0.76	90
girl	0.44	0.39	0.41	90
man	0.58	0.57	0.57	90
woman	0.38	0.39	0.38	90
crocodile	0.82	0.82	0.82	90
dinosaur	0.85	0.89	0.87	90
lizard	0.60	0.56	0.58	90
snake	0.67	0.64	0.66	90
turtle	0.62	0.64	0.63	90
hamster	0.37	0.37	0.37	90
mouse	0.52	0.56	0.53	90
rabbit	0.90	0.89	0.89	90
shrew	0.66	0.73	0.69	90
squirrel	0.63	0.54	0.58	90
maple	0.69	0.73	0.71	90
oak	0.79	0.81	0.80	90
palm	0.78	0.81	0.80	90
pine	0.58	0.63	0.61	90
willow	0.61	0.70	0.65	90
bicycle	0.59	0.63	0.61	90
bus	0.65	0.70	0.67	90
motorcycle	0.63	0.52	0.57	90
pickup truck	0.68	0.59	0.63	90
train	0.84	0.90	0.87	90
lawn-mower	0.66	0.72	0.69	90
rocket	0.51	0.59	0.55	90
streetcar	0.65	0.63	0.64	90
tank	0.46	0.47	0.46	90
tractor	0.80	0.79	0.79	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

In [6]: